

Top 200 Most-Asked

Java Interview Prep — 2026 Edition · Learn & Excel

If your interview is 2-3 weeks out, this is the file. Where the Top 50 gives you the headline questions, this file widens the surface to the 200 questions that account for the bulk of every Java round across Indian MNCs, product companies, GCCs, and startups. It is organized by **frequency tier**, not by topic — so you can spend most of your prep budget on the Tier S questions that show up in 80%+ of interviews, and treat Tier B as the polish layer.

How to use it: read Tier S first and don't move on until you can recite the 30-second answers cold. Tier A is where mid-level rounds live — work through it over a weekend. Tier B is depth-padding for senior rounds and the "tell me more" follow-ups; skim it the week of your interview.

Front matter: Frequency Heatmap

Topic / Tier	Service MNCs	Indian Product Co	Startup / GCC
OOP fundamentals	100%	95%	85%
Collections	95%	95%	85%
String / immutability	90%	80%	70%
Exception handling	85%	75%	70%
Concurrency basics	75%	95%	90%
Concurrency advanced	30%	85%	75%
Spring core	60%	90%	95%
Hibernate / JPA	55%	80%	70%
JVM internals	30%	75%	70%
Java 8 functional	70%	90%	80%
Java 17/21 modern	40%	80%	65%
Design patterns	40%	65%	50%
JDBC / SQL	60%	50%	45%
Reflection / annotations	25%	55%	50%
File I/O & serialization	30%	35%	30%

Use the heatmap to bias your prep: a TCS L1 round will hit OOP + Collections + Exceptions; a Razorpay L3 will skip those and grill you on Concurrency + JVM + Spring depth.

Tier S — Asked in 80%+ of Java interviews (50 Qs)

These are the same 50 questions from [01 Top 50 Most-Asked](#) . Reproduced here so this file stands alone. If you have already worked through Top 50, skim Tier S and move directly to Tier A.

Q1 · HashMap internals — what happens when you call put()?

Asked at: TCS · Infosys · Razorpay · Swiggy · almost every Java interview

Difficulty: Medium · **Topic:** Collections

The question: "Walk me through what happens internally when you call `put("key", "value")` on a `HashMap`."

The 30-second answer: Hash the key → bitwise-AND with `(capacity - 1)` to find a bucket → if empty, place entry; if not, walk the bucket's chain (linked list, or red-black tree if bucket has ≥ 8 nodes AND map size ≥ 64) → resize if `size > capacity × 0.75`.

Step-by-step thought process: 1. "First, `hashCode()` is called on the key, then a supplemental hash mixes the high bits down — this reduces collisions when capacity is small." 2. "Index is computed via `(capacity - 1) & hash` — works because capacity is always a power of 2." 3. "If the bucket is empty, entry goes in. Otherwise we walk the existing chain — comparing each entry's key via `equals()`." 4. "In Java 8+, when a single bucket has 8 or more nodes AND the table's capacity is ≥ 64 , that bucket is converted to a red-black tree — $O(\log n)$ lookups instead of $O(n)$." 5. "After insert, if `size > capacity × loadFactor` (default 0.75), the map resizes — capacity doubles, every entry rehashed."

Edge cases to mention: - Null keys are allowed and stored in bucket 0 - Two keys with the same `hashCode` end up in the same bucket but are distinguished via `equals()` — so a broken `equals()` / `hashCode()` contract causes silent overwrites or duplicates - Treeification requires BOTH conditions (bucket ≥ 8 AND table capacity ≥ 64) — when capacity is below 64, the map resizes instead of treeifying

What NOT to say: - "It's just a hashtable" — interviewer wants depth, not category labels - "It uses chaining" — true but stale; Java 8 added treeification, mention both - "Treeification happens at 8 entries" — wrong, it's 8 entries *in one bucket* + table capacity ≥ 64

Common follow-up: "What's different about `ConcurrentHashMap`'s `put()`?"

Q2 · String immutability — why is String immutable in Java?

Asked at: TCS · Infosys · Accenture · most fresher rounds

Difficulty: Easy · **Topic:** Strings

The question: "Why is `String` immutable in Java? Walk me through the design reasoning."

The 30-second answer: Five reasons — security (paths, URLs, class names can't be mutated after validation), thread-safety (no synchronization needed), hashCode caching (computed once, reused forever — critical for `HashMap` keys), String Pool (sharing requires immutability), and class loader integrity (class names passed to loaders can't change mid-load).

Step-by-step thought process: 1. "Security — String is used for filesystem paths, network URLs, classloader inputs. If mutable, validated-then-mutated attacks become trivial." 2. "Thread safety — multiple threads can share a String reference with zero synchronization, because the value can't change." 3. "HashMap performance — String's hashCode is cached on first call. If String were mutable, the cached hash would become stale, breaking every hash-based collection." 4. "String Pool — JVM interns string literals so identical strings share memory. Sharing only works if values can't be modified through one reference." 5. "Class loading — when `Class.forName("com.foo.Bar")` runs, the class name string flows through the loader. Mutability would let other code race to change it mid-load."

Edge cases to mention: - `String` is `final` so even subclassing can't break the contract - The `value` array is `private final` (Java 9+: `byte[]` instead of `char[]` for compact strings) - `StringBuilder/StringBuffer` give you mutability when you actually need it — use them in loops to avoid creating N intermediate strings

What NOT to say: - "Because Java just made it that way" — non-answer, downgrades you - "It's immutable for performance" — partial; performance is one of five reasons, not the main one

Common follow-up: "What's the difference between `==` and `.equals()` for Strings? When does the String Pool help vs hurt?"

Q3 · ArrayList vs LinkedList — which one when?

Asked at: Almost every Java interview, fresher to senior

Difficulty: Easy-Medium · **Topic:** Collections

The question: "When would you choose ArrayList over LinkedList, and when the reverse?"

The 30-second answer: ArrayList = backed by an array, $O(1)$ random access, $O(n)$ insert/delete in the middle. LinkedList = doubly-linked nodes, $O(n)$ random access, $O(1)$ insert/delete if you already have the node reference. Default to ArrayList unless you're doing heavy middle-of-list mutations with iterators.

Step-by-step thought process: 1. "ArrayList wraps an `Object[]`. Random access via index is one memory lookup — $O(1)$. Adding to the end is amortized $O(1)$; when capacity fills, the array is doubled." 2. "Insert or delete in the middle of an ArrayList requires shifting every subsequent element — $O(n)$." 3. "LinkedList is a doubly-linked list. Random access requires walking from head or tail to the index — $O(n)$." 4. "Insert/delete in LinkedList is $O(1)$ only if you have the node reference (e.g. via an iterator). If you have just an index, you still walk $O(n)$ to find the node." 5. "In practice, ArrayList wins almost always. CPU cache locality favors arrays. LinkedList's per-node memory overhead (prev + next pointers) hurts on large datasets."

Edge cases to mention: - LinkedList is also a Deque — useful if you need queue + stack semantics from one collection - ArrayList's `remove(int)` is $O(n)$; `remove(Object)` is also $O(n)$ because it does a linear scan first - For large insert-heavy workloads, `ArrayDeque` often outperforms both

What NOT to say: - "LinkedList is faster for inserts" — true only with cached node references; misleading as a blanket claim - "Just use Vector for thread safety" — Vector is legacy; use `Collections.synchronizedList()` or `CopyOnWriteArrayList` instead

Common follow-up: "What about CopyOnWriteArrayList? When does that make sense?"

Q4 · == vs equals() vs the hashCode contract

Asked at: Razorpay · Swiggy · most product cos · senior-to-fresher

Difficulty: Medium · **Topic:** OOP fundamentals

The question: "Explain the difference between `==` and `.equals()`. Then explain the hashCode contract."

The 30-second answer: `==` compares references (same object in memory?). `.equals()` compares logical equality (same content). The contract: if `a.equals(b)`, then `a.hashCode() == b.hashCode()` — always. The reverse is NOT required: two objects with the same hashCode can have `equals()` return false. Breaking this contract corrupts HashMap, HashSet, and every hash-based collection.

Step-by-step thought process: 1. "`==` on objects is identity equality — are these two references pointing to the exact same memory location?" 2. "`.equals()` defaults to `==` in Object, but most classes override it to compare logical content (Strings compare characters, Integers compare values)." 3. "The hashCode contract: equal objects (per equals) MUST return the same hashCode. The reverse is not required — collisions are allowed." 4. "If you override `equals()` without overriding `hashCode()`, your objects break HashMap. You'd add a key, then `get()` would return null because the lookup hash points to a different bucket than the stored hash." 5. "The standard implementation: derive hashCode from the same fields used in equals — usually `Objects.hash(field1, field2, ...)`."

Edge cases to mention: - `Objects.equals(a, b)` is null-safe — use it instead of `a.equals(b)` when either side might be null - Records (standardized Java 16, preview from Java 14) auto-generate equals/hashCode based on all components — one big reason to use records for data classes - For mutable fields used in equals/hashCode: never mutate after adding to a hash collection; it corrupts the collection

What NOT to say: - "`equals()` checks if two objects are the same" — vague, says nothing - "Override `equals()` but not `hashCode()` — Java will figure it out" — actively wrong, will be silently broken

Common follow-up: "Why must you use Objects.hash() instead of just adding hashCodes?"

Q5 · volatile vs synchronized vs atomic — when each?

Asked at: Razorpay · Cred · senior backend rounds

Difficulty: Medium-Hard · **Topic:** Concurrency

The question: "What's the difference between `volatile`, `synchronized`, and `AtomicInteger`? Give me a use case where each is the right pick."

The 30-second answer: `volatile` = visibility only (writes flush to main memory, reads from main memory), no atomicity. `synchronized` = mutual exclusion AND visibility, allows compound actions. `Atomic*` = lock-free atomic operations (CAS) for single variables — faster than `synchronized` for simple counters and flags.

Step-by-step thought process: 1. "`volatile` guarantees that a write to the variable is visible to all threads immediately — no thread-local caching. But it does NOT make compound operations atomic. `volatile int counter; counter++;` is still a race condition." 2. "`synchronized` blocks (or methods) acquire a monitor on an object. Only one thread executes inside the block at a time. Also has visibility guarantees — exiting a synchronized block flushes writes, entering one reads fresh values." 3. "`AtomicInteger`, `AtomicLong`, etc. use Compare-And-Swap (CAS) hardware instructions. Lock-free, much faster than `synchronized` for simple operations like `incrementAndGet()`." 4. "Use `volatile` for: status flags read by many threads, written rarely (like `shutdownRequested`)." 5. "Use `synchronized` for: compound operations on shared state (check-then-act, multi-field updates)." 6. "Use `Atomic` for: counters, flags with read-modify-write semantics, where contention is moderate."

Edge cases to mention: - `volatile` ensures happens-before relationship — useful for double-checked locking on singletons (`volatile` on the instance field) - `synchronized` on `this` and on a class's `class` object are different locks — easy to confuse - For very-high-contention counters, `LongAdder` outperforms `AtomicLong` (striped internal state)

What NOT to say: - "`volatile` makes operations atomic" — common mistake, wrong - "Just use `synchronized` everywhere, it's safer" — performance-killing advice, gets you marked as junior

Common follow-up: "What's the double-checked locking pattern, and why does it need `volatile`?"

Q6 · Checked vs unchecked exceptions — when to use which?

Asked at: Most Java interviews, fresher onwards

Difficulty: Easy-Medium · **Topic:** Exception handling

The question: "Difference between checked and unchecked exceptions? When would you create a checked exception vs an unchecked one in your own code?"

The 30-second answer: Checked extends `Exception` (not `RuntimeException`), must be declared or caught — compiler enforces. Unchecked extends `RuntimeException`, no enforcement. Use checked for recoverable conditions the caller MUST handle (e.g. file not found, network unavailable).

Use unchecked for programming errors and conditions the caller can't reasonably recover from (e.g. null argument, illegal state).

Step-by-step thought process: 1. "Java's exception hierarchy: Throwable → Error or Exception. Exception → RuntimeException (unchecked) or all others (checked)." 2. "Checked exceptions force the caller to handle or propagate at compile time — `throws` declaration required." 3. "Unchecked exceptions can be thrown anywhere with no declaration. RuntimeException, NullPointerException, IllegalArgumentException are all unchecked." 4. "When designing my own API: if the caller can REASONABLY recover (retry the network call, ask user for a different file path), make it checked. If it's a bug in the caller (passed null, called methods out of order), make it unchecked." 5. "In practice, modern Java (Spring, etc.) leans heavily toward unchecked — checked exceptions force boilerplate that often gets caught-and-ignored or wrapped meaninglessly. Spring even wraps JDBC's checked SQLExceptions into unchecked DataAccessException."

Edge cases to mention: - Errors (OutOfMemoryError, StackOverflowError) are not exceptions you should catch — they signal JVM-level problems - Lambda expressions can't throw checked exceptions unless the functional interface declares them — a common pain point with `Stream.map()` - Try-with-resources auto-closes `AutoCloseable` resources and propagates suppressed exceptions correctly

What NOT to say: - "Checked exceptions are bad, never use them" — opinionated and absolute; nuanced answer is better - "Just catch Exception everywhere" — anti-pattern, hides real bugs

Common follow-up: "Why doesn't Stream.map() let you throw checked exceptions easily?"

Q7 · JDK vs JRE vs JVM — what's the difference?

Asked at: TCS · Infosys · most fresher rounds

Difficulty: Easy · **Topic:** Java platform

The question: "Explain the difference between JDK, JRE, and JVM."

The 30-second answer: JVM = runtime engine that executes bytecode. JRE = JVM + standard libraries (everything needed to RUN Java apps). JDK = JRE + development tools like `javac`, `jdb`, `jar` (everything needed to BUILD Java apps).

Step-by-step thought process: 1. "JVM (Java Virtual Machine) is the abstract specification of a stack-based machine that executes Java bytecode. HotSpot, OpenJ9, GraalVM are concrete implementations." 2. "JRE (Java Runtime Environment) packages a JVM together with the standard Java class library — `java.lang`, `java.util`, `java.io`, etc." 3. "JDK (Java Development Kit) packages a JRE plus development tools: `javac` (compiler), `jar` (packager), `jdb` (debugger), `javadoc`, plus internal APIs for tools." 4. "Oracle stopped shipping a standalone JRE distribution starting with Java 11. You typically install a JDK and use `jlink` to build a slim runtime image if you need a JRE-equivalent." 5. "For a developer's mental model: write `.java` → compile with `javac` (JDK) → produces `.class` bytecode → executed by JVM (JRE)."

Edge cases to mention: - The JVM doesn't only run Java — Kotlin, Scala, Clojure, Groovy all compile to JVM bytecode - Multiple JVM implementations exist: HotSpot (Oracle), OpenJ9 (IBM/Eclipse), GraalVM (polyglot, ahead-of-time compilation) - Since Java 9, modules let you build a custom JRE containing only the modules your app uses — smaller runtime footprint

What NOT to say: - "JDK runs Java programs" — wrong; JVM runs them. JDK builds them. - "JRE is needed at runtime" — true historically, but Java 9+ blurs this with jlink

Common follow-up: "What does `javac` actually produce? What's bytecode?"

Q8 · final vs finally vs finalize — three different things

Asked at: TCS · Infosys · Cognizant · classic fresher question

Difficulty: Easy · **Topic:** Java language

The question: "What's the difference between `final`, `finally`, and `finalize`?"

The 30-second answer: `final` = keyword for immutability (final variable, final method, final class). `finally` = block that always runs after try/catch (for cleanup). `finalize()` = legacy Object method called by GC before reclaiming an object — deprecated since Java 9, deprecated-for-removal since Java 18 (still present in Java 21 but treat it as gone).

Step-by-step thought process: 1. "`final` has three uses: final variable = can't reassign after init, final method = can't override in subclass, final class = can't extend." 2. "`finally` is the try/catch/finally block — code inside finally runs whether the try block completes normally, throws, or returns early. Used for cleanup: closing resources, releasing locks." 3. "`finalize()` was a method on Object — the GC would call it before reclaiming an object, giving the object a chance to clean up. It was deprecated in Java 9 and marked deprecated-for-removal in Java 18 (JEP 421), and is still present in Java 21 — but treat it as gone." 4. "Modern equivalent of finalize: implement `AutoCloseable` and use try-with-resources, OR use `Cleaner` (Java 9+) which decouples cleanup logic from the object lifecycle."

Edge cases to mention: - `final` parameters can't be reassigned inside the method — useful for closures, prevents accidental mutation - `try-with-resources` is the preferred pattern over manual `finally { close(); }` — handles suppressed exceptions correctly - Even before Java 18, `finalize()` was unreliable — the GC made no guarantees about WHEN or IF it would be called

What NOT to say: - "Finalize is called when an object is destroyed" — vague and outdated; GC reclaims, doesn't "destroy" - Confusing them with one another — interviewer will downgrade you to fresher-tier on this question

Common follow-up: "How would you replace `finalize()` in modern Java?"

Q9 · Singleton pattern — 4 ways to implement it, which is thread-safe?

Asked at: Common at all levels

Difficulty: Medium · **Topic:** Design patterns + concurrency

The question: "Show me how to implement a Singleton in Java. Walk me through the trade-offs."

The 30-second answer: Four common ways — eager init (simple, thread-safe via class loading), lazy init with synchronized method (thread-safe but slow), double-checked locking with volatile (thread-safe and fast), enum (Bloch's preferred — handles serialization and reflection attacks). Enum is the cleanest for most cases.

Step-by-step thought process: 1. "Eager initialization: `private static final Singleton INSTANCE = new Singleton();` — thread-safe because class loading is thread-safe per the JLS. Downside: instantiated even if never used." 2. "Lazy with synchronized: method returns instance, creating it on first call. Thread-safe but every call pays synchronization cost." 3. "Double-checked locking: check instance == null without lock, lock only if null, check again inside lock. Faster than synchronized method but requires `volatile` on the instance field — otherwise the partially-constructed object can be visible to other threads." 4. "Enum singleton: `public enum Singleton { INSTANCE; }` — handles serialization automatically, prevents reflection attacks, lazy by default. Joshua Bloch's recommended approach in Effective Java." 5. "For most code, enum is the right pick. Spring's `@Service` / `@Component` (default scope=singleton) handles this for managed beans."

Edge cases to mention: - Without `volatile` in DCL, the JIT can reorder writes — another thread might see a non-null reference to a not-yet-initialized object - Reflection can bypass private constructors — only the enum approach is truly bulletproof - Serialization can create a new instance via deserialization — enum handles this; other approaches need `readResolve()`

What NOT to say: - "Just use synchronized on getInstance" — works but slow, and shows you don't know DCL - "Singleton is an anti-pattern, never use it" — opinionated; the interviewer wants to see you can implement it correctly

Common follow-up: "Why does double-checked locking specifically require volatile?"

Q10 · Garbage collection — what are the major collectors and when each?

Asked at: Mid-to-senior backend rounds

Difficulty: Medium-Hard · **Topic:** JVM internals

The question: "Walk me through the major garbage collectors in modern Java. When would you choose each?"

The 30-second answer: G1 (default since Java 9) — balanced throughput and pause times, good for most server apps. ZGC and Shenandoah — ultra-low pause (sub-millisecond), for latency-sensitive systems. Parallel GC — throughput-optimized, batch jobs. Serial GC — single-threaded, small heaps only.

Step-by-step thought process: 1. "All major GCs are generational — heap split into young generation (Eden + 2 survivor spaces) and old generation. Most allocations happen in Eden; most objects die young." 2. "G1 (Garbage First) — default since Java 9. Divides heap into regions, prioritizes collecting regions with the most garbage. Targets predictable pause times (`MaxGCPauseMillis`). Good default for most apps." 3. "ZGC — concurrent, low-latency. Pause times under 1ms even on huge heaps (TB-scale). Trade-off: slightly lower throughput. Production-ready since Java 15." 4. "Shenandoah — similar low-pause design, developed at Red Hat. Comparable to ZGC for most workloads." 5. "Parallel GC — older, throughput-optimized, stop-the-world. Good for batch processing where pauses don't matter but total time does." 6. "Serial GC — single-threaded. Only for small heaps and embedded systems."

Edge cases to mention: - G1 has a humongous object threshold — objects $\geq \frac{1}{2}$ region size allocate directly in old gen - ZGC supports colored pointers (uses 64-bit pointer metadata) — required for its concurrent design - Java 21 introduced generational ZGC as opt-in (non-generational remained default in 21); generational became default in Java 24 — improved throughput either way

What NOT to say: - "Java's GC handles everything automatically, don't worry about it" — true for hello-world, wrong for production - "CMS is the best low-latency option" — CMS was REMOVED in Java 14, this dates you badly

Common follow-up: "How do you tune G1 for a Spring Boot app with bursty traffic?"

Q11 · ConcurrentHashMap — how is it different from HashMap?

Asked at: Razorpay · Cred · senior backend rounds

Difficulty: Medium-Hard · **Topic:** Concurrency + Collections

The question: "How does ConcurrentHashMap differ from HashMap, and what changed between Java 7 and Java 8?"

The 30-second answer: HashMap is not thread-safe. ConcurrentHashMap is. Java 7 used segment-based locking (16 segments by default, each locked independently). Java 8 ditched segments — uses CAS for inserts on empty buckets, synchronizes on the first node of a bucket for collisions. Much higher concurrency in Java 8+.

Step-by-step thought process: 1. "HashMap is fast but unsafe for concurrent writes — you can get corrupted state, infinite loops on resize (classic Java 7 bug), or lost updates." 2. "ConcurrentHashMap (Java 7) divided the table into 16 segments, each with its own lock. Writers held only one segment's lock at a time, so up to 16 writers could proceed in parallel." 3. "Java 8 rewrote it: lock-free on empty buckets using `compareAndSwap`, fine-grained synchronization on the first node of a bucket when there's a collision. No more segments." 4. "Reads are completely lock-free in both versions — you can read while others write." 5. " `computeIfAbsent`, `merge`, `compute` are the atomic compound operations — use these instead of check-then-put."

Edge cases to mention: - ConcurrentHashMap forbids null keys and null values (HashMap allows both) — null would create ambiguity in get/contains return values - Iterators are weakly consistent — may or

may not reflect modifications made after iteration started, but won't throw

`ConcurrentModificationException` - For high-concurrency counts, prefer `mappingCount()` (returns `long`) over `size()` (returns `int`)

What NOT to say: - "Just use `Collections.synchronizedMap(new HashMap())`" — that wraps with one giant lock, much slower than `ConcurrentHashMap` - "`ConcurrentHashMap` is always faster than `HashMap`" — false; `HashMap` is faster single-threaded

Common follow-up: "*When would you use `computeIfAbsent` vs `putIfAbsent`?*"

Q12 · Fail-fast vs fail-safe iterators

Asked at: Common in Indian product company senior rounds

Difficulty: Medium · **Topic:** Collections + iterators

The question: "*What's the difference between fail-fast and fail-safe iterators? Give an example of each.*"

The 30-second answer: Fail-fast iterators throw `ConcurrentModificationException` if the underlying collection is modified during iteration (`ArrayList`, `HashMap` iterators). Fail-safe iterators work on a snapshot or use copy-on-write semantics, so concurrent modification is allowed (`ConcurrentHashMap`, `CopyOnWriteArrayList` iterators).

Step-by-step thought process: 1. "Most JCF collections (`ArrayList`, `HashMap`, `HashSet`) keep an internal `modCount` counter. The iterator captures `modCount` at creation; each `next()` checks it hasn't changed." 2. "If something modifies the collection (even via another iterator), `modCount` increments. The next `next()` call detects the mismatch and throws `ConcurrentModificationException`." 3. "This is fail-FAST — fails immediately to prevent silently inconsistent iteration." 4. "Fail-SAFE iterators (`ConcurrentHashMap`, `CopyOnWriteArrayList`, `ConcurrentLinkedQueue`) don't throw on concurrent modification. They either iterate over a snapshot (`CopyOnWriteArrayList`) or are weakly consistent (`ConcurrentHashMap`)." 5. "Important caveat: 'fail-safe' doesn't mean 'sees all updates' — you may miss modifications made during iteration."

Edge cases to mention: - `Iterator.remove()` is the only safe way to modify a fail-fast collection during iteration - Single-threaded code can hit `ConcurrentModificationException` too — if you call `list.remove(x)` directly inside an enhanced for-loop - `Collections.unmodifiableList` returns a wrapper — modification attempts throw immediately, not on next iteration

What NOT to say: - "Fail-safe is always better" — copy-on-write has heavy write cost; fail-fast is correct for single-threaded code - "`ConcurrentHashMap` iterators throw on modification" — wrong, they don't

Common follow-up: "*When would `CopyOnWriteArrayList` actually be the right choice?*"

Q13 · CopyOnWriteArrayList — when is it the right pick?

Asked at: Product cos · senior rounds

Difficulty: Medium · **Topic:** Concurrent collections

The question: "When would you actually use CopyOnWriteArrayList? What are the trade-offs?"

The 30-second answer: Use when reads vastly outnumber writes — listener lists, configuration data, observer patterns. Every write copies the entire underlying array ($O(n)$), but reads need zero synchronization. Iterators are guaranteed snapshot-consistent.

Step-by-step thought process: 1. "On every mutation (add, remove, set), CopyOnWriteArrayList creates a brand new copy of the underlying array with the change applied, then replaces the reference." 2. "Reads — including iteration — happen against whichever array the reference pointed to at read-time. Zero synchronization, no locks." 3. "Iterators capture the array reference at creation, so they iterate a consistent snapshot — modifications during iteration don't affect them and don't throw." 4. "Cost: $O(n)$ per write, plus garbage from the discarded array. Reads are $O(1)$ lock-free." 5. "Sweet spot: small to medium lists, many readers (e.g. event listeners, plugin registries), few writers."

Edge cases to mention: - `iterator().remove()` throws `UnsupportedOperationException` — iterators are snapshot, can't reach back to mutate - For maps, `CopyOnWriteArrayMap` doesn't exist; use `ConcurrentHashMap` instead - If your collection has more than a few thousand elements with even moderate writes, this is the wrong choice — the copy cost dominates

What NOT to say: - "It's just a thread-safe ArrayList" — true but hides the copy-on-write cost - "Use it for queues" — use `ConcurrentLinkedQueue` or `ArrayBlockingQueue` for queues

Common follow-up: "How does CopyOnWriteArraySet relate to this?"

Q14 · TreeMap and red-black trees

Asked at: Common in mid-level rounds

Difficulty: Medium · **Topic:** Sorted collections

The question: "What's the underlying data structure of TreeMap, and what guarantees does it give you over HashMap?"

The 30-second answer: TreeMap is a self-balancing red-black tree. Operations are $O(\log n)$ instead of HashMap's $O(1)$ average. In exchange: keys are kept in sorted order, and you get navigation methods (`floorKey`, `ceilingKey`, `headMap`, `tailMap`, `subMap`).

Step-by-step thought process: 1. "Red-black tree is a self-balancing binary search tree with five invariants — most importantly, no red node has a red parent, and every path from root to null has the same number of black nodes." 2. "These invariants ensure the tree stays approximately balanced — height is $O(\log n)$ — guaranteeing $O(\log n)$ for get/put/remove." 3. "Keys must be Comparable (or you supply a Comparator at construction). The tree uses comparison, not hash." 4. "Navigation methods are

TreeMap's killer feature: `firstKey`, `lastKey`, `floorKey(k)` (largest key $\leq k$), `ceilingKey(k)` (smallest key $\geq k$), `headMap(k)` (entries $< k$), etc." 5. "Use TreeMap when you need ordered iteration, range queries, or 'find closest key' operations. Use HashMap when you only need fast lookup and don't care about order."

Edge cases to mention: - TreeMap doesn't allow null keys (uses comparison; null can't be compared) but allows null values - `firstEntry`, `lastEntry`, `pollFirstEntry`, `pollLastEntry` are useful for priority-queue-like patterns - ConcurrentSkipListMap is the concurrent equivalent — also sorted, also $O(\log n)$, thread-safe

What NOT to say: - "TreeMap is faster than HashMap" — wrong, HashMap is faster for plain lookups - "TreeMap is a B-tree" — wrong, it's a red-black tree

Common follow-up: "How would you find all entries with keys between X and Y?"

Q15 · LinkedHashMap and LRU cache

Asked at: System design adjacent · mid-level rounds

Difficulty: Medium · **Topic:** Specialized collections

The question: "How would you implement an LRU cache in Java in 10 lines?"

The 30-second answer: Extend `LinkedHashMap` with `accessOrder=true` and override `removeEldestEntry()` to return true when size exceeds capacity. LinkedHashMap maintains a doubly-linked list of entries in access order; the eldest (least recently accessed) is at the head, ready to be evicted.

Step-by-step thought process: 1. "LinkedHashMap is HashMap + a doubly-linked list threading through all entries. Iteration order can be insertion order (default) or access order (constructor flag)." 2. "With `accessOrder=true`, every `get()` and `put()` moves the touched entry to the tail of the linked list. The head is always the least-recently-used." 3. "`removeEldestEntry(Map.Entry)` is called after every put. Default returns false. Override to return true when `size() > maxCapacity` — entry gets removed automatically." 4. "Result: a self-maintaining LRU cache with $O(1)$ get/put/evict." 5. "Code: `new LinkedHashMap<K, V>(capacity, 0.75f, true) { protected boolean removeEldestEntry(Map.Entry<K, V> eldest) { return size() > maxCapacity; } }`"

Edge cases to mention: - Not thread-safe — wrap with `Collections.synchronizedMap` if needed (or use Caffeine library for production) - For production use, Caffeine (W-TinyLFU eviction) is dramatically better than LRU for most access patterns - Guava's CacheBuilder is also an option but Caffeine has largely superseded it

What NOT to say: - "Use HashMap + a separate list" — works but you have to maintain both manually, error-prone - "Just use Redis" — out-of-scope, interviewer wants a Java answer

Common follow-up: "What if you needed time-based expiration in addition to LRU?"

Q16 · HashSet vs TreeSet vs LinkedHashSet

Asked at: Fresher to mid-level

Difficulty: Easy-Medium · **Topic:** Sets

The question: "Compare HashSet, TreeSet, and LinkedHashSet."

The 30-second answer: HashSet — hash-based, $O(1)$ ops, no order. TreeSet — red-black tree, $O(\log n)$ ops, sorted order. LinkedHashSet — HashSet + linked list, $O(1)$ ops, insertion order. Pick HashSet by default; TreeSet for sorted; LinkedHashSet when you want predictable iteration.

Step-by-step thought process: 1. "HashSet wraps a HashMap internally — your set elements are the keys, with a dummy constant value. $O(1)$ average for add/remove/contains." 2. "TreeSet wraps a TreeMap — same red-black tree, sorted order, $O(\log n)$ ops. Navigation methods (`first` , `last` , `floor` , `ceiling` , `headSet` , `tailSet`) available." 3. "LinkedHashSet wraps LinkedHashMap — $O(1)$ ops like HashSet, but iteration is in insertion order (or access order with the constructor flag)." 4. "Memory: LinkedHashSet has overhead per element (linked list pointers); TreeSet has tree-node overhead; HashSet is lightest." 5. "Default pick: HashSet. When you need sorted output: TreeSet. When you need stable iteration order (e.g. for testing, deterministic output): LinkedHashSet."

Edge cases to mention: - HashSet allows one null; TreeSet doesn't (uses comparison) - All three require correctly-implemented `equals()/hashCode()` (HashSet/LinkedHashSet) or `Comparable` / `Comparator` (TreeSet) - `EnumSet` is a specialized, bitmap-based set for enums — dramatically faster than HashSet for enum values

What NOT to say: - "TreeSet is faster because it's sorted" — wrong, $\log n > 1$ - "Use LinkedHashSet for thread safety" — none of these are thread-safe; use `ConcurrentSkipListSet` or `CopyOnWriteArraySet`

Common follow-up: "How do you make any of these thread-safe?"

Q17 · Comparable vs Comparator

Asked at: Common at all levels

Difficulty: Easy-Medium · **Topic:** Sorting

The question: "What's the difference between Comparable and Comparator? When would you use each?"

The 30-second answer: Comparable is implemented BY the class itself — defines the class's natural ordering via `compareTo()` . Comparator is a separate object — defines an alternative ordering, useful when you don't own the class or need multiple orderings.

Step-by-step thought process: 1. " `Comparable<T>` has one method: `int compareTo(T other)` . Returns negative if this < other, zero if equal, positive if this > other." 2. "You implement Comparable on a class to give it a natural ordering — Integer compares numerically,

String compares lexicographically, your `Employee` class might compare by ID." 3. "Comparator is external. Useful when: (a) you don't control the class, (b) you need multiple different orderings (e.g. employees by name vs by salary), (c) you want ad-hoc sorting without modifying the class." 4. "Java 8 made Comparators dramatically nicer:

`Comparator.comparing(Employee::getName).thenComparing(Employee::getSalary)` chains naturally." 5. "`reversed()`, `nullsFirst()`, `nullsLast()` are built-in modifiers."

Edge cases to mention: - The contract: `compare(a, b)` and `compare(b, a)` must have opposite signs; transitive ordering required - TreeSet and TreeMap silently misbehave if your Comparable is inconsistent with equals - Modern Java preference: define Comparable for natural ordering, use Comparator factory methods for everything else

What NOT to say: - "Comparable is for sorting, Comparator is for searching" — wrong, both are for ordering - Returning subtractions in `compareTo` (`return this.value - other.value`) — integer overflow risk; use `Integer.compare()` instead

Common follow-up: "How would you sort a List of Employees by department, then by salary descending?"

Q18 · Queue vs Deque vs Stack

Asked at: Common in mid-level rounds

Difficulty: Medium · **Topic:** Collections

The question: "When would you use a Deque vs a Queue vs a Stack in Java? Are Stack and Vector still relevant?"

The 30-second answer: `Stack` and `Vector` are legacy — synchronized, slow, avoid. For LIFO (stack semantics), use `Deque` (`ArrayDeque` recommended). For FIFO (queue), use `Queue` interface with `LinkedList` or `ArrayDeque`. For concurrent queues, use `BlockingQueue` implementations like `LinkedBlockingQueue` or `ArrayBlockingQueue`.

Step-by-step thought process: 1. "`Stack` extends `Vector` — both are from Java 1.0, both fully synchronized (slow), both deprecated in spirit if not officially." 2. "Modern preferred Stack: `Deque<T> stack = new ArrayDeque<>();` then use `push`, `pop`, `peek`." 3. "Modern preferred Queue: `Queue<T> q = new ArrayDeque<>();` or `new LinkedList<>();`. Use `offer`, `poll`, `peek`." 4. "For producer-consumer: `BlockingQueue` implementations — `ArrayBlockingQueue` (bounded), `LinkedBlockingQueue` (optionally bounded), `PriorityBlockingQueue`, `SynchronousQueue`. `put` / `take` block as needed." 5. "For lock-free concurrent queues: `ConcurrentLinkedQueue` (unbounded), `ConcurrentLinkedDeque`."

Edge cases to mention: - `ArrayDeque` is generally faster than `LinkedList` for both queue and stack ops (CPU cache locality) - `PriorityQueue` is a heap-backed priority queue — not FIFO, ordered by Comparator/Comparable - Java 21 added `SequencedCollection` interface — adds `getFirst`, `getLast`, `addFirst`, `addLast`, `reversed()` to ordered collections

What NOT to say: - "Use Stack class for LIFO" — legacy, marks you as not following modern Java -
 "LinkedList is the best Deque" — ArrayDeque outperforms it for almost all access patterns

Common follow-up: "How does BlockingQueue help in a producer-consumer setup?"

Q19 · Iterator vs ListIterator vs Spliterator

Asked at: Mid-level rounds

Difficulty: Medium · **Topic:** Iterators

The question: "What's a Spliterator, and how is it different from a regular Iterator?"

The 30-second answer: Iterator = sequential, forward-only, optional remove. ListIterator = bidirectional, can add/set/remove. Spliterator (Java 8+) = supports parallel processing via `trySplit()`, used internally by streams. It can describe characteristics of the underlying source (ORDERED, SIZED, IMMUTABLE, etc.) so the stream pipeline can optimize.

Step-by-step thought process: 1. "Iterator: `hasNext`, `next`, optional `remove`. One direction. Used by enhanced for-loop." 2. "ListIterator (lists only): adds `hasPrevious`, `previous`, `add`, `set`, `nextIndex`, `previousIndex`. Bidirectional, mutating." 3. "Spliterator: designed for parallel traversal. `tryAdvance(action)` processes one element. `trySplit()` returns a NEW Spliterator covering a portion of the source — the original now covers the rest." 4. "`characteristics()` returns a bitmask: ORDERED, SIZED, SUBSIZED, IMMUTABLE, CONCURRENT, NONNULL, DISTINCT, SORTED. Stream operations check these to pick optimal execution strategies." 5. "You rarely use Spliterator directly — `Collection.stream()` builds one for you. But if you write a custom collection, you might implement one to get good stream performance."

Edge cases to mention: - `Spliterator.estimateSize()` returns the estimated element count if SIZED; otherwise an estimate - Custom Spliterators are essential for IO-bound or generator-style streams (e.g. wrapping `BufferedReader.lines()`) - `Stream.iterate(seed, hasNext, next)` (Java 9+) is a friendlier way to build streams that previously needed custom Spliterators

What NOT to say: - "Spliterator is just a parallel iterator" — incomplete; the characteristics flags are the real win - "Always use parallel streams for performance" — overhead often exceeds benefits for small or non-CPU-bound work

Common follow-up: "When does `parallelStream()` actually outperform `stream()`?"

Q20 · Map.Entry, computeIfAbsent, and the Java 8 Map enhancements

Asked at: Mid-level + senior rounds

Difficulty: Medium · **Topic:** Collections + idiomatic Java

The question: "What new methods were added to Map in Java 8, and when would you use `computeIfAbsent` over `putIfAbsent`?"

The 30-second answer: Java 8 added `getOrDefault`, `putIfAbsent`, `compute`, `computeIfAbsent`, `computeIfPresent`, `merge`, `replace`, `replaceAll`, `forEach`. Use `computeIfAbsent` when you need to LAZILY compute the value (expensive call). Use `putIfAbsent` when you already have the value ready.

Step-by-step thought process: 1. "`getOrDefault(key, default)` — replaces the `map.containsKey(k) ? map.get(k) : default` pattern." 2. "`putIfAbsent(k, v)` — puts only if key absent. Returns existing value if present, null otherwise." 3. "`computeIfAbsent(k, key -> expensiveCompute(key))` — same idea but the value is computed by a function ONLY if the key is absent. Avoids wasting work." 4. "`compute(k, (k, oldV) -> newV)` — always invokes the function, even if oldV is null. Useful for counters: `map.compute(word, (k, v) -> v == null ? 1 : v + 1)`." 5. "`merge(k, value, BiFunction)` — if absent, put value; if present, apply function to (oldValue, value). Cleanest counter increment: `map.merge(word, 1, Integer::sum)`."

Edge cases to mention: - `computeIfAbsent` is atomic on `ConcurrentHashMap` — the function is invoked at most once per key - `compute` / `merge` returning null deletes the entry - `forEach((k, v) -> ...)` replaces the verbose `entrySet().iterator()` loop

What NOT to say: - "putIfAbsent and computeIfAbsent are the same" — wrong; eager vs lazy value construction is the key distinction - "Just use containsKey + get" — works but slow on `ConcurrentHashMap` (two lookups) and not atomic

Common follow-up: "How would you build a word-count Map with one line of Java?"

Q21 · ThreadLocal — what it is and the classic memory leak

Asked at: Razorpay · Cred · senior backend

Difficulty: Medium-Hard · **Topic:** Concurrency

The question: "What's ThreadLocal? Why does it cause memory leaks in thread pool environments?"

The 30-second answer: ThreadLocal stores a value per thread — each thread sees its own copy. Memory leak: in a thread pool, threads are reused. ThreadLocal values aren't cleared between tasks. Over time, ThreadLocal entries accumulate per thread, and the GC can't reclaim them because the thread still holds a reference.

Step-by-step thought process: 1. "`ThreadLocal<T>` is conceptually a map from `Thread` → `T`. Each thread has its own `ThreadLocalMap`, accessed via the `ThreadLocal` instance as the key." 2. "Use cases: per-request context (security principal, transaction ID) without passing it through every method signature; thread-confined caches; `SimpleDateFormat` instances (since `SimpleDateFormat` isn't thread-safe)." 3. "The leak: in a thread pool (Tomcat, `ExecutorService`), thread instances are reused for many tasks. If a

task sets a `ThreadLocal` value and doesn't remove it, the value sticks to that thread forever." 4. "Even worse — `ThreadLocalMap`'s entries use weak references for the KEY (the `ThreadLocal` instance) but strong for the VALUE. If the `ThreadLocal` goes out of scope, the key becomes weakly reachable, but the value is still strongly held." 5. "Fix: always pair `threadLocal.set(value)` with `threadLocal.remove()` in a finally block, OR use Java 21's `ScopedValue` for the same use case without the leak risk."

Edge cases to mention: - `InheritableThreadLocal` copies values to child threads at creation time — usually NOT what you want with thread pools - `ScopedValue` (Java 21) is the modern replacement — immutable, scoped to a `run` block, automatically cleared - Major frameworks (Spring, SLF4J MDC) have `ThreadLocal` cleanup as a common bug source

What NOT to say: - "ThreadLocal is just thread-safe storage" — true but misses the lifecycle/leak issue - "Just don't use ThreadLocal" — too absolute; it has valid uses if you're disciplined

Common follow-up: "How does Java 21's `ScopedValue` fix these issues?"

Q22 · Deadlock — how does it happen, how do you detect/avoid it?

Asked at: Common in mid-to-senior rounds

Difficulty: Medium-Hard · **Topic:** Concurrency

The question: "Walk me through a classic deadlock scenario and tell me how you'd prevent it."

The 30-second answer: Deadlock requires four conditions (Coffman's): mutual exclusion, hold-and-wait, no preemption, circular wait. Classic scenario: Thread A locks resource 1 then tries for 2; Thread B locks 2 then tries for 1. Avoidance: enforce a global lock ordering (always acquire resources in the same order), use `tryLock` with timeout, or use higher-level constructs (concurrent collections).

Step-by-step thought process: 1. "Coffman's four conditions — all four must hold for deadlock. Break any one and deadlock is impossible." 2. "Classic two-thread, two-lock deadlock: A holds L1 wants L2; B holds L2 wants L1. Both block forever." 3. "Detection: jstack on a running JVM shows BLOCKED threads with their wait targets. Java even logs 'Found one Java-level deadlock' when threads hold mutual locks." 4. "Prevention #1 — lock ordering: define a global order for all locks (e.g. by ID). Always acquire in ascending order. Breaks the circular-wait condition." 5. "Prevention #2 — `Lock.tryLock(timeout)` instead of unbounded acquire. If timeout fires, release everything you hold and retry." 6. "Prevention #3 — avoid holding locks across calls to unknown code (especially callbacks, lambdas). You don't know what locks the callee might try to acquire."

Edge cases to mention: - Livelock: threads aren't blocked but keep retrying and getting nowhere (e.g. two polite people stepping aside in a corridor) - Starvation: a thread never gets a chance to run because higher-priority threads dominate — not technically deadlock but adjacent problem - Synchronized blocks acquire a single monitor; you can't selectively unlock them — `ReentrantLock` gives more control

What NOT to say: - "Just don't use locks" — sometimes you must; advice is unhelpful - "Increase thread count to avoid deadlock" — actually makes it worse

Common follow-up: "How do you detect deadlocks in a production system without taking a thread dump?"

Q23 · CompletableFuture — chaining async work without callback hell

Asked at: Cred · Razorpay · Swiggy · backend rounds

Difficulty: Medium-Hard · **Topic:** Async + concurrency

The question: "Show me how you'd use CompletableFuture to call three services in parallel and combine the results."

The 30-second answer: `CompletableFuture.supplyAsync(call1)`, similar for call2 and call3, then `CompletableFuture.allOf(f1, f2, f3).thenApply(v -> combineResults(f1.join(), f2.join(), f3.join()))`. Use `thenCompose` to chain dependent async calls (flatMap), `thenCombine` to combine two independent ones.

Step-by-step thought process: 1. "`CompletableFuture.supplyAsync(supplier)` runs the supplier on the ForkJoinPool (or specified Executor) and returns a future." 2. "For parallel-then-combine: kick off all three with `supplyAsync`, get three futures. `allOf` returns a `CompletableFuture<Void>` that completes when all complete. After that, `.join()` on each gives the result." 3. "For chaining (B depends on A's result): `futureA.thenCompose(a -> callB(a))`. `thenCompose` is flatMap; `thenApply` is map." 4. "For combining two independent ones: `futureA.thenCombine(futureB, (a, b) -> ...)`." 5. "For fallback on error: `.exceptionally(throwable -> fallbackValue)` or `.handle((value, throwable) -> ...)`."

Edge cases to mention: - Default executor is the common ForkJoinPool — if your async work is IO-bound and blocks, you can starve the pool. Pass a dedicated Executor. - `.join()` is unchecked; `.get()` throws checked InterruptedException + ExecutionException - Java 21 virtual threads + structured concurrency (preview) is the new shiny — often a cleaner alternative

What NOT to say: - "Use Future.get() in a loop" — defeats the purpose; you're blocking - "Use RxJava" — out of scope unless asked; CompletableFuture is the question

Common follow-up: "How does this change with Java 21 virtual threads?"

Q24 · ExecutorService and the dangers of newFixedThreadPool

Asked at: Senior backend rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "What's wrong with `Executors.newFixedThreadPool(10)` in production?"

The 30-second answer: It uses an unbounded `LinkedBlockingQueue` for tasks. If submission rate > processing rate, the queue grows unboundedly — eventually OOM. Production code should construct `ThreadPoolExecutor` directly with a bounded queue and a sensible `RejectedExecutionHandler`.

Step-by-step thought process: 1. "`Executors.newFixedThreadPool(10)` is a convenience method. Under the hood: `new ThreadPoolExecutor(10, 10, 0, MILLISECONDS, new LinkedBlockingQueue<Runnable>())`." 2. "The `LinkedBlockingQueue` with no capacity argument is unbounded. If tasks arrive faster than 10 threads can process them, the queue fills with millions of tasks before anything fails — `OutOfMemoryError`, with confused stack traces." 3. "`newCachedThreadPool()` is worse — it creates unlimited threads, can OOM faster." 4. "Production: construct `ThreadPoolExecutor` directly. Pick a bounded queue (e.g. `new ArrayBlockingQueue<>(1000)`) and a `RejectedExecutionHandler` — `CallerRunsPolicy` (pushes back on submitter), `AbortPolicy` (throws), or your own that drops to a dead-letter queue." 5. "Pre-compute pool size: CPU-bound ~ `Runtime.getRuntime().availableProcessors()`; IO-bound ~ much higher; mixed = profile."

Edge cases to mention: - Spring's `@Async` defaults to a `SimpleAsyncTaskExecutor` (new thread per task!) unless configured. Always configure a `TaskExecutor` bean. - Java 21 virtual threads change the calculus — for IO-bound work, `Executors.newVirtualThreadPerTaskExecutor()` is now sensible - `ThreadFactory` lets you set thread names — critical for debugging stack dumps

What NOT to say: - "newFixedThreadPool is the standard choice" — popular but production-dangerous - "Just use more threads" — backpressure, not threads, is the answer

Common follow-up: "How does this change with Java 21 virtual threads?"

Q25 · synchronized — the difference between method-level and block-level

Asked at: Common at all levels

Difficulty: Medium · **Topic:** Synchronization

The question: "What's the difference between `synchronized` on a method versus a `synchronized` block?"

The 30-second answer: `synchronized` on an instance method = locks on `this`. `synchronized` on a static method = locks on the Class object. `synchronized(obj) { ... }` block = locks on `obj`. Block-level is preferred because it lets you (a) lock on a private dedicated object (no external code can lock it), (b) shrink the critical section to just the code that needs it.

Step-by-step thought process: 1. "`public synchronized void m() { ... }` is equivalent to `public void m() { synchronized(this) { ... } }`." 2. "`public static synchronized void m() { ... }` locks on `MyClass.class` — the Class object." 3. "`synchronized(this)` is

dangerous in libraries — external code can also acquire `this` as a lock, causing deadlocks you can't predict." 4. "Best practice: declare a private final Object: `private final Object lock = new Object();` and use `synchronized(lock) { ... }`. No one outside can lock it." 5. "Block scope lets you shrink the critical section to JUST the racy part. Method-level synchronization is often coarser than necessary, hurting throughput."

Edge cases to mention: - `synchronized` is reentrant — same thread can re-acquire the same lock without deadlocking itself - `ReentrantLock` offers `tryLock`, fairness, interruptible acquisition, and condition variables — capabilities `synchronized` doesn't have - Locking on a boxed Long/Integer/String is unsafe — the JVM may intern values, you might be locking on the same object as unrelated code

What NOT to say: - "synchronized is always slow, avoid it" — modern JVMs have biased locking and lightweight locking; uncontended synchronized is fast - "Lock on `Class.class` for static methods" — works but unusual; the implicit lock is the same Class object

Common follow-up: "When would you use `ReentrantLock` over `synchronized`?"

Q26 · Producer-consumer with BlockingQueue

Asked at: Common in design rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "Design a producer-consumer system in Java. How would you handle backpressure?"

The 30-second answer: Use a bounded `BlockingQueue` between producers and consumers. Producers call `put()` (blocks if full); consumers call `take()` (blocks if empty). The bounded queue provides natural backpressure — when full, producers slow down.

Step-by-step thought process: 1. "`BlockingQueue<Task>` interface; common impl: `ArrayBlockingQueue<>(capacity)` for bounded, `LinkedBlockingQueue` for optionally-bounded." 2. "Producer threads: `queue.put(task)` — blocks if the queue is full. Naturally throttles producers when consumers can't keep up. This is backpressure." 3. "Consumer threads: `queue.take()` — blocks if empty. No busy-wait, no polling." 4. "Bounded is critical — unbounded queues just defer the problem until OOM." 5. "For shutdown: poison-pill pattern (insert a sentinel object consumers check for and exit on), or use `ExecutorService` + `future` for clean shutdown."

Edge cases to mention: - `offer(task, timeout)` and `poll(timeout)` are the non-blocking variants with timeout — useful when you need to log "queue full, dropping" instead of blocking - For very-high throughput: Disruptor library (LMAX) outperforms `BlockingQueue` by orders of magnitude - `SynchronousQueue` has zero capacity — every put must directly match a take, useful for handoff patterns

What NOT to say: - "Use `wait()` and `notifyAll()`" — outdated; `BlockingQueue` is the modern idiom - "Use a List and synchronize on it" — reinventing the wheel badly

Common follow-up: "What if you needed different priority levels for tasks?"

Q27 · ReentrantLock vs synchronized — when to choose which?

Asked at: Senior rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "When would you use `ReentrantLock` instead of `synchronized`?"

The 30-second answer: When you need: (1) `tryLock` with timeout, (2) interruptible lock acquisition, (3) fairness policy, (4) multiple `Condition` variables (more flexible than one wait/notify), (5) lock metrics (`isHeldByCurrentThread`, `getHoldCount`). Otherwise, `synchronized` is simpler and equally fast.

Step-by-step thought process: 1. "`synchronized` is concise, automatic release on exception, and the JVM heavily optimizes it (biased locking, lightweight locking, lock coarsening). Default choice." 2. "Switch to `ReentrantLock` when you need a capability `synchronized` lacks:" 3. "`tryLock()` or `tryLock(timeout)` — attempt acquisition without blocking forever. Useful for deadlock avoidance." 4. "`lockInterruptibly()` — block but allow interrupt. `synchronized` blocks are not interruptible." 5. "`new ReentrantLock(true)` — fair lock; threads acquire in FIFO order. Slower but prevents starvation." 6. "Multiple `Condition` objects on one lock — replaces the limitation that `synchronized` has only one wait set per monitor. Classic example: producer-consumer with separate `notFull` and `notEmpty` conditions."

Edge cases to mention: - `ReentrantLock` requires manual `lock.unlock()` in a `finally` block — exception-unsafe if you forget - `ReentrantReadWriteLock` allows multiple concurrent readers OR one writer — good for read-heavy data structures - `StampedLock` (Java 8+) adds optimistic reads — faster than `ReentrantReadWriteLock` under heavy read load

What NOT to say: - "ReentrantLock is always faster than synchronized" — wrong; modern JVMs make `synchronized` very fast in uncontended cases - "synchronized is deprecated" — completely false

Common follow-up: "What's the optimistic-read pattern in `StampedLock`?"

Q28 · wait/notify vs Condition — what changed?

Asked at: Mid-level + senior rounds

Difficulty: Medium · **Topic:** Concurrency primitives

The question: "How does `Condition` (from `java.util.concurrent.locks`) improve over `wait/notify`?"

The 30-second answer: Object's `wait/notify` has ONE wait queue per object — you can't selectively wake the right kind of waiter. `Condition` (paired with `Lock`) lets you have MULTIPLE condition variables on

one lock — `notFull` and `notEmpty` for a bounded buffer, etc. Plus `await(timeout, unit)` is friendlier than `wait(millis)`.

Step-by-step thought process: 1. "Object.wait() / notify() / notifyAll() work in synchronized blocks. One wait set per object monitor." 2. "Classic problem: bounded buffer with producers waiting on 'not full' and consumers on 'not empty'. With one wait set, you must call notifyAll() and waiters check their condition — wakes up everyone, wasted work." 3. " `Lock` + multiple `Condition` : `final Condition notFull = lock.newCondition(); final Condition notEmpty = lock.newCondition();` . Producer calls `notFull.await()` — only waits on full-buffer. Consumer signals `notFull.signal()` after taking — only wakes producers." 4. "Cleaner semantics, less spurious wakeup, better throughput in contention." 5. "Also: `awaitNanos` , `awaitUntil(date)` , `awaitUninterruptibly()` give finer control than `wait()` ."

Edge cases to mention: - Always wait in a loop checking the condition predicate — spurious wakeups exist in both wait/notify and Condition - `signal()` wakes one waiter; `signalAll()` wakes all. Prefer signalAll if uncertain — correctness over performance - BlockingQueue / Semaphore wrap these primitives — usually you should use the higher-level construct

What NOT to say: - "wait/notify and Condition are the same" — they aren't; the multi-condition support is the key win - "Use wait without holding the lock" — runtime exception; both wait and await require holding their respective lock

Common follow-up: "Why do you always wait in a while-loop instead of an if?"

Q29 · Memory model — happens-before relationship

Asked at: Senior rounds at top product cos

Difficulty: Hard · **Topic:** JMM

The question: "What's the happens-before relationship in the Java Memory Model? Give an example."

The 30-second answer: Happens-before is a partial ordering on operations across threads. If A happens-before B, then B sees A's effects (writes visible, reads see the new value). Established by: synchronization (lock release happens-before subsequent acquire), volatile writes (happens-before subsequent volatile reads of the same variable), thread start, thread join, final field freezes.

Step-by-step thought process: 1. "Without happens-before, the JVM and CPU are free to reorder operations within a thread — your reads might see stale values, writes might be invisible to other threads." 2. "The JMM defines specific operations that establish happens-before: synchronized exit → synchronized entry on same monitor; volatile write → subsequent volatile read of same variable; Thread.start() → first action in started thread; final action in thread → Thread.join() in another thread." 3. "Practical example: double-checked locking requires `volatile` on the instance field. Without it: thread A's writes (object construction) might not happen-before thread B's read (the null check), and B might see the reference assigned before the object is fully constructed." 4. " `AtomicInteger.set` and `get` also establish happens-before — that's why Atomic classes are useful for inter-thread

communication beyond just numerical operations." 5. "If you have no happens-before relationship, you have no guarantee that one thread's writes are visible to another."

Edge cases to mention: - Final fields have special "freeze" semantics — guaranteed visible to other threads after the constructor returns - Static initializers establish happens-before for all reads of static fields — useful for safe singletons via inner-class idiom - Volatile is transitive in interesting ways — see "JSR-133 cookbook" for the formal model

What NOT to say: - "Threads share memory, so they see each other's writes" — wrong, that's the whole problem JMM solves - "Volatile makes everything thread-safe" — only for visibility, not for compound operations

Common follow-up: *"How does this connect to the double-checked locking pattern?"*

Q30 · ForkJoinPool and work-stealing

Asked at: Senior backend + parallelism-heavy rounds

Difficulty: Medium-Hard · **Topic:** Parallelism

The question: *"What's ForkJoinPool, and how does work-stealing work?"*

The 30-second answer: ForkJoinPool is a thread pool optimized for divide-and-conquer recursion. Each worker thread has its own deque of tasks. When a worker's deque is empty, it STEALS tasks from another worker's deque. Used internally by parallel streams and CompletableFuture by default.

Step-by-step thought process: 1. *"Designed for tasks that can be split into smaller tasks (`RecursiveTask` / `RecursiveAction`). Submit a big task, it forks subtasks, the pool processes them in parallel, you join the results."* 2. *"Each worker has a local deque (double-ended queue). The worker pushes/pops from one end (LIFO — good cache locality for the just-created subtask)."* 3. *"When a worker's deque is empty, it tries to steal from another worker's deque — but from the OTHER end (FIFO). This minimizes contention with the owning worker."* 4. *"Result: idle threads pick up work automatically; no central queue contention; good cache locality for the busy path."* 5. *"Java's `ForkJoinPool.commonPool()` is shared by parallel streams, CompletableFuture, and any code that uses it directly. Be careful about blocking IO in pool threads — it starves the pool."*

Edge cases to mention: - For blocking IO, use `ManagedBlocker` to signal the pool to spawn a replacement thread - Java 21 virtual threads largely replace the need for ForkJoinPool for IO-bound work - Parallel stream performance depends heavily on the Spliterator's split quality — for poorly-splitting sources, parallel is slower

What NOT to say: - "Work-stealing means stealing tasks from the main thread" — wrong, workers steal from each other - "Always use `parallelStream()` for performance" — pool starvation, splitting overhead, and contention often make sequential faster

Common follow-up: *"Why might `parallelStream()` actually be slower than `stream()`?"*

Q31 · Stream API — lazy evaluation explained

Asked at: Common at all levels

Difficulty: Medium · **Topic:** Streams

The question: "Explain lazy evaluation in Java streams. What's the difference between intermediate and terminal operations?"

The 30-second answer: Intermediate operations (`map` , `filter` , `sorted` , etc.) are LAZY — they build up a pipeline but don't execute. Terminal operations (`collect` , `forEach` , `reduce` , `count` , etc.) TRIGGER execution and consume the stream. Until the terminal op runs, nothing in the chain executes — useful for `findFirst` over an infinite stream, etc.

Step-by-step thought process: 1. " `Stream` is a lazy pipeline of operations. `stream.filter(p).map(f)` doesn't iterate the source — it returns a new `Stream` object describing the intended pipeline." 2. "Terminal operations (`collect` , `count` , `forEach` , `findFirst` , `reduce` , `toArray`) consume the stream and trigger execution. After a terminal op, the stream is closed — you can't use it again." 3. "Laziness enables short-circuiting: `findFirst()` after `filter()` stops as soon as one match is found, even on an infinite stream from `Stream.iterate()` or `Stream.generate()` ." 4. "Laziness also enables fusion: the runtime can combine adjacent map+filter operations into one pass over the source, instead of N intermediate collections." 5. "Caveat: `peek()` is sometimes elided if the runtime determines its result is unused — don't rely on `peek` for side effects you need. "

Edge cases to mention: - A `Stream` can only be consumed once — `s.count()` then `s.collect()` throws `IllegalStateException` - Sort is partially lazy — it's intermediate, but to know the order it must consume the entire upstream before emitting - Parallel streams change execution strategy but the laziness contract is the same

What NOT to say: - "Stream is just a fancy iterator" — misses the lazy + parallel + functional model - "Always materialize to a list with `.collect()` " — fine for finite, fast streams; wrong for infinite or huge ones

Common follow-up: "What's the difference between `map` and `flatMap` ?"

Q32 · Parallel streams — when do they actually help?

Asked at: Senior rounds, often as a trap question

Difficulty: Medium-Hard · **Topic:** Parallelism

The question: "When does `.parallelStream()` actually perform better than `.stream()` , and when is it a trap?"

The 30-second answer: Helps when: (1) CPU-bound work per element is non-trivial, (2) source splits well (ArrayList, ranges — not LinkedList or generated streams), (3) you have many elements (rough rule: at least thousands), (4) you're using the commonPool appropriately. Trap when: tasks are IO-bound, source is hard to split, or the work per element is tiny.

Step-by-step thought process: 1. "`parallelStream()` uses `ForkJoinPool.commonPool()` — by default sized to `Runtime.availableProcessors() - 1` (one worker fewer than logical processors, leaving room for the submitting thread)." 2. "For CPU-bound work, parallelism speedup is roughly bounded by core count, minus overhead." 3. "The source matters: ArrayList and `IntStream.range` split cleanly (random access, known size). LinkedList must walk to split — defeats the purpose. `Stream.iterate` is sequential by design." 4. "Per-element work cost matters: simple map operations are dominated by stream overhead, often slower than sequential. Heavy work per element wins." 5. "Element count matters: under ~1000 elements, sequential is usually faster because parallelization setup costs more than the savings." 6. "Trap: IO-bound work in parallel streams clogs the common ForkJoinPool, slowing every other parallel operation in the JVM."

Edge cases to mention: - `forEachOrdered` preserves encounter order but kills parallelism — use `forEach` if order doesn't matter - Custom parallel executor: there's no clean API, but submitting via `ForkJoinPool.submit(() -> stream.parallel().count())` runs in your pool - Java 21 virtual threads + sequential streams is often the better answer for IO-bound work

What NOT to say: - "Parallel is always faster" — common mistake, often false - "Use parallel streams for IO-bound work" — actively bad advice

Common follow-up: "How would you parallelize IO-bound work in Java 21?"

Q33 · Optional — what NOT to do with it

Asked at: Code review style questions, all levels

Difficulty: Medium · **Topic:** Idiomatic Java

The question: "Show me the right and wrong ways to use Optional in Java."

The 30-second answer: Optional was designed for return types where absence is meaningful (e.g. `findById()`). DON'T use it for: fields, parameters, collection types, or as a generic null-replacement. DON'T call `.get()` without checking presence. DO use `.map`, `.flatMap`, `.orElse`, `.ifPresent` for chaining.

Step-by-step thought process: 1. "Optional is a return-type contract — 'this method may not return a value, and you must handle that.' Designed for stream terminals (`findFirst`, `findAny`, `reduce` with no identity) and 'lookup by ID' patterns." 2. "Anti-pattern #1: `Optional` as field. Adds an extra object, doesn't serialize cleanly, and forces every read to unwrap." 3. "Anti-pattern #2: `Optional` as method parameter. Caller can pass `Optional.empty()` OR null OR a real value — three states instead of two. Just overload or use null." 4. "Anti-pattern #3: `optional.get()` without `isPresent()` check — same `NullPointerException`-equivalent, but more verbose." 5. "Idiomatic

chaining: `findUser(id).map(User::getEmail).orElse("unknown@example.com")` . 6.
 " `ifPresent` for side effects, `orElseGet(supplier)` if the default is expensive to compute."

Edge cases to mention: - `Optional.of(value)` throws if value is null;
`Optional.ofNullable(value)` doesn't — pick deliberately - `Optional<List<T>>` is almost always wrong — return empty list instead - Java 10 added `orElseThrow()` (no arg) — throws `NoSuchElementException`, useful when absence is "impossible"

What NOT to say: - "Use Optional everywhere to avoid NullPointerExceptions" — adds boilerplate without solving root cause - "Optional is faster than null checking" — has allocation cost, marginally slower

Common follow-up: "What's `orElseGet` vs `orElse` ?"

Q34 · Method references vs lambdas

Asked at: Common at all levels

Difficulty: Easy-Medium · **Topic:** Functional Java

The question: "When would you use a method reference instead of a lambda?"

The 30-second answer: When the lambda just delegates to an existing method. `s -> s.length()` becomes `String::length`. `(a, b) -> Integer.compare(a, b)` becomes `Integer::compare`. Cleaner to read, no extra closure, sometimes more efficient. Four kinds: static, bound instance, unbound instance, constructor.

Step-by-step thought process: 1. "`Type::staticMethod` — `Integer::parseInt`. Equivalent to `s -> Integer.parseInt(s)` ." 2. "`instance::instanceMethod` — `out::println`. The instance is captured." 3. "`Type::instanceMethod` — `String::length`. The instance becomes the first parameter — `s -> s.length()` ." 4. "`Type::new` — constructor reference. `ArrayList::new` is `() -> new ArrayList()` ." 5. "Use them when the lambda body is just a method call — primarily a readability win. Bytecode-wise, both lambdas and method references compile to `invokedynamic` via `LambdaMetafactory`, so there's no consistent performance difference."

Edge cases to mention: - `this::method` is fine in OO code — captures `this` implicitly, equivalent to `arg -> this.method(arg)` - Ambiguous method references (overloaded methods, multiple matches) need a cast to clarify - A method reference is just sugar — same Functional Interface invariants apply

What NOT to say: - "Method references are faster than lambdas" — sometimes true, but trivial in practice; don't pick on perf grounds - "Use method references everywhere" — sometimes the lambda body adds clarity (renaming variables for readability)

Common follow-up: "What's a `Functional Interface` ?"

Q35 · `forEach` vs `for-each` loop — which when?

Asked at: Code review, mid-level

Difficulty: Easy-Medium · **Topic:** Idiomatic Java

The question: "When would you use `Collection.forEach()` vs the enhanced `for-loop`? Are they interchangeable?"

The 30-second answer: Functionally similar for simple iteration. Differences: (1) `forEach` can't break early or use `continue`. (2) Enhanced `for-loop` can mutate the collection only via `Iterator.remove`; `forEach` shouldn't mutate. (3) `forEach` reads better in chains; enhanced `for` is more flexible for complex logic.

Step-by-step thought process: 1. "`for (T item : collection) { ... }` is the enhanced `for-loop` — desugars to an `Iterator`-based loop. Supports `break`, `continue`, `return`." 2. "`collection.forEach(item -> { ... })` is the lambda-based `forEach`. Cannot break — `return` inside the lambda only exits the current iteration (like `continue`)." 3. "For chained stream operations, `stream().filter().map().forEach()` reads cleanly. For standalone iteration with control flow, enhanced `for` is more flexible." 4. "`forEach` doesn't guarantee order on parallel streams — use `forEachOrdered` if you need it." 5. "Concurrent modification: enhanced `for-loop` with `Iterator.remove()` is the safe way to remove during iteration. `forEach` followed by `collection.removeAll(toRemove)` is the lambda equivalent."

Edge cases to mention: - Map's `forEach((k, v) -> ...)` replaces `for (Map.Entry<K,V> e : map.entrySet()) { ... }` — usually cleaner - `Collection.removeIf(predicate)` is the cleanest "remove while iterating" pattern - `forEach` is part of `Iterable`, so any `Iterable` (not just `Collection`) supports it

What NOT to say: - "forEach is faster" — usually the same; for tiny operations, enhanced-`for` can be slightly faster (no lambda boxing) - "Always use `forEach` in modern Java" — sometimes worse readability

Common follow-up: "How would you remove items from a list while iterating?"

Q36 · Collectors — beyond `toList` and `toMap`

Asked at: Common in idiomatic-Java rounds

Difficulty: Medium · **Topic:** Streams

The question: "What Collectors do you reach for beyond `toList()` and `toMap()`?"

The 30-second answer: `groupingBy` for partitioning by key, `partitioningBy` for boolean split, `joining` for string concatenation, `counting` and `summingInt` / `averagingInt` for aggregations. Chain these with downstream collectors for sophisticated reductions in one pass.

Step-by-step thought process: 1. "`groupingBy(classifier)` → `Map<K, List<T>>`. Group employees by department:

`employees.stream().collect(groupingBy(Employee::getDept))` ." 2. "`groupingBy(classifier, downstream)` chains another collector. "`groupingBy(Employee::getDept, counting())` → `Map<String, Long>` of dept → headcount." 3. "`partitioningBy(predicate)` → `Map<Boolean, List<T>>` . Split adults from minors in one pass." 4. "`joining(delimiter, prefix, suffix)` → `String` . Concatenate names with commas: `joining(", ", "[", ""])` ." 5. "`teeing(c1, c2, merger)` (Java 12+) feeds the stream to two collectors and combines. E.g. compute min and max in one pass."

Edge cases to mention: - `toMap` throws on duplicate keys unless you supply a merge function: `toMap(K, V, (existing, replacement) -> existing)` - `groupingBy` uses a `HashMap` by default; use the 3-arg version with `TreeMap::new` for sorted output - `Collectors.collectingAndThen(downstream, finisher)` post-processes the result — e.g. `collectingAndThen(toList(), Collections::unmodifiableList)` returns an immutable list

What NOT to say: - "Just collect to List and process after" — slower, loses one-pass benefit - "Stream collectors are complicated, just use loops" — defeats the point of streams

Common follow-up: "How would you compute the average salary per department in one pass?"

Q37 · Lambdas vs anonymous inner classes

Asked at: Fresher to mid-level

Difficulty: Easy-Medium · **Topic:** Lambdas

The question: "What's the difference between a lambda and an anonymous inner class?"

The 30-second answer: Both implement a functional interface. Lambda is sugar over `invokedynamic` (no `.class` file generated), inherits the enclosing `this`, can only access effectively-final variables. Anonymous inner class generates an extra class, creates a new `this` (and a synthetic reference to the enclosing instance), can access effectively-final variables only too.

Step-by-step thought process: 1. "Lambda: `Runnable r = () -> doSomething();` — no `.class` file, generated at runtime via `invokedynamic`. `this` inside the lambda refers to the enclosing class instance." 2. "Anonymous inner class: `Runnable r = new Runnable() { public void run() { doSomething(); } };` — generates an `EnclosingClass$1.class`, has its own `this` (the inner instance), holds an implicit reference to the enclosing instance via `EnclosingClass.this` ." 3. "Memory: lambdas without captured state can be a single shared instance (constant). Anonymous classes allocate fresh per invocation." 4. "Variable capture: both can only capture effectively-final local variables — i.e. variables you don't reassign after the lambda/inner-class is defined." 5. "Use lambdas for short functional bodies. Use anonymous inner classes when you need multiple methods, or when you need a fresh `this` (rare)."

Edge cases to mention: - Anonymous classes can extend abstract classes; lambdas can only implement single-abstract-method interfaces - Lambdas don't introduce a new scope — variable

shadowing rules are different - Memory footprint: a no-capture lambda is essentially zero allocation per call (cached); an anonymous inner class is at least one allocation

What NOT to say: - "They're the same thing" — close, but `this` semantics differ - "Lambdas are syntactic sugar for inner classes" — close, but `invokedynamic` doesn't generate an inner class

Common follow-up: "What does 'effectively final' mean?"

Q38 · Functional interface — what makes one?

Asked at: Fresher to mid-level

Difficulty: Easy-Medium · **Topic:** Lambdas

The question: "What's a functional interface? Can you list the most-used ones in `java.util.function`?"

The 30-second answer: A functional interface has exactly ONE abstract method (SAM — Single Abstract Method). Lambdas can be assigned to any functional interface. `@FunctionalInterface` annotation makes the compiler enforce this. Most-used: `Function<T,R>`, `Predicate<T>`, `Consumer<T>`, `Supplier<T>`, `BiFunction<T,U,R>`.

Step-by-step thought process: 1. "A functional interface has exactly one abstract method. Default and static methods don't count." 2. "`@FunctionalInterface` is optional but recommended — the compiler errors if you accidentally add a second abstract method." 3. "Standard library has `Runnable`, `Callable`, `Comparator` (all SAM) plus the `java.util.function` package: `Function`, `Predicate`, `Consumer`, `Supplier`, plus primitive specializations (`IntFunction`, `LongPredicate`, etc.) to avoid boxing." 4. "You can compose them: `Predicate.and()`, `Predicate.or()`, `Function.andThen()`, `Function.compose()`." 5. "Custom functional interfaces are useful when standard ones don't fit — e.g. a `TriFunction<A, B, C, R>` doesn't exist in the JDK."

Edge cases to mention: - Interfaces with one abstract method + multiple defaults are still functional interfaces (e.g. `Comparator`) - Two interfaces with the same method signature are NOT compatible for lambda assignment — you must pick a concrete one - Method references give you the same functional-interface flexibility as lambdas

What NOT to say: - "Functional interface means it works with streams" — sort of, but the definition is just SAM - Listing 12 functional interfaces when asked to name "a few" — bloat, picks the bad ones

Common follow-up: "What's the difference between `Function` and `Consumer`?"

Q39 · default methods — why were they added?

Asked at: Mid-level rounds

Difficulty: Easy-Medium · **Topic:** Java 8 features

The question: "Why were default methods added to interfaces in Java 8? What problem did they solve?"

The 30-second answer: To allow adding methods to existing interfaces without breaking existing implementations. The driver was Stream API: Collection had to gain `stream()`, `parallelStream()`, `forEach()`, `removeIf()` — without `default`, every existing Collection implementation would have had to add these.

Step-by-step thought process: 1. "Pre-Java-8: adding a new abstract method to an interface broke every existing implementation. This blocked Java from evolving its core APIs." 2. "Java 8 added `default` methods — implemented in the interface itself. New implementers don't have to override; existing implementers continue to work." 3. "Used heavily by Java 8 to add Stream support to Collection (`stream()`, `parallelStream()`, `forEach()`), to add `compute*` / `merge` to Map, etc." 4. "Allowed multiple inheritance of behavior (not state) — you can implement two interfaces with default methods that don't conflict. Conflicts must be resolved explicitly via override." 5. "Caveat: default methods on functional interfaces don't change SAM-ness — you can still lambda-assign them."

Edge cases to mention: - Diamond problem: if two interfaces have conflicting defaults, the implementing class MUST override and pick (`InterfaceA.super.method()` or `InterfaceB.super.method()`) - A class's concrete method always wins over interface defaults (no diamond conflict needed) - Static methods on interfaces are also new in Java 8 — bound to the interface, not inherited

What NOT to say: - "Default methods make Java multi-inheritance" — partial truth; state isn't inherited, only behavior - "Use default methods instead of abstract classes" — abstract classes still have their place (state, constructors)

Common follow-up: "What happens if a class implements two interfaces with the same default method?"

Q40 · CompletableFuture exceptionally vs handle

Asked at: Senior backend rounds

Difficulty: Medium · **Topic:** Async error handling

The question: "Difference between `exceptionally`, `handle`, and `whenComplete` on `CompletableFuture`?"

The 30-second answer: `exceptionally(throwable -> fallback)` runs ONLY on exception, recovers with a value. `handle((value, throwable) -> result)` runs on BOTH success and failure, lets you transform either. `whenComplete((value, throwable) -> sideEffect)` runs on both but doesn't transform — for side effects like logging.

Step-by-step thought process: 1. "`exceptionally` : takes a Function from Throwable to T. Invoked only if the future completed exceptionally. Result is a new future with the recovered value." 2. "`handle` : takes a BiFunction of (T, Throwable). Invoked on either outcome — exactly one of value/throwable will be

null. Result is a new future with the transformed value." 3. "`whenComplete` : takes a BiConsumer of (T, Throwable). Invoked on either outcome. Result is a future with the SAME value as the original (or rethrows the original exception). For side effects: logging, metrics." 4. "Use exceptionally when you have a sensible fallback. Use handle when the success and failure paths produce the same type but via different logic. Use whenComplete for observation without transformation." 5. "Chain order matters: `future.thenApply(f).exceptionally(g)` — g handles exceptions from BOTH the original future AND f."

Edge cases to mention: - `exceptionallyCompose` (Java 12+) is the async equivalent of `exceptionally` — recovery via another async call - `whenComplete` rethrows the original exception even if the consumer throws — be careful logging shouldn't mask original errors - Java 12+ `exceptionallyAsync`, `handleAsync` etc. for executor-control versions

What NOT to say: - "They're all the same, just pick one" — they aren't - "Use try-catch on get()" — works but blocks; defeats async

Common follow-up: "How would you log an async failure without changing the eventual value?"

Q41 · Records — what they are and when to use them

Asked at: Razorpay · Cred · Swiggy · senior + mid 2024+

Difficulty: Easy-Medium · **Topic:** Java 16+ features

The question: "What are records in Java 16+? When would you use one instead of a regular class?"

The 30-second answer: Records are compact syntax for immutable data classes. `public record Point(int x, int y) {}` auto-generates: constructor, accessors `x()` / `y()`, `equals()`, `hashCode()`, `toString()`. Use them for DTOs, value objects, multi-return values — anywhere you'd otherwise write a class with only getters.

Step-by-step thought process: 1. "`public record User(String name, String email, int age) {}` — one line. Equivalent to a class with: final fields, canonical constructor, accessors (no `get` prefix — just `name()`), `equals` based on all components, `hashCode` based on all components, `toString` listing all components." 2. "Records are implicitly `final`. Cannot extend other classes. Can implement interfaces." 3. "Components are immutable — no setters generated. You CAN add validation in a compact constructor: `public User { if (age < 0) throw new IllegalArgumentException(); }`" 4. "Use cases: REST API request/response DTOs, multi-return values from methods (instead of `Pair` / `Tuple`), map keys (auto-correct `equals`/`hashCode`), event objects." 5. "NOT for: mutable entities (e.g. JPA `@Entity` often needs mutability), classes that need inheritance hierarchies."

Edge cases to mention: - You can override the auto-generated accessor or `toString` / `equals` / `hashCode` if needed — useful for defensive copying of mutable component types - Records can have static factory methods and additional non-component methods - Pattern matching for records (Java 21) lets you destructure: `if (obj instanceof User(var name, var email, var age))`

What NOT to say: - "Records replace Lombok @Data" — only for immutable cases; Lombok still wins for mutable entities - "Records can have mutable fields" — wrong, components are always final

Common follow-up: "How does record pattern matching work in Java 21?"

Q42 · Sealed classes — what problem do they solve?

Asked at: Senior rounds at product cos

Difficulty: Medium · **Topic:** Java 17 features

The question: "What are sealed classes (Java 17), and when would you use one over an abstract class?"

The 30-second answer: Sealed classes restrict which classes can extend them. `public sealed class Shape permits Circle, Square, Triangle { }` — only those three classes can extend Shape. Useful for modeling closed domains (like algebraic data types) and for exhaustive pattern matching.

Step-by-step thought process: 1. "Sealed = restricted inheritance. The `permits` clause lists allowed subtypes; the compiler enforces that no other class can extend the sealed type." 2. "Each permitted subtype must be one of: `final` (closed), `sealed` (further restricted), or `non-sealed` (re-opens to any subtype)." 3. "Use when: you have a closed set of variants (`Result = Success | Failure`; `HTTPMethod = GET | POST | PUT | DELETE`; `Shape = Circle | Square | Triangle`)." 4. "Killer feature: pattern matching switch (Java 21) checks exhaustiveness. `switch (shape) { case Circle c -> ...; case Square s -> ...; case Triangle t -> ...; }` — compiler errors if you miss a case." 5. "For libraries: lets you evolve the type hierarchy without breaking users — opposite of abstract classes where anyone could extend unexpectedly."

Edge cases to mention: - Sealed types and permitted subtypes must be in the same module (or same package if not modular) - Sealed interfaces work the same way - Records often work well as the leaf types of a sealed hierarchy — `sealed interface Result permits Success, Failure { }`
`record Success(String value) implements Result { }`
`record Failure(String reason) implements Result { }`

What NOT to say: - "Sealed is just a fancy abstract" — misses the exhaustiveness + closed-domain win - "Use sealed everywhere instead of abstract" — wrong fit for open hierarchies (e.g. plugins, extensions)

Common follow-up: "How does sealed help pattern matching switch?"

Q43 · Pattern matching for switch (Java 21)

Asked at: Razorpay · Swiggy · 2024+ senior rounds

Difficulty: Medium · **Topic:** Java 21 features

The question: "What's pattern matching for switch in Java 21? Give an example with a sealed hierarchy."

The 30-second answer: switch can now match on type patterns and decompose data. Combined with sealed types, you get exhaustiveness checking and concise dispatch — like a lightweight version of pattern matching in Kotlin or Scala. Cleaner than instanceof chains.

Step-by-step thought process: 1. "Pre-Java-21: dispatch by type via if-else-instanceof chains, or visitor pattern. Both verbose." 2. "Java 21 pattern matching for switch lets you switch on Object and match by type, optionally binding the value:" 3.

```
java\nString describe(Shape s) {\n return switch (s) {\n case Circle c -> \"radius \" + c.radius();\n case Square sq -> \"side \" + sq.side();\n case Triangle t -> \"triangle\";\n }; \n}\n
```

 4. "If Shape is sealed, the compiler enforces exhaustiveness — adding a new permitted subtype causes compile errors anywhere you forgot to handle it. Safe refactoring." 5. "Guard patterns let you add conditions:

```
case Square sq when sq.side() > 10 -> \"big square\"
```

 ." 6. "Combined with records (Java 21): record patterns destructure —

```
case Circle(double radius) -> radius * 2
```

 ."

Edge cases to mention: -

```
case null
```

 is now allowed in switch (Java 21) — gives explicit null handling instead of NPE - Patterns can be nested:

```
case Container(var content)
```

 etc. - The arrow form (

```
->
```

) doesn't fall through; the colon form (

```
:
```

) still does — prefer arrow for safety

What NOT to say: - "It's like Kotlin's when" — true but generic; show you've used it - "It's just sugar for instanceof" — partly true but the exhaustiveness + records destructuring is more than sugar

Common follow-up: "How do guards work? Show me a switch where guards matter."

Q44 · Virtual threads (Java 21) — what changes?

Asked at: Razorpay · Cred · Swiggy · Meesho 2024+

Difficulty: Medium-Hard · **Topic:** Java 21 features

The question: "What are virtual threads in Java 21? When do they help vs traditional platform threads?"

The 30-second answer: Virtual threads are lightweight threads managed by the JVM, not the OS. You can have millions of them. Blocking IO unmounts the virtual thread from its carrier (an OS thread), so blocking doesn't waste resources. Huge win for IO-bound server workloads (HTTP handlers, DB calls). No benefit for CPU-bound work.

Step-by-step thought process: 1. "Traditional Java threads (now called 'platform threads') are 1:1 with OS threads. Each takes ~1MB stack + OS scheduling overhead. Practical limit: a few thousand." 2. "Virtual threads are scheduled by the JVM onto a small pool of platform threads (carriers). They use small stacks (~few KB) that grow as needed." 3. "On blocking IO (file read, socket read, JDBC call), the virtual thread UNMOUNTS from its carrier — the OS thread is free to run another virtual thread. Massive concurrency without thread starvation." 4. "Create one per request:

```
Executors.newVirtualThreadPerTaskExecutor()
```

 . Spring Boot 3.2+ supports running Tomcat with virtual threads via a single config flag." 5. "Sweet spot: web servers, microservices, gateway apps — any code that's mostly waiting on IO. NOT useful for CPU-bound work (compression, encryption, ML inference)."

Edge cases to mention: - `synchronized` blocks PIN the virtual thread to its carrier — can't unmount. Use `ReentrantLock` for code that might block while holding a lock. - `ThreadLocal` still works but is more expensive per virtual thread — Java 21's `ScopedValue` is the replacement - Native code (JNI) also pins the virtual thread until the native call returns

What NOT to say: - "Virtual threads make CPU-bound work faster" — false; you can't beat physical cores - "Just use them everywhere" — pinning issues mean you still need to know what blocks

Common follow-up: "What does 'pinning' mean and when does it happen?"

Q45 · Structured concurrency (Java 21 preview)

Asked at: Forward-looking senior rounds

Difficulty: Medium-Hard · **Topic:** Java 21 features

The question: "What's structured concurrency in Java 21, and what problem does it solve?"

The 30-second answer: A new API (preview in Java 21) that treats a group of concurrent tasks as a single unit of work — like try-with-resources for tasks. If the parent scope exits, child tasks are automatically cancelled. Prevents thread leaks and orphaned async work that plagued `CompletableFuture` chains.

Step-by-step thought process: 1. "Problem: `CompletableFuture` chains have no parent-child relationship. If you spawn three async calls and forget to handle one, it runs to completion in the background — silently wasting resources." 2. "Structured concurrency: `try (var scope = new StructuredTaskScope.ShutdownOnFailure()) { var f1 = scope.fork(() -> callA()); var f2 = scope.fork(() -> callB()); scope.join(); scope.throwIfFailed(); ... }`" 3. "All forks live within the scope. When the scope exits (normally or via exception), any still-running forks are cancelled." 4. "Two strategies built-in: `ShutdownOnFailure` (cancel siblings on first failure — call A and B in parallel, fail fast) and `ShutdownOnSuccess` (cancel siblings on first success — race them, take the winner)." 5. "Together with virtual threads: clean, structured, scalable concurrency that reads top-to-bottom."

Edge cases to mention: - Still in preview as of Java 21 — JEP 453, expected to stabilize in a future release - Custom scopes via subclassing `StructuredTaskScope` for application-specific policies - Designed to work with the existing Thread API — `Thread.currentThread()` works as expected inside forks

What NOT to say: - "It's like async/await" — closer, but Java's version is more about lifecycle management - "Use it everywhere" — preview API, lock-in risk if you ship before GA

Common follow-up: "How does this compose with virtual threads?"

Q46 · Sequenced collections (Java 21)

Asked at: Java-21-aware rounds

Difficulty: Easy-Medium · **Topic:** Java 21 features

The question: "What's the new `SequencedCollection` interface in Java 21? What gap does it fill?"

The 30-second answer: A new interface for collections with a defined encounter order. Adds uniform methods: `getFirst()`, `getLast()`, `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`, `reversed()`. Implemented by `List`, `LinkedHashSet`, `LinkedHashMap`, `Deque`. Replaces the awkward `list.get(0)` / `list.get(list.size()-1)` patterns.

Step-by-step thought process: 1. "Pre-21: getting the first/last element required different code for each collection — `list.get(0)`, `set.iterator().next()`, `deque.getFirst()`. Inconsistent and ugly." 2. "`SequencedCollection` is a new interface (under `Collection` in the hierarchy) with first/last access methods." 3. "Three new interfaces: `SequencedCollection`, `SequencedSet`, `SequencedMap` (separate hierarchies — `Map` isn't a `Collection`)." 4. "`List` and `Deque` now extend `SequencedCollection`; `LinkedHashSet` implements `SequencedSet`; `LinkedHashMap` implements `SequencedMap`. Your existing `ArrayList`, `LinkedList`, `ArrayDeque`, `LinkedHashSet`, `LinkedHashMap` all gain the new methods automatically." 5. "`reversed()` returns a view that iterates in reverse — no copying. Modifications to the view affect the original."

Edge cases to mention: - Not all collections are sequenced — `HashSet` and `HashMap` aren't (no defined order) - `reversed()` on a `List` returns a `List` view; iteration order is reversed but indices still go 0..size - Slight gotcha: `addFirst` on an immutable collection throws `UnsupportedOperationException`

What NOT to say: - "It's just a wrapper around the old methods" — true sort of, but the uniformity is the win - "Use it instead of `List`" — it's not a replacement, just an enhancement

Common follow-up: "How does this change how you'd implement an LRU cache?"

Q47 · Text blocks — what changed in Java 15

Asked at: Easy, modern Java rounds

Difficulty: Easy · **Topic:** Java 15 features

The question: "What's a text block in Java 15? Solve me a problem you'd actually use it for."

The 30-second answer: Triple-quoted multi-line strings. Eliminates the escape-hell of embedded JSON, SQL, HTML in Java source. `"""multi-line text"""`. Automatic indentation normalization based on the closing delimiter position.

Step-by-step thought process: 1. "Pre-15: multi-line strings required `\n` and `+` concatenation, especially painful for embedded JSON/XML/SQL." 2. "Java 15: text blocks. Open with `"""`, content on subsequent lines, close with `"""`. The position of the closing delimiter determines the 'indentation baseline' — leading whitespace up to that column is stripped." 3. "Example: embedding SQL

— `String sql = "\"\"\"\n SELECT id, name, email\n FROM users\n WHERE active = true\n \"\"\"\";` — clean, readable, no concatenation." 4. "Embedded quotes don't need escaping inside text blocks (except triple-quote sequences themselves)." 5. "Newlines are normalized to `\n` regardless of source file line endings."

Edge cases to mention: - `String.formatted(args)` (Java 15+) is a fluent alternative to `String.format(template, args)` — pairs nicely with text blocks - Embedded expressions are NOT supported (no string interpolation as of Java 21) — template strings were preview in Java 21 + 22 (JEP 430, 459), then withdrawn in Java 23 - Trailing newline can be suppressed with `\` at the end of a line

What NOT to say: - "Use string concatenation, it's just as readable" — no, embedded JSON in concatenation is unreadable - "Text blocks support string interpolation" — wrong, not yet

Common follow-up: "How would you do string interpolation in Java?"

Q48 · Java 21 ScopedValue vs ThreadLocal

Asked at: Java-21-aware senior rounds

Difficulty: Medium-Hard · **Topic:** Java 21 features

The question: "What's `ScopedValue` (Java 21), and why is it preferred over `ThreadLocal` in modern Java?"

The 30-second answer: `ScopedValue` is an immutable, scoped alternative to `ThreadLocal`. Value is set for the duration of a specific code block (`ScopedValue.runWhere`), then automatically cleared. Solves `ThreadLocal`'s leak problems in virtual-thread / thread-pool environments.

Step-by-step thought process: 1. "`ThreadLocal` sticks values to threads. In a thread pool or virtual-thread environment, threads are reused, and `ThreadLocal` values leak across requests unless explicitly cleared." 2. "`ScopedValue` is bound for the duration of a `run` or `call` block: `ScopedValue.runWhere(USER, currentUser, () -> doWork());`. After the lambda returns, the binding is automatically removed." 3. "Immutable per binding — can't be reassigned mid-scope. Nesting allowed (inner `runWhere` shadows outer)." 4. "Designed to play well with virtual threads — millions of virtual threads each with bounded scoped values, no leak risk." 5. "Use cases: per-request user/tenant context, transaction context, observability tags."

Edge cases to mention: - Reads are slightly cheaper than `ThreadLocal` (no map lookup, just a thread-local stack) - Inherits across virtual thread forks within the same structured concurrency scope - Still in preview as of Java 21 (requires `--enable-preview`) — API expected to stabilize in a future release

What NOT to say: - "`ScopedValue` replaces `ThreadLocal` everywhere" — gradually; both exist for now - "It's like Spring's `RequestContextHolder`" — adjacent but lower-level

Common follow-up: "How does this work with virtual threads forking children?"

Q49 · Spring Boot 3 + Java 17 baseline — what changed?

Asked at: Spring backend rounds 2024+

Difficulty: Medium · **Topic:** Spring ecosystem

The question: "You're upgrading a Spring Boot 2 app to Spring Boot 3. What are the major breaking changes?"

The 30-second answer: (1) Java 17 minimum (was 8). (2) Jakarta EE 9 migration — `javax.*` packages renamed to `jakarta.*` across the entire app. (3) New observability stack — Micrometer Observation replaces Sleuth. (4) Native compilation via Spring AOT. (5) Hibernate 6 with new query semantics.

Step-by-step thought process: 1. "Java 17 baseline — your build must use JDK 17+. Many libraries also dropped Java 8/11 support around the same time." 2. " `javax.*` → `jakarta.*` is the most disruptive change. Every `javax.persistence`, `javax.servlet`, `javax.validation` import needs updating. IDE refactor tools help; openrewrite recipe automates much of it." 3. "Spring Sleuth (distributed tracing) is replaced by Micrometer Tracing + Micrometer Observation API." 4. "Spring AOT processing + GraalVM native image support — your jar can build to a standalone binary with sub-second cold start. Useful for serverless." 5. "Hibernate 6 changes: stricter SQL generation, new ID generators, and some JPQL behavior changes — test your queries carefully."

Edge cases to mention: - Spring Security 6 has breaking changes too (lambda-style DSL replaces fluent builder) - Spring Boot 3.2 added virtual thread support — one config flag flips Tomcat to virtual threads - Native compilation has constraints (reflection, dynamic proxies need hints) — not zero-effort

What NOT to say: - "Just bump the version, it'll work" — Jakarta migration alone is a multi-day refactor on real codebases - "Native compilation is faster at runtime" — startup yes, throughput often slightly worse

Common follow-up: "How would you enable virtual threads in a Spring Boot 3 app?"

Q50 · "What's your favourite Java feature added in the last 3 years?"

Asked at: Closing question, all levels

Difficulty: Soft / signal-question · **Topic:** Self-awareness

The question: "What's your favourite Java feature added in the last 3 years, and why?"

The 30-second answer: This is a signal question. They're checking if you actually keep up with the language. Pick something concrete, recent (Java 17 or later), and explain a real use case where it changed your code for the better. Records, pattern matching, virtual threads, sealed classes — any of these work if you can talk about them concretely.

Step-by-step thought process: 1. "This is a personality/curiosity signal. Wrong answer: 'I don't know, I just write code.' Correct answer: pick one and explain how it affected your work." 2. "Strong picks for 2024-26:" 3. " **Virtual threads (Java 21)** — 'I migrated our gateway service to virtual threads and our p99 latency dropped 30% because we stopped queuing on the thread pool.'" 4. " **Records (Java 16)** — 'I deleted ~2000 lines of DTO boilerplate when we adopted records across our REST layer.'" 5. " **Pattern matching for switch (Java 21)** — 'Our event-handling code became exhaustive at compile time — we used to find missed event types in prod.'" 6. " **Text blocks (Java 15)** — 'We embedded SQL in code without losing readability or fighting concatenation.'"

Edge cases to mention: - If the interviewer is on an old Java version, mention it but pivot: "Records are great but I know team's on Java 11 — I've also gotten a lot of value from the Java 8 stream API improvements." - Show curiosity: "I've been playing with structured concurrency in Java 21 — too early to use in production but I like where it's heading." - Don't just list — pick ONE and tell a 30-second story about it

What NOT to say: - "I don't really follow Java updates" — instant downgrade - "Lambdas" (added in Java 8, ~12 years ago) — shows you stopped paying attention - "Generics" (Java 5) — even worse

Common follow-up: "Have you tried virtual threads in production? What surprised you?"

Tier A — Asked in 50-80% of Java interviews (80 Qs)

These questions live in the bulk of mid-level interview rounds — past the headline Top 50 and into the meat of what gets asked in a typical 45-60 minute Java round. Work through Tier A over a weekend; you don't need to memorize, but you should be able to sketch the answer within 20 seconds for any of them.

Q51 · HashSet vs TreeSet vs LinkedHashSet — when each?

Asked at: TCS · Infosys · Razorpay · Swiggy · most mid-level rounds

Difficulty: Easy-Medium · **Topic:** Collections

The question: "Walk me through the difference between HashSet, TreeSet, and LinkedHashSet. When would you reach for each one?"

The 30-second answer: HashSet = backed by HashMap, O(1) add/contains, no ordering. LinkedHashSet = HashSet + a doubly-linked list of insertion order, O(1) ops, predictable iteration. TreeSet = backed by a red-black tree, O(log n) ops, sorted order (natural or via Comparator). Default to HashSet unless you need ordering.

Step-by-step thought process: 1. "All three implement Set, so duplicates are rejected via `equals()` / `hashCode()`. The difference is iteration order and operation cost." 2. "HashSet — no ordering at all. The iteration order can change after a resize. Use when you only care about membership." 3. "LinkedHashSet — maintains insertion order. Same O(1) ops as HashSet, slightly more memory because of the linked list pointers. Use when iteration order needs to be predictable, e.g. when you're

deduplicating a sequence and want to keep first-seen order." 4. "TreeSet — sorted order, but every op becomes $O(\log n)$. It also gives you NavigableSet methods like `floor`, `ceiling`, `headSet`, `tailSet` — useful for range queries." 5. "Concrete pick: I store unique `merchantId`s seen during a batch in a HashSet; if I need to dedupe order events while preserving arrival order, LinkedHashSet; if I need to find 'the next refund slot after timestamp T', TreeSet."

Edge cases to mention: - TreeSet requires either elements implementing `Comparable` or a Comparator — otherwise `ClassCastException` at runtime on first insert - TreeSet uses `compareTo()` for equality, NOT `equals()` — easy way to silently lose elements if your `compareTo` returns 0 for "different" objects - LinkedHashSet has a constructor `LinkedHashSet<>(map, accessOrder=true)` — wait, that's LinkedHashMap, not Set. LinkedHashSet is always insertion-order

What NOT to say: - "TreeSet is just a sorted HashSet" — wrong, underlying data structure is completely different - "Use LinkedHashSet for thread safety" — none of these are thread-safe by default

Common follow-up: "How would you make a TreeSet thread-safe?"

Q52 · HashMap vs Hashtable vs ConcurrentHashMap

Asked at: TCS · Infosys · Wipro · Cognizant · almost every L1/L2 round

Difficulty: Easy-Medium · **Topic:** Collections

The question: "Compare HashMap, Hashtable, and ConcurrentHashMap. Which one would you pick for a multi-threaded application?"

The 30-second answer: HashMap = unsynchronized, fastest, nulls allowed. Hashtable = legacy, every method synchronized on `this`, no nulls. ConcurrentHashMap = modern, bucket-level striping (Java 8: CAS + synchronized on bucket head), high concurrency, no nulls. For multi-threaded code, always pick ConcurrentHashMap — Hashtable is effectively deprecated.

Step-by-step thought process: 1. "HashMap is unsynchronized — fastest in single-threaded code, but two threads writing concurrently can corrupt the internal structure (infinite loop on resize in Java 7 was the classic bug)." 2. "Hashtable is the old JDK 1.0 class — every method is `synchronized` on the table object. Coarse-grained lock, terrible throughput under contention. Disallows null keys and values." 3. "ConcurrentHashMap (Java 8+) gave up the segment-based locking of Java 7 and switched to a per-bucket strategy: CAS for insert into empty bucket, synchronized on the first node when there's a collision." 4. "Iteration on ConcurrentHashMap is weakly consistent — it won't throw `ConcurrentModificationException`, but it may or may not reflect updates that happen during iteration." 5. "Default pick today: HashMap if single-threaded, ConcurrentHashMap if shared across threads. Never new Hashtable."

Edge cases to mention: - ConcurrentHashMap doesn't allow null keys or null values — historical reason: ambiguity between "key missing" and "key mapped to null" in concurrent context - `Collections.synchronizedMap(new HashMap<>())` wraps HashMap with method-level

synchronization — same throughput problem as Hashtable, only use if you need a stable iterator with external locking - ConcurrentHashMap has `compute`, `computeIfAbsent`, `merge` which are atomic — use them instead of get-then-put

What NOT to say: - "Hashtable is thread-safe and HashMap isn't, so use Hashtable" — outdated by ~15 years - "ConcurrentHashMap locks the whole map on writes" — wrong, that's Hashtable

Common follow-up: "What does ConcurrentHashMap do under the hood when two threads call `computeIfAbsent` on the same key?"

Q53 · ConcurrentHashMap internals — what changed in Java 8?

Asked at: Razorpay · Cred · Swiggy · Flipkart · senior backend rounds

Difficulty: Medium-Hard · **Topic:** Concurrency / Collections

The question: "What changed in ConcurrentHashMap from Java 7 to Java 8 internally?"

The 30-second answer: Java 7 used 16 fixed segments, each a mini-HashMap with its own ReentrantLock — concurrency level was capped at 16. Java 8 dropped segments, switched to one big table where insert into an empty bucket uses CAS, and collisions sync on the first node of that bucket. Treeification kicks in at 8 nodes per bucket (same as HashMap).

Step-by-step thought process: 1. "In Java 7, the map was split into Segments — a Segment was essentially a mini-HashMap protected by a ReentrantLock. Default 16 segments, so at most 16 threads could write concurrently without blocking." 2. "Each segment maintained its own count and resized independently. Memory overhead was significant — every segment carried its own table and lock." 3. "Java 8 rewrote it: one table, no segments. Insert into an empty bucket uses Unsafe's CAS. When a bucket has at least one node, the writer acquires `synchronized` on the head node of the chain." 4. "Counts are tracked via a `LongAdder` -style striped counter (`CounterCell` array) to avoid contention on a single `size` field." 5. "Treeification at 8 nodes per bucket — same as HashMap. So worst-case bucket lookup is $O(\log n)$ instead of $O(n)$."

Edge cases to mention: - `size()` is approximate — it sums the striped counter, but writes that race during the sum can be off by a few - Iterators are weakly consistent — they see a snapshot of buckets as of when they reached each one - `forEach`, `reduce`, and `search` have parallel variants that take a parallelism threshold — used when you actually have a huge map

What NOT to say: - "It still uses segments" — wrong post-Java 8 - "ConcurrentHashMap is just a HashMap with synchronized methods" — that's `Collections.synchronizedMap`, completely different mechanism

Common follow-up: "What's the cost of `computeIfAbsent` if the same key is hit by 100 threads simultaneously?"

Q54 · EnumMap and EnumSet — why bother?

Asked at: Goldman · JP Morgan India · Morgan Stanley · senior rounds

Difficulty: Medium · **Topic:** Collections

The question: "When would you choose EnumMap over HashMap, and EnumSet over HashSet?"

The 30-second answer: When the key (or element) is an enum, EnumMap/EnumSet are dramatically faster and smaller. EnumMap is internally an array indexed by enum ordinal — $O(1)$ without hashing. EnumSet is a bitmask (a `long` for ≤ 64 values, `long[]` otherwise) — set operations become bitwise AND/OR.

Step-by-step thought process: 1. "Both are part of the JDK since Java 5 but underused. The trick: when your keys are enum constants, you don't need hashing — the ordinal IS the index." 2. "EnumMap holds an internal `Object[]` of size equal to the number of enum constants. `get(key)` is just `array[key.ordinal()]` — no hashing, no equals, no boxing." 3. "EnumSet is even better — for enums with ≤ 64 constants, the entire set is a single `long`. `contains(v)` is `(bits & (1L << v.ordinal())) != 0`. `addAll(otherSet)` is one OR instruction." 4. "Concrete example: I track order states (`PLACED`, `PAID`, `SHIPPED`, `DELIVERED`, `CANCELLED`) — EnumMap is faster, smaller, and iterates in declaration order." 5. "Both reject nulls and disallow non-enum keys at compile time — type system enforces correctness."

Edge cases to mention: - EnumMap iteration is always in enum declaration order — no LinkedEnumMap needed - EnumSet has nice factory methods: `EnumSet.of(STATE_A, STATE_B)`, `EnumSet.complementOf(set)`, `EnumSet.range(LOW, HIGH)` - Neither is thread-safe — wrap with `Collections.synchronizedMap` / `synchronizedSet` if you must, but usually you don't need concurrent enum-keyed maps

What NOT to say: - "Just use HashMap, it's general-purpose" — leaves performance on the table for no reason - "EnumSet is a sorted set" — it iterates in declaration order, not sorted order

Common follow-up: "What happens to EnumSet if you have 1000 enum constants?"

Q55 · Iterator vs ListIterator vs Spliterator

Asked at: Most Java mid-level rounds · Infosys · Wipro · Wells Fargo

Difficulty: Medium · **Topic:** Collections

The question: "What's the difference between Iterator, ListIterator, and Spliterator?"

The 30-second answer: Iterator = forward-only, can remove. ListIterator = bidirectional + can add/set/remove, only available on Lists. Spliterator = Java 8, designed for parallel traversal — can split itself into halves for fork-join, reports characteristics (SIZED, SORTED, ORDERED, etc.).

Step-by-step thought process: 1. "Iterator (since Java 1.2) has `hasNext`, `next`, and `remove`. Forward-only. Works on any Collection." 2. "ListIterator extends Iterator with `hasPrevious`,

`previous`, `add`, `set`. Bidirectional and mutating. Only Lists expose it via `listIterator()`. " 3. "Spliterator (Java 8) was added for the Stream API. Its key method is `trySplit()` — it returns a new Spliterator covering half the remaining elements, so parallel streams can fork the work." 4. "Spliterators report characteristics — SIZED (size is known), ORDERED, SORTED, DISTINCT, IMMUTABLE, CONCURRENT, NONNULL, SUBSIZED. The stream pipeline uses these to optimize." 5. "Practical use: I rarely write Spliterators by hand. I use them indirectly via `stream()` or `parallelStream()`. Custom Spliterators are useful when wrapping a non-standard data source for streaming."

Edge cases to mention: - `Iterator.remove()` is the ONLY safe way to mutate a collection during iteration — calling `list.remove()` inside an enhanced for-loop throws `ConcurrentModificationException` - `ListIterator's nextIndex()` and `previousIndex()` help when you need both position and value - A Spliterator that can't split returns null from `trySplit()` — the stream still works, just serial

What NOT to say: - "Spliterator is just an Iterator" — misses the split + characteristics design - "ListIterator works on any Collection" — wrong, Lists only

Common follow-up: "How does a Spliterator decide when to stop splitting?"

Q56 · fail-fast vs fail-safe iterators

Asked at: TCS · Infosys · Accenture · standard L1/L2 question

Difficulty: Medium · **Topic:** Collections

The question: "What's the difference between a fail-fast and a fail-safe iterator? Give me examples."

The 30-second answer: Fail-fast = throws `ConcurrentModificationException` if the collection is structurally modified during iteration (via the collection's own methods or another thread). Fail-safe = doesn't throw; iterates over a snapshot or a weakly-consistent view. `ArrayList`, `HashMap`, `HashSet` are fail-fast. `CopyOnWriteArrayList`, `ConcurrentHashMap` are fail-safe.

Step-by-step thought process: 1. "Fail-fast iterators track a `modCount` field — every structural modification (add, remove, clear) increments it. On `next()`, the iterator compares `modCount` to its captured `expectedModCount` — mismatch throws CME." 2. "This is a best-effort detection — it's NOT a thread-safety guarantee. The CME is raised on a best-effort basis; you can have races where it's missed." 3. "Fail-safe iterators work on a snapshot or a concurrent-aware structure. `CopyOnWriteArrayList` iterates over the array as it was when the iterator was created — writes after that don't affect iteration." 4. "ConcurrentHashMap's iterator is weakly consistent — it sees buckets as they were when the iterator reached them. No CME, but you don't get a clean snapshot either." 5. "Practical rule: if you must mutate during iteration, use `iterator.remove()` (works on fail-fast iterators) or pick a concurrent collection."

Edge cases to mention: - Calling `list.remove(item)` inside `for (T item : list)` triggers CME — that's just an iterator under the hood - `iterator.remove()` is allowed because it updates

`expectedModCount` along with `modCount` - `CopyOnWriteArrayList` is fail-safe but expensive — every write copies the whole array; only sensible for read-heavy workloads (event listeners, observers)

What NOT to say: - "Fail-safe means thread-safe" — partially true but misleading; the iterator is safe, mutations may still race - "Use try/catch on CME" — terrible code smell; fix the iteration, don't swallow it

Common follow-up: "How would you safely remove items from an `ArrayList` while iterating?"

Q57 · WeakHashMap and IdentityHashMap — when?

Asked at: Senior rounds · Cred · Goldman · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Collections

The question: "What do `WeakHashMap` and `IdentityHashMap` solve? Give a real use case for each."

The 30-second answer: `WeakHashMap` holds keys by weak reference — when the key has no strong references elsewhere, the GC can reclaim it and the entry is removed from the map. Useful for caches and metadata tied to object lifetime. `IdentityHashMap` uses `==` for key comparison instead of `equals()` — useful when you need to track objects by identity, not value.

Step-by-step thought process: 1. "Regular `HashMap` holds strong references — keys never get GC'd as long as they're in the map. That's a memory leak if you're using objects as cache keys and forget to remove them." 2. "`WeakHashMap` wraps each key in a `WeakReference`. When the GC determines the key is otherwise unreachable, the reference is cleared and the entry is removed on the next map operation." 3. "Use case: I cache parsed configurations keyed by a `Merchant` object. If the merchant is no longer in use anywhere else, I don't want the cache to keep it alive." 4. "`IdentityHashMap` replaces `equals()` and `hashCode()` with `==` and `System.identityHashCode()`. Two different objects with the same content are different keys." 5. "Use case: I'm walking an object graph and tracking which objects I've already visited — I want identity-based deduplication, not value-based. Serialization frameworks do this."

Edge cases to mention: - `WeakHashMap` values are still strong references — if a value points back to the key, the entry never gets cleaned up - `IdentityHashMap` is also used for thread-stack tracking and some reflection internals - Neither is thread-safe; both have subtle gotchas — read the Javadoc before using

What NOT to say: - "`WeakHashMap` is for thread safety" — wrong, completely unrelated - "`IdentityHashMap` is faster than `HashMap`" — irrelevant comparison; they solve different problems

Common follow-up: "What happens to a `WeakHashMap` entry if the value strongly references the key?"

Q58 · PriorityQueue — internals and gotchas

Asked at: Razorpay · Swiggy · most product co rounds, often via DSA

Difficulty: Medium · **Topic:** Collections

The question: "How is PriorityQueue implemented in Java? What are the common mistakes when using it?"

The 30-second answer: PriorityQueue is a binary min-heap stored in an array. `offer` and `poll` are $O(\log n)$; `peek` is $O(1)$. It's NOT sorted in iteration order — iterating gives you heap order, not sort order. Not thread-safe; use `PriorityBlockingQueue` for concurrent code.

Step-by-step thought process: 1. "Internally it's an `Object[]` holding a heap. Parent of index `i` is at $(i-1)/2$, children at `2i+1` and `2i+2`. Default capacity is 11 and grows by ~50% when full." 2. "`offer` adds to the end, then sifts up to maintain heap property — $O(\log n)$. `poll` removes root, moves last element to root, sifts down — $O(\log n)$. `peek` is just `array[0]`." 3. "By default, it's a MIN-heap using natural ordering. For a max-heap, pass `Comparator.reverseOrder()` or a custom comparator." 4. "Critical gotcha: `iterator()` does NOT iterate in priority order — it's heap order. If you need sorted output, repeatedly poll until empty (and that mutates the heap)." 5. "`remove(Object o)` is $O(n)$ — it scans the array linearly to find `o`, then sifts. For a fast removal-by-object, use a separate index map."

Edge cases to mention: - Nulls are rejected — `NullPointerException` on `offer` - Custom comparator with inconsistent `compare` (not following total ordering) produces undefined behavior - For producer-consumer between threads, use `PriorityBlockingQueue` — same semantics, but offers `take()` that blocks until an element arrives

What NOT to say: - "PriorityQueue is sorted" — wrong, only the head is guaranteed to be the smallest - "Iteration gives sorted order" — wrong, it's heap order

Common follow-up: "How would you implement a top-K-frequent-elements problem with a PriorityQueue?"

Q59 · Deque — ArrayDeque vs LinkedList

Asked at: Most Java mid-level rounds · Wipro · Cognizant

Difficulty: Easy-Medium · **Topic:** Collections

The question: "For a stack or queue use case, would you pick ArrayDeque or LinkedList?"

The 30-second answer: ArrayDeque almost always. It's a circular array, $O(1)$ add/remove at either end, cache-friendly, no per-element overhead. LinkedList works but each node carries prev/next pointers, hurts cache locality, and most operations are slower in practice.

Step-by-step thought process: 1. "For stack semantics — `push` and `pop` — `Stack` is legacy (synchronized, extends `Vector`). The recommended replacement is `Deque<T> stack = new`

`ArrayDeque<>()`; then use `push()` and `pop()` . " 2. "For queue semantics — FIFO — `Deque<T> queue = new ArrayDeque<>();` then `offer()` and `poll()` . Or use `ArrayDeque` directly as a `Queue` . " 3. "ArrayDeque is a circular buffer — head and tail indices wrap around. Resize doubles the capacity when full." 4. "LinkedList does the same job but every element has 24 bytes of node overhead on top of the value. CPU prefetching doesn't help linked lists." 5. "Concrete pick: I'm implementing a recursion-to-iteration conversion (DFS with explicit stack) — ArrayDeque every time."

Edge cases to mention: - ArrayDeque rejects nulls; LinkedList accepts them. If null is a sentinel for "no value" in your design, you can't use ArrayDeque - Neither is thread-safe — use

`LinkedBlockingDeque` or `ConcurrentLinkedDeque` for concurrent code - `ArrayDeque.iterator()` and `descendingIterator()` are both fail-fast

What NOT to say: - "Use Stack class — it's the Java way" — Stack is legacy, the docs recommend Deque - "LinkedList is faster because it's a linked structure" — almost always wrong on modern CPUs

Common follow-up: "What's the difference between `ConcurrentLinkedDeque` and `LinkedBlockingDeque`?"

Q60 · Comparable vs Comparator — design and use

Asked at: Almost every Java interview

Difficulty: Easy-Medium · **Topic:** Collections / OOP

The question: "Explain the difference between Comparable and Comparator. Which one would you use when?"

The 30-second answer: Comparable = natural ordering, implemented inside the class itself via `compareTo()` . Comparator = external ordering, you pass it to sort/treemap when you want a different order. Use Comparable for the class's "obvious" order (e.g. dates by chronology); use Comparator for everything else and for classes you don't own.

Step-by-step thought process: 1. "Comparable is a single-method interface — `int compareTo(T other)` . Returns negative if this < other, 0 if equal, positive if this > other. Implemented by Integer, String, LocalDate — anything with one obvious natural order." 2. "Comparator is also a functional interface — `int compare(T a, T b)` . You pass an instance to `Collections.sort(list, comparator)` or to a TreeMap/TreeSet constructor." 3. "Java 8 made Comparator much nicer: `Comparator.comparing(Order::getAmount).thenComparing(Order::getCreatedAt).reversed()` reads naturally." 4. "Use Comparable when the class has ONE canonical order — e.g. money amounts, timestamps. Use Comparator when there are multiple valid sort orders, or when sorting a third-party class." 5. "Both must follow total ordering — antisymmetry, transitivity, consistency. A broken comparator corrupts TreeSet and gives wildly wrong sort results."

Edge cases to mention: - `compareTo` must be consistent with `equals` — strictly, if `compareTo` returns 0, `equals` should return true. TreeSet/TreeMap use compareTo for equality, so inconsistency

causes "missing" elements - `Comparator.naturalOrder()` and `reverseOrder()` exist for cases where you'd otherwise write a one-liner - `Comparator.nullsFirst(comparator)` / `nullsLast` handle nulls gracefully — useful when sorting columns that may have nulls

What NOT to say: - "Comparable is for objects, Comparator is for primitives" — completely wrong - "Use Comparable so you can sort with `Collections.sort(list)` without arguments" — true but incomplete — explain WHY (natural order)

Common follow-up: "How does `Comparator.comparing()` work under the hood?"

Q61 · ExecutorService — types and when each

Asked at: Razorpay · Swiggy · Cred · most product co backend rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "Walk me through the different ExecutorService implementations you get from `Executors.*`. When would you use each?"

The 30-second answer: `FixedThreadPool(n)` — N permanent threads, unbounded queue. `CachedThreadPool` — threads spawned on demand, idle threads die after 60s, unbounded thread creation. `SingleThreadExecutor` — 1 thread, unbounded queue, sequential. `ScheduledThreadPool(n)` — for delayed/periodic tasks. For production, prefer `new ThreadPoolExecutor(...)` directly — the Executors factories have dangerous defaults.

Step-by-step thought process: 1. "FixedThreadPool is the workhorse — N threads, an unbounded `LinkedBlockingQueue`. Risk: if producers outpace consumers, the queue grows unbounded and you OOM." 2. "CachedThreadPool spawns a thread per task if no idle thread is available. Threads die after 60s idle. Risk: a flood of tasks spawns unlimited threads and crashes the JVM." 3. "SingleThreadExecutor is one thread + unbounded queue. Useful when you need strict ordering of tasks but want them off the calling thread." 4. "ScheduledThreadPool gives you `schedule(...)` and `scheduleAtFixedRate(...)`. Important: if a scheduled task throws, subsequent runs are silently cancelled — always wrap with try/catch." 5. "For real systems, I instantiate ThreadPoolExecutor directly with a bounded queue and a sensible RejectedExecutionHandler — usually `CallerRunsPolicy` to backpressure the producer."

Edge cases to mention: - Always call `executor.shutdown()` followed by `awaitTermination` on JVM exit — orphan threads keep the JVM running - `ThreadPoolExecutor` has a customizable `ThreadFactory` — use it to name threads (`payment-worker-1`) so thread dumps are debuggable - For Java 21+ virtual threads, `Executors.newVirtualThreadPerTaskExecutor()` is the modern alternative for I/O-heavy work

What NOT to say: - "Always use FixedThreadPool" — fine for small workloads, dangerous at scale because of the unbounded queue - "CachedThreadPool scales automatically" — yes, into OOM

Common follow-up: *"What's the difference between `scheduleAtFixedRate` and `scheduleWithFixedDelay`?"*

Q62 · Future vs CompletableFuture — what's the difference?

Asked at: Razorpay · Cred · Flipkart · Swiggy · most product co rounds

Difficulty: Medium-Hard · **Topic:** Concurrency

The question: *"What does `CompletableFuture` give you that plain `Future` doesn't?"*

The 30-second answer: `Future` = poll-and-block. You submit a task, `get()` blocks until done, `cancel()` may stop it. `CompletableFuture` = composable. You chain `thenApply`, `thenCompose`, `thenCombine`, register `whenComplete` callbacks, build pipelines of dependent async operations, handle errors with `exceptionally` — all without blocking.

Step-by-step thought process: 1. *"`Future` (Java 5) lets you submit work and `get()` the result. That's it. You can't react to completion, you can't chain another async step, you can't combine multiple futures elegantly."* 2. *"`CompletableFuture` (Java 8) implements `Future` + `CompletionStage`. `thenApply` chains a transformation, `thenCompose` chains another async step (flatMap-like), `thenCombine` joins two futures."* 3. *"Error handling: `exceptionally` provides a fallback value if the chain failed. `handle` always runs and gets both result and throwable."* 4. *"Concrete: I fetch the customer, then in parallel fetch their orders and their wallet balance, then build a response. `CompletableFuture.supplyAsync(...).thenCombine(...)` does this cleanly."* 5. *"The default executor for `CompletableFuture` is `ForkJoinPool.commonPool()` — if your work is blocking I/O, pass your own executor as the second arg to every async method."*

Edge cases to mention: - `cancel()` on `CompletableFuture` only sets the state to cancelled — it doesn't actually interrupt the running task (unlike `Future` from `ExecutorService`) - A single throw inside a chain propagates as `CompletionException` wrapping the original — your handlers must unwrap - `allOf(...)` returns `CompletableFuture<Void>` — use `thenApply` after it to collect results from each individual future

What NOT to say: - "Future and `CompletableFuture` are basically the same" — misses the entire point of composability - "CompletableFuture is faster than `Future`" — wrong framing; it's a different programming model

Common follow-up: *"What's the difference between `thenApply` and `thenCompose`?"*

Q63 · ReentrantLock vs synchronized — when each?

Asked at: Razorpay · Cred · Wells Fargo · senior rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "When would you reach for `ReentrantLock` instead of the `synchronized` keyword?"

The 30-second answer: `synchronized` is simpler and integrates with the JVM — biased locking, monitor inflation, just works for 90% of cases. `ReentrantLock` gives you `tryLock` (non-blocking attempt), `tryLock(timeout)`, fair ordering, multiple condition variables via `newCondition()`, and lockable across method boundaries. Use `ReentrantLock` when you need any of those features.

Step-by-step thought process: 1. "`synchronized` is a JVM-level construct — concise, ties to a monitor on an object, automatically released when the block exits (even on exception)." 2. "`ReentrantLock` is a class. You explicitly `lock()` and must `unlock()` in a finally block — otherwise you leak the lock and deadlock everything else." 3. "Things you can't do with `synchronized`: `tryLock()` to attempt acquisition without blocking, `lockInterruptibly()` to back out if interrupted, `tryLock(5, SECONDS)` to time out." 4. "`ReentrantLock` supports fair ordering via constructor flag — threads acquire in FIFO order rather than first-to-grab. Costs throughput, but avoids starvation." 5. "With `ReentrantLock` you can create multiple Condition objects (`lock.newCondition()`) — equivalent to multiple wait sets on the same monitor. `synchronized` has one wait set."

Edge cases to mention: - Both are reentrant — the same thread can lock multiple times without deadlocking itself, and must unlock the same number of times - Java 24's JEP 491 means `synchronized` no longer pins virtual threads — pre-Java 24, you needed `ReentrantLock` to avoid pinning - Always `try { lock.lock(); ... } finally { lock.unlock(); }` — never call `lock()` outside a try block

What NOT to say: - "ReentrantLock is faster than `synchronized`" — modern JVMs optimize `synchronized` aggressively; benchmark before assuming - "`synchronized` is deprecated" — completely wrong

Common follow-up: "Why was the JEP 491 change in Java 24 important for virtual threads?"

Q64 · CountdownLatch vs CyclicBarrier vs Semaphore vs Phaser

Asked at: Razorpay · Cred · senior backend rounds

Difficulty: Hard · **Topic:** Concurrency

The question: "When would you use `CountDownLatch`, `CyclicBarrier`, `Semaphore`, and `Phaser`? Give me one use case each."

The 30-second answer: `CountDownLatch` = one-shot "wait for N events" gate (e.g. wait for N services to be ready before starting). `CyclicBarrier` = reusable "wait for N threads to reach a barrier, then all proceed". `Semaphore` = N permits to control concurrent access to a resource (e.g. limit DB connections). `Phaser` = dynamic, multi-phase barrier where threads can register/deregister between phases.

Step-by-step thought process: 1. "`CountDownLatch` is initialized with a count. `countDown()` decrements; `await()` blocks until count is zero. One-shot — once at zero, it stays there." 2. "Use `CountDownLatch` for: 'block this thread until N async setup tasks finish'. I use it in tests — fire N concurrent calls, latch waits until they all return." 3. "`CyclicBarrier` is initialized with parties. Each thread

calls `await()` ; when all N have arrived, they're all released simultaneously. Resets and is reusable." 4. "Use `CyclicBarrier` for: parallel-phase algorithms. All workers compute phase 1, sync at barrier, all proceed to phase 2." 5. "Semaphore holds N permits. `acquire()` blocks if none available; `release()` returns one. Use it to throttle: 'no more than 10 concurrent calls to the payment gateway'." 6. "Phaser is the flexible one — parties can register and deregister between phases, multiple phases supported. Replaces both `CountDownLatch` and `CyclicBarrier` in newer code when you need that flexibility."

Edge cases to mention: - `CountDownLatch` can't be reset — if you need reuse, use `CyclicBarrier` or `Phaser` - `CyclicBarrier` supports a barrier action (`Runnable` run when the last thread arrives) — useful for merging phase results - Semaphore has `tryAcquire()` and `tryAcquire(timeout)` — same throttling pattern as connection pools

What NOT to say: - "They're all basically the same" — they solve different problems - "Just use synchronized for everything" — non-answer that admits you don't know the tools

Common follow-up: "How would you implement a connection pool using Semaphore?"

Q65 · AtomicInteger and CAS — how does it work?

Asked at: Razorpay · Cred · Goldman · senior rounds

Difficulty: Medium-Hard · **Topic:** Concurrency

The question: "How does `AtomicInteger` achieve thread safety without locks?"

The 30-second answer: It uses Compare-And-Swap (CAS) — a hardware instruction that atomically reads a memory location, compares it to an expected value, and writes a new value only if the comparison matches. `AtomicInteger` wraps a `volatile int` and uses `Unsafe`'s CAS primitive in a retry loop.

Step-by-step thought process: 1. "Under the hood: `AtomicInteger` holds a `volatile int value` plus an internal handle to `Unsafe` (or `VarHandle` in modern Java)." 2. "`incrementAndGet()` does: read current value, compute $current+1$, call `compareAndSet(current, current+1)` . If another thread changed value in the meantime, CAS fails and we retry." 3. "CAS maps to a single hardware instruction — `CMPXCHG` on x86, `LDREX/STREX` pair on ARM. Atomic in hardware, no OS-level lock needed." 4. "Cost is much lower than synchronized for low-to-moderate contention. For very high contention, CAS retries thrash — `LongAdder` is faster (striped per-thread cells)." 5. "The `Atomic` family covers `Integer`, `Long`, `Boolean`, `Reference`, plus `AtomicReferenceArray` and field updaters for atomic operations on fields you don't own."*

Edge cases to mention: - ABA problem: a value can change from A to B and back to A — CAS sees A and succeeds, even though the value was modified. Use `AtomicStampedReference` if you need to detect ABA - `getAndAdd(delta)` is preferred over `incrementAndGet()` if you need addition by a value other than 1 - For very high write contention on a counter, `LongAdder` and `LongAccumulator` distribute writes across cells and sum on read

What NOT to say: - "AtomicInteger uses synchronized internally" — wrong, the whole point is lock-free - "CAS is always faster than synchronized" — true at low contention, false at very high contention

Common follow-up: "What's the ABA problem and how do you handle it?"

Q66 · ThreadLocal — uses and gotchas

Asked at: Most Spring backend rounds · Razorpay · Wells Fargo

Difficulty: Medium · **Topic:** Concurrency

The question: "What is ThreadLocal? What are the typical pitfalls?"

The 30-second answer: ThreadLocal gives each thread its own copy of a variable — no sharing, no synchronization needed. Common uses: per-request context in web servers (e.g. tracing IDs), SimpleDateFormat per-thread caches. Main pitfall: in thread-pooled environments, you MUST call `remove()` at the end of each task or you leak state into the next task — and the value (often a large object) prevents GC.

Step-by-step thought process: 1. " `ThreadLocal<T>` is a holder. Each thread that calls `get()` sees its own value. `set(value)` stores into the calling thread's slot." 2. "Internally, each Thread has a `ThreadLocalMap` keyed by ThreadLocal instances. The map is per-thread, not per-ThreadLocal." 3. "Classic use: SimpleDateFormat is not thread-safe. Wrapping it in a ThreadLocal lets each thread reuse its own formatter." 4. "In Spring MVC, request-scoped data (trace ID, tenant ID, security context) lives in ThreadLocals so any code in the request can access them without passing them everywhere." 5. "Critical pitfall: thread pools reuse threads. If a request sets a ThreadLocal and doesn't `remove()` it, the next request handled by that thread sees stale data. Always clear in a finally block, or use a filter/interceptor that wraps the request."

Edge cases to mention: - ThreadLocal entries can leak memory if the ThreadLocal object is GC'd but the thread keeps the value alive in its map — `remove()` is the cure - `InheritableThreadLocal` propagates the value to child threads created from this thread — but NOT to thread pool threads - Java 21 added `ScopedValue` as a modern replacement for ThreadLocal — immutable, scoped to a block, virtual-thread friendly

What NOT to say: - "ThreadLocal is thread-safe sharing" — contradiction; it's per-thread NOT shared - "Just set it once and forget" — leaks state into pooled threads

Common follow-up: "How does ScopedValue improve on ThreadLocal?"

Q67 · Fork-Join framework — when to use it

Asked at: Goldman · Morgan Stanley · senior rounds

Difficulty: Hard · **Topic:** Concurrency

The question: "What is the Fork-Join framework and when would you use it?"

The 30-second answer: Fork-Join (Java 7) is designed for divide-and-conquer parallelism — recursive problems where you split work into halves, solve in parallel, combine results. The pool has work-stealing: idle threads steal tasks from busy threads' queues. Use it for CPU-bound recursive problems; not great for blocking I/O.

Step-by-step thought process: 1. "The framework has `ForkJoinPool`, `RecursiveTask<V>` (returns a value), and `RecursiveAction` (no return)." 2. "You implement `compute()`. If the task is small enough, solve directly. Otherwise, split into subtasks, `fork()` them, then `join()` each." 3. "Work stealing: every worker thread has its own deque. When a thread runs out of its own work, it steals from another's deque. This balances load without central coordination." 4. "`ForkJoinPool.commonPool()` is the default pool used by parallel streams. Size defaults to `Runtime.availableProcessors() - 1`." 5. "Use cases: parallel quicksort, parallel matrix operations, large array reductions. Bad fit: tasks that block on I/O — they tie up worker threads and starve other work."

Edge cases to mention: - Avoid blocking I/O in fork-join tasks — use `ManagedBlocker` if you must block, so the pool can spawn a compensating thread - Parallel streams use the common pool — a slow parallel stream can starve other parts of your app - Java 21's virtual threads are usually a better choice for I/O-heavy concurrency

What NOT to say: - "Fork-Join is for any kind of concurrency" — it's specifically for recursive CPU-bound work - "Work stealing means there's no scheduler" — there is, it's just decentralized

Common follow-up: "How does `parallelStream()` decide how many threads to use?"

Q68 · BlockingQueue family — which when?

Asked at: Razorpay · Swiggy · Walmart Labs · most backend rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "Walk me through the BlockingQueue implementations and when you'd pick each."

The 30-second answer: `ArrayBlockingQueue` = bounded, single-lock, fixed capacity.
`LinkedBlockingQueue` = optionally bounded, two-lock (separate head and tail), better throughput.
`PriorityBlockingQueue` = unbounded priority heap, no FIFO. `SynchronousQueue` = zero-capacity hand-off — every put blocks until a take. `DelayQueue` = elements become available after their delay expires.

Step-by-step thought process: 1. "For producer-consumer with bounded capacity (backpressure on producers when consumer is slow): `ArrayBlockingQueue`." 2. "For higher throughput with separate locks on head and tail, set a sensible capacity — bounded `LinkedBlockingQueue`. Unbounded is dangerous in production." 3. "For priority-ordered consumption: `PriorityBlockingQueue`. Unbounded, so still need flow control upstream." 4. "For hand-off semantics where you want zero queuing: `SynchronousQueue`."

CachedThreadPool uses it internally. 5. "For 'process this task in 5 minutes': DelayQueue. Each element must implement `Delayed` ; only elements whose delay has expired can be taken."

Edge cases to mention: - `LinkedTransferQueue` extends `LinkedBlockingQueue` with `transfer()` — block until a consumer takes the element (like `SynchronousQueue` but with optional queueing) - `take()` blocks indefinitely; `poll(timeout)` returns null if nothing arrives — almost always prefer poll with timeout in production - All `BlockingQueue` impls throw `InterruptedException` from blocking methods — propagate properly, don't swallow

What NOT to say: - "Use `LinkedBlockingQueue`, it's the most flexible" — unbounded by default is a footgun - "BlockingQueue is just a Queue with locks" — misses the timeout, take, drainTo, hand-off semantics

Common follow-up: "What happens in a `SynchronousQueue` if the producer puts and no consumer is waiting?"

Q69 · Producer-consumer with BlockingQueue — code it

Asked at: Wells Fargo · Walmart Labs · most backend rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "Implement a producer-consumer pattern using `BlockingQueue`."

The 30-second answer: One `BlockingQueue` (bounded), one or more producer threads calling `queue.put()`, one or more consumers calling `queue.take()`. `BlockingQueue` handles all the locking and waiting — your code becomes trivial. Signal shutdown via a poison pill (a sentinel object) so consumers know when to exit.

Step-by-step thought process: 1. "Pick a bounded queue — `new ArrayBlockingQueue<Order>(1000)`. Bounded means producers backpressure when consumers fall behind." 2. "Producer thread loops: build an `Order`, call `queue.put(order)`. `put` blocks if the queue is full." 3. "Consumer thread loops: `Order order = queue.take(); process(order);`. `take` blocks if the queue is empty." 4. "For graceful shutdown, the producer puts a known poison pill (or a special 'shutdown' `Order`). Consumer checks for it and exits the loop. With multiple consumers, put N pills for N consumers." 5. "Handle `InterruptedException` properly: catch, restore the interrupt flag via `Thread.currentThread().interrupt()`, then break out of the loop."

Edge cases to mention: - Don't use a sentinel value like `null` for poison pill — `BlockingQueue` rejects nulls - For multiple producers AND multiple consumers, the queue still works — it's thread-safe — but coordinating shutdown is trickier (use `AtomicInteger` countdown of remaining producers) - For unbounded throughput, consider `LinkedTransferQueue` — slightly better performance under contention

What NOT to say: - "Use `wait/notify` directly" — that's what `BlockingQueue` exists to avoid - "Make it unbounded so producers never block" — that's just moving the OOM to a different line

Common follow-up: "How would you implement backpressure if producers can't be blocked?"

Q70 · Deadlock — how to detect and prevent

Asked at: Most senior backend rounds · Razorpay · Goldman

Difficulty: Medium · **Topic:** Concurrency

The question: "How do you detect a deadlock in production? How would you prevent one?"

The 30-second answer: Detect: take a thread dump (`jstack <pid>` or `kill -3`) — the JVM reports any monitor-deadlock cycles it can detect. Prevent: acquire locks in a consistent global order, use `tryLock(timeout)` to fail fast instead of deadlocking, minimize lock scope, prefer immutable data and message-passing over shared mutable state.

Step-by-step thought process: 1. "Classic deadlock: thread A holds lock 1 and wants lock 2; thread B holds lock 2 and wants lock 1. Both wait forever." 2. "Detection in production: `jstack <pid>` prints all threads, their stacks, and any 'Found one Java-level deadlock' summary at the bottom. Modern monitoring (Datadog, NewRelic) flags this automatically." 3. "Prevention strategy 1: consistent lock ordering. If every thread that needs both locks acquires them in the same order, no cycle is possible." 4. "Prevention strategy 2: ReentrantLock with `tryLock(timeout)` — if you can't acquire the second lock in N seconds, release the first and retry." 5. "Prevention strategy 3: design out the problem. Use immutable objects, message queues, single-writer patterns. The locks you don't take can't deadlock."

Edge cases to mention: - `jstack` only detects monitor-based deadlocks (from `synchronized`) — Lock-based deadlocks (ReentrantLock) need `ThreadMXBean.findDeadlockedThreads()` to detect programmatically - Livelock is a related issue — threads keep retrying without progress; harder to detect than deadlock - Database deadlocks are separate — those happen at the DB level; the JVM thread is just waiting on a row lock

What NOT to say: - "Use synchronized on this everywhere" — doesn't prevent deadlock; encourages it - "Deadlocks resolve themselves eventually" — they don't, that's the definition

Common follow-up: "Walk me through a real deadlock you've debugged."

Q71 · Stream collectors — what's available?

Asked at: Razorpay · Swiggy · Cred · most mid-level rounds

Difficulty: Medium · **Topic:** Java 8 Functional

The question: "What collectors do you reach for most often in the Stream API?"

The 30-second answer: `toList()` , `toSet()` , `toMap(keyFn, valueFn)` , `groupingBy(keyFn)` , `partitioningBy(predicate)` , `joining(", ")` , `counting()` , `summingInt` / `averagingDouble` , `reducing` , `mapping(...)` and `filtering(...)` as

downstream collectors. Java 16 added `toUnmodifiableList/Set/Map` which return immutable results.

Step-by-step thought process: 1. "`Collectors.toList()` is the workhorse — collect a stream into a List. Java 16 added `Stream.toList()` directly which returns unmodifiable; `Collectors.toList()` returns `ArrayList`." 2. "`toMap(keyFn, valueFn)` — careful: throws on duplicate keys. Use the 3-arg overload with a merge function: `toMap(Order::getId, Function.identity(), (a, b) -> a)` to keep first or merge." 3. "`groupingBy(Order::getStatus)` returns `Map<Status, List<Order>>`. Pass a downstream collector to transform the value: `groupingBy(Order::getStatus, counting())` returns `Map<Status, Long>`." 4. "`partitioningBy(Order::isPaid)` returns `Map<Boolean, List<Order>>` — split into two buckets. Often clearer than `groupingBy` when the key is binary." 5. "`joining(", ", "[", "]")` for string joining with separator + prefix + suffix. Beats `StringBuilder` manually."

Edge cases to mention: - `Collectors.toMap` with duplicate keys throws `IllegalStateException` — always pass a merge function if duplicates are possible - `groupingBy` returns a `HashMap` by default; use the 3-arg overload to get a `TreeMap` (`groupingBy(keyFn, TreeMap::new, downstream)`) - For null keys, `groupingBy` accepts them but `Collectors.toMap` does not — common surprise

What NOT to say: - "Collectors are just for collecting into lists" — they're a mini-DSL for terminal reductions - "Use `forEach` and a manual map" — defeats the point of streams

Common follow-up: "How would you group orders by status and sum amount within each group?"

Q72 · Parallel streams — when do they help?

Asked at: Razorpay · Swiggy · most product co backend rounds

Difficulty: Medium-Hard · **Topic:** Java 8 Functional

The question: "When should you actually use a parallel stream? When would it hurt?"

The 30-second answer: Parallel streams help when: (1) the data set is large enough for splitting to amortize overhead, (2) the per-element work is CPU-bound and stateless, (3) the source splits cheaply (`ArrayList` yes, `LinkedList` no). They hurt for: small datasets, blocking I/O, contended shared state, or when you're already inside another parallel context.

Step-by-step thought process: 1. "Parallel streams use `ForkJoinPool.commonPool()` by default — same pool that other parts of your app may use. A slow parallel stream starves them." 2. "For meaningful speedup, you need: $N \gg \text{threads} \times \text{overhead}$. As a rule of thumb, ~10,000 elements with non-trivial work per element is the breakeven for many workloads." 3. "Splittable sources matter: `ArrayList`, arrays, `IntStream.range` — all split cheaply ($O(1)$ bisection). `LinkedList` must walk to find the midpoint — kills parallelism." 4. "Blocking I/O in a parallel stream is a disaster — you tie up the common pool, your app's other parallel work stalls. Use `CompletableFuture` or virtual threads for I/O concurrency

instead." 5. "Stateful operations (sorted on a non-comparable, limit on an unordered stream) make parallel streams either slower or buggy — read the Stream API contract."

Edge cases to mention: - `.parallel()` doesn't preserve order in unordered streams; use `.forEachOrdered()` if you need encounter order - Java 21 virtual threads do NOT help with parallel streams (parallel streams are CPU-parallel, not I/O-parallel) - Use `ManagedBlocker` if you absolutely must block in a parallel stream — but really, prefer `CompletableFuture`

What NOT to say: - "Always use parallel streams for performance" — usually slower for small data and I/O-bound work - "Parallel streams use threads I can configure" — they use the common pool, period (without ugly hacks)

Common follow-up: "Why does using parallel stream with database calls usually slow things down?"

Q73 · Optional — proper use and abuse

Asked at: Most modern Java rounds · Razorpay · Swiggy · Walmart Labs

Difficulty: Easy-Medium · **Topic:** Java 8 Functional

The question: "What is Optional intended for? What are common misuses?"

The 30-second answer: Optional is a return type for methods that may have no result — it forces the caller to handle the absence explicitly. Use it as a return type. Don't use it as a field, parameter, or in collections — it adds overhead and the spec explicitly warns against it.

Step-by-step thought process: 1. "Original intent (from the JDK designers): return type for methods like `findFirst()`, `stream().max()` — anywhere you'd otherwise return null." 2. "`Optional.empty()` represents absence. `Optional.of(value)` requires non-null. `Optional.ofNullable(maybe)` accepts null and produces empty." 3. "Consume via `ifPresent(x -> ...)`, `orElse(default)`, `orElseGet(supplier)`, `orElseThrow()`, `map / flatMap`. The whole point is no `.get()` without a presence check." 4. "Don't use as a field — Optional isn't Serializable, adds an allocation per field, and tools/frameworks (Jackson, JPA) don't always play well." 5. "Don't use as a method parameter — overloading or just passing null is clearer. Don't put inside a collection — `List<Optional<T>>` is a code smell; use a different structure."

Edge cases to mention: - `Optional.get()` without a presence check throws `NoSuchElementException` — same bug as a `NullPointerException` with extra steps - `orElse(expensiveValue)` always evaluates the default; use `orElseGet(() -> ...)` for lazy evaluation - For primitive types, `OptionalInt`, `OptionalLong`, `OptionalDouble` exist — avoid `Optional<Integer>` boxing

What NOT to say: - "Optional is a null replacement everywhere" — explicitly discouraged in the API docs - "Use Optional in fields for null safety" — adds overhead, breaks frameworks

Common follow-up: "What's the difference between `orElse` and `orElseGet`?"

Q74 · Method references — when more readable than lambdas?

Asked at: Most Java mid-level rounds

Difficulty: Easy · **Topic:** Java 8 Functional

The question: "When would you use a method reference instead of a lambda?"

The 30-second answer: Use a method reference when the lambda is just a single method call passing its arguments through unchanged — `s -> s.toUpperCase()` becomes `String::toUpperCase`. It's shorter and signals "this is a direct call, no extra logic". When you need to transform arguments or add logic, stick with the lambda.

Step-by-step thought process: 1. "Four forms: `ClassName::staticMethod` (e.g. `Integer::parseInt`), `ClassName::instanceMethod` (e.g. `String::length`), `instance::method` (e.g. `out::println`), `ClassName::new` (constructor reference)." 2. "Method references shine when the lambda is literally just a delegation: `.map(s -> s.toUpperCase())` is noise compared to `.map(String::toUpperCase)`." 3. "They lose clarity when context matters: `customers.forEach(this::process)` is fine, but `customers.forEach(c -> process(c, currentTime))` can't be a reference because of the second argument." 4. "Constructor references are useful in factories — `stream.map(Order::new)` turns a stream of inputs into a stream of new Orders." 5. "Don't over-apply: if `c -> c.getName().toLowerCase()` is doing two things, leaving it as a lambda may be clearer than splitting into helper methods just to get a reference."

Edge cases to mention: - For overloaded methods, the compiler picks based on the functional interface's signature — sometimes ambiguous, then you must use a lambda - Bound method references (`instance::method`) capture the instance at the point of reference — be careful with mutable state - For streams over arrays, `Arrays::asList` and `Arrays::stream` are constructor-like references

What NOT to say: - "Always replace lambdas with method references" — overreach; sometimes lambda is clearer - "Method references are faster" — same compiled bytecode in most cases

Common follow-up: "What's the difference between `String::length` and `s -> s.length()` at the bytecode level?"

Q75 · reduce() vs collect() — when each?

Asked at: Senior rounds · Razorpay · Swiggy

Difficulty: Medium-Hard · **Topic:** Java 8 Functional

The question: "What's the difference between `Stream.reduce` and `Stream.collect`? When would you use each?"

The 30-second answer: `reduce` produces a single immutable result from associative combination — sum, product, max. `collect` produces a mutable container (List, Map, StringBuilder) via a supplier + accumulator + combiner. Use reduce for value reductions; use collect for building containers.

Step-by-step thought process: 1. "`reduce(identity, accumulator)` — repeatedly applies the accumulator (BinaryOperator) starting from identity. Result type matches the stream's element type." 2. "`reduce(identity, accumulator, combiner)` — three-arg form for streams where the result type differs (and for parallel correctness)." 3. "`collect(supplier, accumulator, combiner)` — the low-level form. Supplier creates the container, accumulator adds an element, combiner merges two partial containers (for parallel)." 4. "`collect(Collector)` — the high-level form. Collector is a wrapper bundling supplier + accumulator + combiner + finisher. The `Collectors.*` factory has the common ones." 5. "Practical rule: if I'm summing or maxing or producing a single value, `reduce`. If I'm building a List/Map/String/joined output, `collect`."

Edge cases to mention: - For parallel reduce, the operation MUST be associative — `a op b op c` must equal `(a op b) op c` equal `a op (b op c)` - `reduce` without identity returns `Optional<T>` — the stream could be empty - `collect` is the only way to perform mutable reduction safely in parallel — never accumulate into a shared mutable collection inside reduce

What NOT to say: - "reduce and collect are interchangeable" — they have different semantics and parallel behaviors - "reduce works for any combination" — only for associative operations in parallel

Common follow-up: "Why is reduce dangerous if your combiner isn't associative?"

Q76 · Functional interfaces — which built-in to use?

Asked at: Most Java mid-level rounds

Difficulty: Easy-Medium · **Topic:** Java 8 Functional

The question: "What are the main functional interfaces in `java.util.function`? When would you pick each?"

The 30-second answer: `Function<T,R>` takes T, returns R. `Predicate<T>` takes T, returns boolean. `Consumer<T>` takes T, returns nothing (side effect). `Supplier<T>` takes nothing, returns T. `BiFunction<T,U,R>`, `BiPredicate`, `BiConsumer` for two args. `UnaryOperator<T>` and `BinaryOperator<T>` for same-type-in-same-type-out.

Step-by-step thought process: 1. "Function — transformations. `Function<Order, BigDecimal> getAmount = Order::getAmount;`" 2. "Predicate — filtering, testing. `Predicate<Order> isPaid = o -> o.getStatus() == PAID;` then `stream.filter(isPaid)`" 3. "Consumer — side effects, especially in `forEach`. `Consumer<String> log = System.out::println;`" 4. "Supplier — lazy generators. `Supplier<UUID> ids = UUID::randomUUID;`. Used in `Optional.orElseGet(supplier)`" 5. "For primitives, there are specialized versions to avoid boxing: `IntPredicate`, `ToIntFunction<T>`, `IntFunction<R>`, `ObjIntConsumer<T>`, etc. Use these in hot paths."

Edge cases to mention: - Predicate has `and`, `or`, `negate` for composing — `isPaid.and(isLarge)` is cleaner than nested lambdas - Function has `andThen` and `compose` — composable transformations - For three or more args, no built-in TriFunction — you'd define your own

What NOT to say: - "Just use Function for everything" — losing intent and forcing boxing - "I write my own functional interfaces" — overkill 95% of the time; the built-ins cover what you need

Common follow-up: "What's the difference between `Function.compose` and `Function.andThen`?"

Q77 · Stream lazy evaluation — what does it mean?

Asked at: Razorpay · Swiggy · most product co rounds

Difficulty: Medium · **Topic:** Java 8 Functional

The question: "What does it mean that streams are lazy? Why does it matter?"

The 30-second answer: Intermediate operations (`map`, `filter`, `sorted`, `limit`) don't run when called — they build up a pipeline. The pipeline only executes when you call a terminal operation (`collect`, `forEach`, `count`, etc.). This enables short-circuiting and fusion — `limit(10)` after a filter only processes elements until 10 pass.

Step-by-step thought process: 1. "`stream.filter(p).map(f).collect(toList())` — when you call filter, nothing runs. Same for map. The pipeline is built but inert." 2. "`collect` is terminal. It pulls elements one by one through the pipeline: each element flows through filter, then map, then into the collector." 3. "Short-circuiting: `findFirst`, `limit`, `anyMatch`, `allMatch`, `noneMatch` can stop early. `stream.filter(p).findFirst()` stops at the first element that passes `p` — doesn't process the rest." 4. "Fusion: the JIT can often fold multiple intermediate ops into a single loop, avoiding intermediate collections." 5. "A stream can only be consumed once — calling `count()` then `collect()` on the same stream throws `IllegalStateException`. Recreate the stream if you need to traverse twice."

Edge cases to mention: - Infinite streams (`Stream.iterate(0, i -> i+1)`) only work because of laziness — must use a short-circuiting op like `limit(n)` to terminate - Stateful intermediate ops (`sorted`, `distinct`) buffer elements — break the lazy property for the rest of the pipeline - Side effects in intermediate ops are evaluated unpredictably if at all — never put state-mutating code in `peek` or `map`

What NOT to say: - "Streams just iterate over a collection" — misses the entire execution model - "Lazy means asynchronous" — it doesn't, lazy and async are different

Common follow-up: "What happens if you call `.filter().peek().filter().collect()`? When does `peek` run?"

Q78 · Spring Bean lifecycle — full picture

Asked at: Most Spring backend rounds · Razorpay · Swiggy · Walmart

Difficulty: Medium · **Topic:** Spring Core

The question: "Walk me through the full lifecycle of a Spring bean."

The 30-second answer: Instantiation → populate properties (dependency injection) → `*Aware` callbacks (`BeanNameAware`, `ApplicationContextAware`) → `BeanPostProcessor.before` → `@PostConstruct` and `InitializingBean.afterPropertiesSet` → custom init method → `BeanPostProcessor.after` → bean is ready to serve → on shutdown: `@PreDestroy` → `DisposableBean.destroy` → custom destroy method.

Step-by-step thought process: 1. "Container loads bean definitions (XML, annotations, Java config). Creates the bean — constructor injection happens here. Singleton beans are typically eagerly created at startup." 2. "Field/setter dependencies are populated next. Aware interfaces let beans get context references (`BeanName`, `BeanFactory`, `ApplicationContext`)." 3. "BeanPostProcessors get a chance to wrap the bean — this is how AOP proxies (`@Transactional`, `@Async`) get inserted. `postProcessBeforeInitialization` runs before init callbacks." 4. "Initialization callbacks run in order: `@PostConstruct` (JSR-250), then `InitializingBean.afterPropertiesSet`, then any custom init method declared in `@Bean(initMethod=...)`." 5. " `postProcessAfterInitialization` runs — this is where AOP proxies are commonly wired in. Then the bean is in the ready-to-use pool." 6. "On shutdown: `@PreDestroy`, then `DisposableBean.destroy`, then custom destroy method."*

Edge cases to mention: - Prototype beans don't get destroy callbacks — Spring instantiates them but doesn't track them after handoff - `@PostConstruct` runs AFTER all injection is complete — that's why field-injected dependencies are safe to use there but not in the constructor - A bean throwing in `afterPropertiesSet` aborts container startup

What NOT to say: - "Beans are just created with new" — misses Spring's entire value proposition - "@PostConstruct is the constructor" — wrong, it runs after constructor + injection

Common follow-up: "When does the AOP proxy get applied — before or after `@PostConstruct`?"

Q79 · @Autowired vs @Inject vs @Resource

Asked at: Most Spring rounds · TCS · Infosys · Wipro · Cognizant

Difficulty: Easy-Medium · **Topic:** Spring Core

The question: "What's the difference between `@Autowired`, `@Inject`, and `@Resource`?"

The 30-second answer: `@Autowired` is Spring-specific; injects by type by default. `@Inject` is JSR-330 (`jakarta.inject`); also by type. `@Resource` is JSR-250; injects by NAME first, then by type. Most Spring projects use `@Autowired`; `@Inject` is portable across DI containers; `@Resource` gives you name-based injection for free.

Step-by-step thought process: 1. "`@Autowired` — Spring's own. Default `required=true`. Combine with `@Qualifier("beanName")` when multiple candidates exist." 2. "`@Inject` (from `jakarta.inject` since Spring 6 / Spring Boot 3) — standard JSR-330. Same type-first behavior. No `required=false` equivalent; use `Optional<T>` instead." 3. "`@Resource` (from `jakarta.annotation`) — JSR-250. If `name` attribute is set, uses that bean name. Otherwise, by field/parameter name, falling back to type." 4. "For new code: prefer constructor injection (no annotation needed since Spring 4.3 with single constructor) over field injection. Testability + immutability." 5. "Concrete: `@Autowired` on the field is fine for legacy code, but `private final PaymentService paymentService;` injected via constructor is what I write in new code."

Edge cases to mention: - When multiple beans match, `@Autowired` fails unless one is `@Primary` or you add `@Qualifier` - `@Resource` without `name` falls back to field name — confusing if you rename the field - Constructor injection makes the dependency required and the field final — both wins

What NOT to say: - "All three are basically the same" — name-vs-type resolution actually matters - "Use field injection because it's shorter" — kills testability

Common follow-up: "Why is constructor injection preferred over field injection?"

Q80 · @Component vs @Service vs @Repository vs @Controller

Asked at: TCS · Infosys · most Spring rounds

Difficulty: Easy · **Topic:** Spring Core

The question: "What's the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?"

The 30-second answer: All four are stereotypes — they register the class as a Spring bean.

`@Component` is the generic one. `@Service`, `@Repository`, `@Controller` are semantic specializations.

`@Repository` adds automatic JPA/JDBC exception translation. `@Controller` (and `@RestController`) are recognized by Spring MVC's handler mapping. Functionally, `@Service` is equivalent to `@Component`.

Step-by-step thought process: 1. "All four are meta-annotated with `@Component`. Spring's component scan picks them all up the same way." 2. "`@Service` is a marker for business-logic beans. Functionally identical to `@Component`. The annotation exists for clarity in code reviews and architecture diagrams." 3. "`@Repository` adds value: a `BeanPostProcessor` (`PersistenceExceptionTranslationPostProcessor`) wraps the bean so any Hibernate/JDBC native exceptions become Spring's `DataAccessException` hierarchy." 4. "`@Controller` is recognized by Spring MVC — methods annotated with `@RequestMapping` / `@GetMapping` become URL handlers." 5. "`@RestController` = `@Controller` + `@ResponseBody` on every method — returns are serialized to JSON instead of treated as view names."

Edge cases to mention: - You can write custom stereotypes — meta-annotate your own annotation with `@Component` - If you skip the stereotype and use only `@Bean` in a `@Configuration` class, the class isn't a stereotype but the returned object is a bean - For multi-module apps, set `basePackages` on `@ComponentScan` or `@SpringBootApplication` to avoid scanning the world

What NOT to say: - "They're all the same" — misses @Repository's exception translation - "@Service is for services so it runs on a different thread" — completely wrong

Common follow-up: "What does @Repository actually add over @Component?"

Q81 · @Transactional propagation — full breakdown

Asked at: Razorpay · Swiggy · Cred · most product co backend rounds

Difficulty: Hard · **Topic:** Spring Core

The question: "Explain @Transactional propagation modes. When would you use REQUIRES_NEW vs REQUIRED vs NESTED?"

The 30-second answer: REQUIRED (default) — join existing transaction or start one. REQUIRES_NEW — suspend the outer transaction and start a new one. NESTED — use a savepoint in the outer transaction. MANDATORY — must be inside an existing transaction. NEVER — must NOT be in a transaction. SUPPORTS — use one if present. NOT_SUPPORTED — suspend any existing transaction.

Step-by-step thought process: 1. "REQUIRED is the workhorse. Same behavior whether called standalone or nested — caller's commit/rollback wins." 2. "REQUIRES_NEW for audit logs and notifications — even if the outer business transaction rolls back, we still want the audit row committed. Watch out: the outer transaction is suspended; if the inner one deadlocks waiting for resources held by the outer, you're stuck." 3. "NESTED uses a JDBC savepoint. Inner rollback rolls back to the savepoint; outer commit/rollback applies to everything. Only works with JDBC; not supported by JTA." 4. "MANDATORY is a defensive check — 'fail loudly if I'm called without a transaction' for repository-layer methods that must be in one." 5. "NEVER and NOT_SUPPORTED for read-only operations that shouldn't be in a transaction (long reports, calls to external systems)."

Edge cases to mention: - @Transactional only works on PUBLIC methods called THROUGH a Spring proxy — internal `this.method()` calls bypass the proxy and bypass the transaction. Self-invocation gotcha - REQUIRES_NEW requires a new connection from the pool — if your pool is small and the outer transaction also holds one, you can deadlock the pool - Default rollback rule: rolls back on RuntimeException only. Checked exceptions DON'T roll back unless you specify `rollbackFor`

What NOT to say: - "Just use REQUIRED everywhere" — sometimes you genuinely need REQUIRES_NEW for audit/notification semantics - "Spring handles all the connections automatically" — true at runtime, but you must understand the model to avoid pool deadlocks

Common follow-up: "What happens if a method annotated @Transactional calls another @Transactional method on the same bean?"

Q82 · Spring profiles and conditional beans

Asked at: Most Spring Boot rounds · Razorpay · Walmart Labs

Difficulty: Medium · **Topic:** Spring Boot

The question: "How do Spring profiles work, and when would you use `@ConditionalOn...` annotations?"

The 30-second answer: Profiles are named configuration sets activated via

`spring.profiles.active=prod` (or env var `SPRING_PROFILES_ACTIVE`). Beans/configs annotated with `@Profile("prod")` only register when that profile is active. `@ConditionalOn...` goes deeper — register a bean only if a property is set, a class is on the classpath, another bean exists, etc.

Step-by-step thought process: 1. "`@Profile("dev")` on a `@Configuration` or `@Bean` — only loaded when 'dev' profile is active. Multiple profiles supported: `@Profile({"dev", "test"})` ." 2.

"Activation: `spring.profiles.active=prod` in `application.properties`, command line `-Dspring.profiles.active=prod`, or env var `SPRING_PROFILES_ACTIVE=prod` ." 3. "Profile-specific config files: `application-prod.yml` is loaded automatically when 'prod' is active, overriding values from `application.yml` ." 4. "`@ConditionalOnProperty` — load only if a property has a specific value. Useful for feature flags: `@ConditionalOnProperty("feature.new-checkout.enabled")` ." 5. "`@ConditionalOnClass` / `@ConditionalOnMissingBean` — Spring Boot's autoconfiguration is built on these. 'If Tomcat is on the classpath, configure embedded Tomcat'."

Edge cases to mention: - `@Profile("!prod")` negates — load when 'prod' is NOT active - Beans without `@Profile` load in all profiles — be explicit if a bean is environment-specific -

`@ConditionalOnMissingBean` is evaluated AFTER all user beans are processed — used for sensible defaults that user code can override

What NOT to say: - "Use system properties for environment switching" — fragile compared to profiles - "`@Profile` is just a comment" — it's evaluated at bean registration time

Common follow-up: "How does Spring Boot's autoconfiguration use `@Conditional` under the hood?"

Q83 · Spring Boot autoconfiguration — how does it work?

Asked at: Most Spring Boot rounds · Razorpay · Swiggy · Wells Fargo

Difficulty: Medium-Hard · **Topic:** Spring Boot

The question: "How does Spring Boot's autoconfiguration mechanism work?"

The 30-second answer: At startup, Spring Boot reads `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (was `spring.factories` before Boot 3) to find all autoconfiguration classes. Each is a `@Configuration` with `@ConditionalOn...` annotations that decide whether it applies based on classpath, properties, and existing beans. Run `--debug` flag to see what got configured and why.

Step-by-step thought process: 1. "`@SpringBootApplication` includes `@EnableAutoConfiguration`. That triggers the `AutoConfigurationImportSelector` to read the imports file." 2. "Each entry is a class like `DataSourceAutoConfiguration`. It's a `@Configuration` class loaded with all its `@ConditionalOn...` guards." 3. "Common guards: `@ConditionalOnClass(DataSource.class)` — only run if `DataSource` is on classpath. `@ConditionalOnMissingBean` — only register if user hasn't defined one. `@ConditionalOnProperty` — only if a property is set." 4. "Result: include `spring-boot-starter-data-jpa`, get JPA + Hibernate + a `DataSource` configured from your properties. Add your own `@Bean DataSource` and Spring Boot backs off." 5. "Debugging: `--debug` flag prints the autoconfiguration report — every condition's positive and negative matches. The actuator `/conditions` endpoint shows the same in production."

Edge cases to mention: - Boot 3 changed from `spring.factories` to `AutoConfiguration.imports` — older tutorials are outdated - `@EnableAutoConfiguration`'s `exclude` lets you remove a specific config — useful when a starter brings in something you don't want - Custom starters: build your own autoconfiguration by writing your own `META-INF/spring/...imports` file

What NOT to say: - "Spring Boot just guesses what you want" — it's a deterministic system of conditions, not magic - "Autoconfiguration runs every request" — it runs at startup only

Common follow-up: "How would you write your own custom autoconfiguration?"

Q84 · Spring scopes — singleton vs prototype vs request

Asked at: Most Spring rounds

Difficulty: Easy-Medium · **Topic:** Spring Core

The question: "What scopes does Spring support and when would you use each?"

The 30-second answer: singleton (default) — one instance per container. prototype — new instance every time the bean is requested. request — one per HTTP request. session — one per HTTP session. application — one per `ServletContext`. websocket — one per `WebSocket` session. Almost everything is singleton in practice.

Step-by-step thought process: 1. "Singleton — the default. Container creates one instance, returns it everywhere. Stateless services, repositories, controllers — all singleton." 2. "Prototype — new instance every time `getBean()` or `@Autowired` is resolved. Use for stateful beans that need fresh state per use." 3. "Request — one instance per HTTP request. Useful for holding request context (current user, locale). Web-only." 4. "Session — one per HTTP session. Use for user-specific state. Beware: session beans get serialized if you have session replication." 5. "Critical gotcha: injecting a prototype/request bean into a singleton — the singleton captures one instance forever. Use `ObjectProvider<T>`, `Provider<T>`, or scoped proxy (`@Scope(value=..., proxyMode=TARGET_CLASS)`) to get a fresh instance per call."

Edge cases to mention: - Singleton scope in Spring is per-container, not JVM-wide — different from the singleton design pattern - Prototype beans don't get destruction callbacks — Spring drops the reference after handoff - For thread-per-request models with virtual threads, request scope still works (Spring's request context propagates)

What NOT to say: - "Spring singletons are like the singleton design pattern" — close but not identical, mention per-container - "Prototype beans are thread-safe" — depends on the bean; scope doesn't grant thread safety

Common follow-up: *"How do you safely inject a prototype bean into a singleton?"*

Q85 · Heap structure — Young Gen, Old Gen, Metaspace

Asked at: Senior backend rounds · Razorpay · Goldman · Walmart

Difficulty: Medium · **Topic:** JVM internals

The question: *"Walk me through the structure of the JVM heap in modern Java."*

The 30-second answer: Heap splits into Young Generation (Eden + two Survivor spaces, S0 and S1) and Old Generation. New objects go to Eden. After a minor GC, surviving objects move to a Survivor space; after several minor GCs (typically 15), they're promoted to Old Gen. Metaspace (off-heap since Java 8) holds class metadata. Large objects can be allocated directly in Old Gen.

Step-by-step thought process: 1. *"Young Gen is where new allocation happens. Eden is the largest part; S0 and S1 are the two survivor spaces."* 2. *"Minor GC (Young Gen collection): live objects in Eden + one survivor move to the other survivor. Eden and the source survivor become empty."* 3. *"Tenuring threshold (default 15 minor GCs survived) — objects that keep surviving get promoted to Old Gen. The assumption: most objects die young."* 4. *"Old Gen GC is more expensive — full GCs touch the whole heap and historically were stop-the-world. Modern collectors (G1, ZGC, Shenandoah) make this concurrent."* 5. *"Metaspace (off-heap, replaced PermGen in Java 8) holds class definitions. Grows dynamically — set `-XX:MaxMetaspaceSize` if you don't want it unbounded."*

Edge cases to mention: - TLAB (Thread-Local Allocation Buffer) — each thread gets a private region in Eden so allocation is just a pointer bump, no synchronization - Humongous objects (larger than half a G1 region) go straight to Old Gen - ZGC and Shenandoah are concurrent low-latency collectors — they target sub-10ms pauses on heaps up to TBs

What NOT to say: - "There's also PermGen" — removed in Java 8 - "Old Gen is collected by every GC" — minor GC only touches Young Gen

Common follow-up: *"What's the difference between a minor GC and a major GC?"*

Q86 · GC algorithms — G1 vs ZGC vs Parallel

Asked at: Senior backend rounds · Razorpay · Goldman · Walmart Labs

Difficulty: Hard · **Topic:** JVM internals

The question: "Walk me through the major GC algorithms. When would you pick each?"

The 30-second answer: Serial GC — single-threaded, small heaps, simple apps. Parallel GC — throughput-focused, multi-threaded for both young and old. G1 GC (default since Java 9) — region-based, predictable pause targets, default for most apps. ZGC — concurrent, sub-millisecond pauses, multi-TB heaps. Shenandoah — similar to ZGC, Red Hat-led. For modern Java, G1 by default, ZGC when latency matters more than throughput.

Step-by-step thought process: 1. "Serial GC (`-XX:+UseSerialGC`) — single-threaded mark-sweep-compact. Smallest footprint. Containers with one CPU and ~100MB heap." 2. "Parallel GC (`-XX:+UseParallelGC`) — multi-threaded but still stop-the-world. Maximizes throughput; long pauses on big heaps. Was the default through Java 8." 3. "G1 (`-XX:+UseG1GC` , default since Java 9) — splits heap into ~2000 regions, each can be Eden/Survivor/Old/Humongous. Predictable pause goal via `-XX:MaxGCPauseMillis` ." 4. "ZGC (`-XX:+UseZGC` , production-ready since Java 15, generational in Java 21+, default-generational in Java 23) — concurrent collector with sub-10ms pauses (often sub-millisecond). For latency-sensitive services with multi-GB to TB heaps." 5. "Shenandoah — Red Hat's concurrent collector. Similar profile to ZGC. Often available in OpenJDK distributions."

Edge cases to mention: - Generational ZGC (Java 21 JEP 439, default in Java 23 JEP 474) — major throughput improvement over single-gen ZGC - For containerized apps, the JVM auto-detects container memory limits since Java 10 — set heap with `-XX:MaxRAMPercentage` rather than fixed `-Xmx` - Tail latency matters more than average latency for user-facing APIs — pick ZGC over G1 if p99 matters

What NOT to say: - "CMS is still recommended" — removed in Java 14 - "Just use defaults" — fine for many apps, but knowing why the default fits helps in interviews

Common follow-up: "What's the trade-off between throughput and latency in GC choice?"

Q87 · JIT compilation tiers — C1 and C2

Asked at: Senior rounds · Goldman · JP Morgan · Morgan Stanley

Difficulty: Hard · **Topic:** JVM internals

The question: "How does the JIT compiler work in HotSpot? What's tiered compilation?"

The 30-second answer: HotSpot runs bytecode through an interpreter first. Hot methods get compiled by C1 (client compiler) for quick startup, then re-compiled by C2 (server compiler) for peak performance. Tiered compilation (default since Java 8) progresses through 5 levels — interpreter → C1 with profiling → C2 — collecting profile data along the way so C2 can make aggressive speculative optimizations.

Step-by-step thought process: 1. "Interpretation is slow but starts immediately. Methods get an invocation counter — once it crosses a threshold (10K by default in tiered), the method is queued for compilation." 2. "C1 (client compiler) — fast compilation, modest optimization. Produces decent code quickly. Adds profile data collection (branch counts, type profiles)." 3. "C2 (server compiler) — slower compilation, aggressive optimization. Uses profile data from C1 to inline hot calls, eliminate dead branches, speculate on types." 4. "Speculative optimizations can be wrong — C2 inserts **uncommon traps**. When a trap fires (e.g. a previously-unseen type), execution falls back to the interpreter and the method is re-profiled." 5. "GraalVM and the Graal compiler are alternative C2 replacements written in Java itself — sometimes faster on certain workloads."

Edge cases to mention: - AOT compilation (Java 9 jaotc, removed in Java 17; modern alternative is GraalVM native-image) compiles ahead of time — no warmup, but no speculative optimizations either - JIT compilation happens on background threads — see warmup behavior in microbenchmarks - Use JMH for benchmarking — it handles warmup correctly so you measure peak JIT performance

What NOT to say: - "Java is interpreted" — true initially, false after warmup - "C1 and C2 do the same thing" — they have very different optimization strategies

Common follow-up: "What's a deoptimization trap and when does it happen?"

Q88 · Class loaders — delegation model

Asked at: Most Spring rounds · Razorpay · Goldman

Difficulty: Medium-Hard · **Topic:** JVM internals

The question: "Explain the class loader hierarchy and the parent-first delegation model."

The 30-second answer: Bootstrap loader (native, loads **java.*** from rt.jar / modules) → Platform/Extension loader (loads **java.sql.***, etc.) → Application/System loader (loads classpath classes). Parent-first delegation: every loader asks its parent first; only loads itself if parent can't find the class. This prevents your code from shadowing core classes.

Step-by-step thought process: 1. "Three (or more) loaders form a chain. Bootstrap is the root — written in C++, loads core JDK classes." 2. "Platform loader (Java 9+) — loads platform modules like java.sql, java.xml. Was called the extension loader pre-Java 9 with a different scope." 3. "Application/System loader — loads your classpath and module path. This is what **Class.forName(name)** uses by default." 4. "Delegation: when asked to load **com.merchant.Order**, the app loader first asks its parent (platform), which asks bootstrap. Bootstrap doesn't find it, returns null. Platform doesn't find it, returns null. App loader then searches its classpath." 5. "Why parent-first? Security — if you put a **java.lang.String** on the classpath, your version is ignored because bootstrap loads its String first."

Edge cases to mention: - App servers (Tomcat, WebLogic) use NON-standard child-first loaders for web apps — your app's classes shadow the server's. That's why version conflicts get weird - Each loader has its own namespace — **com.foo.Bar** loaded by loader A and the same class loaded by loader B are

DIFFERENT classes, can't cast between them - OSGi and JPMS modules add their own layering on top of class loading

What NOT to say: - "There's just one class loader" — depends on the app, but rarely true - "Class loading is single-threaded" — it's thread-safe but multi-threaded

Common follow-up: *"What's a ClassCastException across class loaders and how do you debug it?"*

Q89 · Memory leaks in Java — common causes

Asked at: Senior backend rounds · Razorpay · Goldman

Difficulty: Medium-Hard · **Topic:** JVM internals

The question: *"What are the common causes of memory leaks in a long-running Java app?"*

The 30-second answer: ThreadLocal not removed (lingers in pool threads), static collections that grow unbounded (caches, listeners), unclosed resources (streams, connections), inner classes holding outer references, classloader leaks in hot-deploy environments, listener registrations never deregistered.

Step-by-step thought process: 1. "Java has GC, but 'leak' means reachable objects that should no longer be reachable. Common cause: a long-lived object holding a reference to something short-lived." 2. "Static caches — `static Map<String, Result> cache = new HashMap<>();` and you never evict. Use `Caffeine` or `Guava` cache with size or time eviction instead." 3. "ThreadLocal — in a thread pool, if you set a ThreadLocal and never remove it, the value lives as long as the thread does. Add a `finally { tl.remove(); }`." 4. "Listener registrations — `EventBus.register(this)` in a short-lived object that's never deregistered. The bus holds a strong reference forever." 5. "Classloader leaks — hot-deploy app server unloads the webapp, but a `Thread` started by your code still holds a reference to the old classloader. Whole old classloader graph leaks."

Edge cases to mention: - Use `jcmd <pid> GC.heap_dump <path>` then analyze with Eclipse MAT for production heap dumps - `WeakReference` / `SoftReference` / `PhantomReference` give you control when caches must be GC-aware - Native memory leaks (NIO ByteBuffer, JNI) don't show up in heap dumps — use Native Memory Tracking (`-XX:NativeMemoryTracking=summary`)

What NOT to say: - "Java has GC, so there are no leaks" — naive - "Just call `System.gc()` more often" — doesn't fix leaks

Common follow-up: *"You've got a heap growing 100MB per hour. How do you debug it?"*

Q90 · String pool and interning

Asked at: Most Java mid-level rounds

Difficulty: Medium · **Topic:** JVM internals / Strings

The question: "What is the String pool and what does `.intern()` do?"

The 30-second answer: The String pool is a JVM-managed cache (in heap since Java 7) of String literals — `"hello"` in source code goes there automatically. `.intern()` puts a programmatically-created String into the pool (or returns the existing pooled instance if one matches). Two interned identical Strings can be compared with `==`.

Step-by-step thought process: 1. "At class load, every String literal in your source becomes an entry in the pool. `String a = "hi"; String b = "hi";` — both reference the same pooled String. `a == b` is true." 2. "`new String("hi")` creates a new object on the heap, distinct from the pooled one. `a == new String("hi")` is false." 3. "`.intern()` says 'either give me the pooled String equal to this one, or add this one to the pool and return it'. After interning, `==` works." 4. "The pool was in PermGen pre-Java 7, in main heap since. Default pool table size 60013, tunable via `-XX:StringTableSize`." 5. "When is interning worth it? When you have lots of duplicate strings in memory (e.g. parsed log lines, repeated dictionary keys). Saves memory but costs CPU on each intern call."

Edge cases to mention: - `==` on Strings is almost always wrong — use `.equals()`. Only exception: comparing known-interned values like enum names - Java 9 introduced compact strings (`byte[]` instead of `char[]` for ASCII-only) — independent of the pool but cuts String memory by ~50% - For high-throughput interning, consider a `ConcurrentHashMap`-based custom intern rather than `String.intern()` — JDK's intern table is global

What NOT to say: - "All Strings live in the pool" — only literals and intern'd ones; `new String` does not - "Intern is free" — has CPU cost; profile before sprinkling everywhere

Common follow-up: "What's the difference between `String a = \"x\" + \"y\"` and `String a = \"xy\"`?"

Q91 · Composition over inheritance — why?

Asked at: Most Java mid-level rounds

Difficulty: Medium · **Topic:** OOP

The question: "What does 'favor composition over inheritance' mean and why?"

The 30-second answer: Inheritance creates tight coupling — subclass depends on superclass internals, breaks if the parent changes. Composition holds another object as a field and delegates to it — looser coupling, more flexible (you can swap the composed object). The classic example: a stack should HAVE-A list, not BE-A list, otherwise it inherits methods it shouldn't expose.

Step-by-step thought process: 1. "Inheritance is `is-a`. Composition is `has-a`. Misusing inheritance leaks superclass methods you don't want." 2. "Concrete: if `Stack extends ArrayList`, your stack accidentally exposes `add(index, element)` and `remove(index)` — callers can corrupt stack semantics." 3. "Composition: `class Stack { private final List<E> items = new ArrayList<>(); void push(...) { items.add(...); } }`. Only the methods you want

are exposed." 4. "Inheritance also fragilizes: if `ArrayList` adds a new method in a later JDK, your subclass might silently break invariants." 5. "Use inheritance for true is-a relationships within a small, controlled hierarchy. Use composition (often with interfaces) for everything else."

Edge cases to mention: - Records (Java 16+) cannot extend other classes — Java is pushing toward composition for data classes - Default methods in interfaces give you behavior reuse without inheritance — adopted heavily in modern Java APIs - Some patterns (Template Method) genuinely need inheritance — composition isn't a hammer for every nail

What NOT to say: - "Inheritance is always bad" — overreach; it has its place - "Composition is faster" — performance is roughly the same; this is a design argument

Common follow-up: "What does the Liskov Substitution Principle say about inheritance?"

Q92 · Liskov Substitution Principle — concrete example

Asked at: Most senior Java rounds · Razorpay · Cred

Difficulty: Medium-Hard · **Topic:** OOP

The question: "State the Liskov Substitution Principle and give a real example of violating it."

The 30-second answer: A subtype must be substitutable for its supertype without changing program correctness. Classic violation: `Square extends Rectangle`. If `setWidth(5)` and `setHeight(10)` works on `Rectangle` (area 50), it can't work on `Square` (a `Square` would force both to 10, breaking callers that assumed independent dimensions).

Step-by-step thought process: 1. "From Barbara Liskov: if S is a subtype of T , then objects of type T can be replaced with objects of type S without breaking the program." 2. "Square-Rectangle is the textbook violation. Math says a square IS a rectangle, but in OOP, mutating a `Rectangle` via `setWidth/ setHeight` should be independent — `Square` breaks that." 3. "Real-world violation I've seen: `ReadOnlyList extends ArrayList`. Adding throws `UnsupportedOperationException`. Caller types as `ArrayList` and expects to add — runtime crash." 4. "How to detect: any subclass that throws on a method it inherited, or weakens preconditions/strengthens postconditions, violates LSP." 5. "Fix: separate `ReadOnlyList` and `MutableList` into distinct interfaces. Don't force inheritance to model a 'special case' that breaks parent contracts."

Edge cases to mention: - Java's `Collections.unmodifiableList` violates LSP technically — it throws on mutation. Pragmatic trade-off the JDK made - Sealed classes (Java 17+) give you a way to control the hierarchy and reason about all subtypes - Pre/post conditions: subtype can ACCEPT more (weaker precondition) and PRODUCE less (stronger postcondition) without violating

What NOT to say: - "LSP says inheritance is bad" — it doesn't; it says inheritance has a contract - "LSP is theoretical, doesn't apply to real code" — every cast or polymorphic call relies on it

Common follow-up: "How would you redesign `Square` and `Rectangle` to avoid this?"

Q93 · Interface Segregation Principle

Asked at: Senior rounds · Razorpay · Walmart Labs

Difficulty: Medium · **Topic:** OOP

The question: "Explain Interface Segregation. Show me when a 'fat interface' is a problem."

The 30-second answer: Don't force clients to depend on methods they don't use. Split fat interfaces into smaller, role-specific ones. Classic example: `Worker` interface with `work()` and `eat()`. A `RobotWorker` is forced to implement `eat()` even though it makes no sense — split into `Workable` and `Eatable`.

Step-by-step thought process: 1. "Symptom: an interface with 20 methods, where implementations regularly leave half as no-ops or throw `UnsupportedOperationException`." 2. "Real example: JDK's `Map` is fat — `put`, `remove`, `clear` shouldn't apply to read-only views. JDK provides `unmodifiableMap` that throws on writes — LSP violation born from ISP violation." 3. "Better design: split into `ReadableMap` and `WritableMap`. Read-only views implement only `ReadableMap`. Callers depend on the narrowest interface they need." 4. "In Java 8+, default methods make small interfaces ergonomic — you can ship sensible defaults without forcing all implementers to write boilerplate." 5. "Pragmatic rule: if 50% of implementers throw or no-op a method, that method belongs in a separate interface."

Edge cases to mention: - Don't over-split — 50 tiny interfaces is also hard to use. Aim for cohesive roles, not method-per-interface - Marker interfaces (`Serializable`, `Cloneable`) are an ISP-related anti-pattern in modern Java; prefer annotations - For external libraries, you often can't change the interface — use Adapter pattern to give callers a narrower view

What NOT to say: - "Just use one big interface for simplicity" — that's exactly the anti-pattern - "Default methods solve everything" — they help, but don't eliminate the design question

Common follow-up: "How do default methods relate to interface segregation?"

Q94 · Immutability — how to design an immutable class properly

Asked at: Most Java mid-level rounds · Razorpay · Wells Fargo

Difficulty: Medium · **Topic:** OOP

The question: "How do you make a class truly immutable in Java?"

The 30-second answer: Make the class `final` (or use sealed). Make all fields `private final`. Don't expose setters. Defensively copy mutable arguments in the constructor. Defensively copy any mutable fields when returning them from getters. Or use records (Java 16+), which handle the boilerplate but still need defensive copies for mutable components.

Step-by-step thought process: 1. "Class must be `final` — otherwise a subclass can add mutable state and break the contract." 2. "All fields `private final` — final ensures field can't be re-assigned. Private blocks external access." 3. "No setters. Constructor sets everything; that's it." 4. "Defensive copy on input: if constructor takes a `List<String>`, copy it: `this.items = List.copyOf(items);`. Otherwise the caller still holds a mutating reference." 5. "Defensive copy on output: getter for a mutable field should return a copy or an unmodifiable view: `return List.copyOf(items);`. Records don't auto-do this."

Edge cases to mention: - Records (Java 16+) give you immutability for the record's own fields but mutable components (a List inside a record) are still mutable through their getters - For deep immutability of complex graphs, you may need to wrap children too — use `Collections.unmodifiableList`, `Map.copyOf`, `Set.copyOf` - `BigDecimal` and `LocalDate` are immutable — chain returns a new instance, doesn't mutate

What NOT to say: - "Just mark the class final, that makes it immutable" — incomplete; fields could still be mutable - "Immutable means thread-safe and that's it" — also enables hash safety, defensive freezing, fewer bugs

Common follow-up: "What's the difference between an immutable List and an unmodifiable List?"

Q95 · Builder pattern vs Factory pattern

Asked at: Most Java mid-level rounds

Difficulty: Easy-Medium · **Topic:** OOP / Design Patterns

The question: "When would you use Builder pattern vs Factory pattern?"

The 30-second answer: Builder for constructing one object with many optional parameters — readable, immutable result, no telescoping constructors. Factory for choosing which concrete type to instantiate based on input — hides the concrete class from callers. They solve different problems and can be combined.

Step-by-step thought process: 1. "Builder: when an object has many fields, especially optional ones. `Order.builder().merchantId(...).amount(...).build();` is far cleaner than a 10-arg constructor with most args as null." 2. "Lombok's `@Builder` annotation generates the builder for you. Spring uses builder pattern heavily — `WebClient.builder()`, `Customizer.withDefaults()`." 3. "Factory: when callers shouldn't know the concrete type. `PaymentGateway.forProvider(\"razorpay\")` returns the right implementation; caller just sees `PaymentGateway`." 4. "Static factory method on the class itself: `Optional.of(value)`, `List.of(a, b, c)`. Often clearer than `new` because the name conveys intent." 5. "Both can combine: a factory method that internally uses a builder, or a builder that produces different concrete subtypes based on configuration."

Edge cases to mention: - For objects with 3-4 required fields and no optionals, a plain constructor is still best - Builder with required fields can use staged builders:

`OrderBuilder.merchantId(...).amount(...).optional(...).build()` — type system enforces required-first - Records make builders less necessary for simple data classes — but if you need optionals or validation, builder still wins

What NOT to say: - "Always use Builder for everything" — overkill for simple constructors - "Factory and Builder are the same" — they solve different problems

Common follow-up: "How would you design a Builder for an immutable Order with required and optional fields?"

Q96 · try-with-resources — how does it work?

Asked at: Most Java mid-level rounds

Difficulty: Easy-Medium · **Topic:** Exception handling

The question: "What does try-with-resources do, and what does the resource need to implement?"

The 30-second answer: try-with-resources auto-closes resources at the end of the block. The resource must implement `AutoCloseable` (or `Closeable`, which is older and throws `IOException`). On exception, the resource is still closed; if close itself throws, the original exception is preserved and the close exception is added as a suppressed exception.

Step-by-step thought process: 1. "Syntax: `try (Connection conn = dataSource.getConnection(); Statement st = conn.createStatement()) { ... }`. Multiple resources, semicolon-separated." 2. "Resources are closed in REVERSE order of declaration. So `st` closes before `conn` — important when resources depend on each other." 3. "Resource type must implement `AutoCloseable` (single method: `close() throws Exception`). `Closeable` is a subtype that narrows the throws to `IOException`." 4. "Java 9+ allows referring to an existing effectively-final variable in the try header: `try (existingResource) { ... }`. Avoids reassignment." 5. "Exception handling: if the body throws AND close throws, the body's exception is primary; the close exception is added via `Throwable.addSuppressed()`. Recover both via `getSuppressed()`."

Edge cases to mention: - Resources are closed even on `return` inside the block — finally semantics - A null resource throws on close — wrap with a non-null check or use an empty Optional pattern - Don't catch `InterruptedException` inside try-with-resources for `InputStream/OutputStream` — propagate properly

What NOT to say: - "It's just syntactic sugar for try/finally" — true but missed the suppressed exception mechanism - "Resource must implement Closeable" — `AutoCloseable` is the actual requirement (`Closeable` is a subtype)

Common follow-up: "What's a suppressed exception and how do you read it?"

Q97 · Custom exceptions — when and how

Asked at: Most Spring backend rounds · Razorpay · Walmart

Difficulty: Easy-Medium · **Topic:** Exception handling

The question: "When would you create a custom exception class? How would you design it?"

The 30-second answer: Create one when the caller needs to programmatically distinguish your error from generic ones — e.g. `PaymentDeclinedException` vs a generic `RuntimeException`. Usually extend `RuntimeException` (unchecked). Include relevant context (`orderId`, `attemptedAmount`, `declineCode`) as fields. Override `getMessage()` for a useful log line.

Step-by-step thought process: 1. "Don't create exceptions just for namespacing — `MyAppException` extends `RuntimeException` adds nothing if it has no fields. Use existing exceptions like `IllegalStateException` for programming errors." 2. "Do create when callers need to switch on type: `catch (PaymentDeclinedException e)` vs `catch (PaymentTimeoutException e)` triggers different recovery." 3. "Default to unchecked (extends `RuntimeException`) — Spring and modern Java prefer this. Use checked only when callers MUST handle it (rare)." 4. "Include context: `public PaymentDeclinedException(String orderId, BigDecimal amount, String reason) { super(...); this.orderId = orderId; ... }`." 5. "In Spring REST, pair with `@ResponseStatus(BAD_REQUEST)` or an `@ExceptionHandler` that maps to an HTTP response. Don't leak stack traces to callers."

Edge cases to mention: - Always preserve the cause: `throw new PaymentException("failed", originalIOException);` — losing the cause hides the real reason - For libraries you publish, document which exceptions your API throws and under what conditions - Avoid swallowing exceptions silently — `catch (Exception e) { /* ignore */ }` is the most expensive bug pattern

What NOT to say: - "Use checked exceptions for everything to be safe" — modern Java tends the other way; checked exceptions create boilerplate - "Just throw `RuntimeException` directly" — losing the type means callers can't differentiate

Common follow-up: "How would you map a custom exception to a 400 status code in Spring?"

Q98 · Exception chaining — preserve the cause

Asked at: Most Java mid-level rounds

Difficulty: Easy-Medium · **Topic:** Exception handling

The question: "What is exception chaining and why does it matter?"

The 30-second answer: Exception chaining wraps a low-level exception inside a higher-level one while preserving the original — `new ServiceException("payment failed", originalSQLException)`. The original is accessible via `getCause()`. Critical because losing the cause hides the actual reason in stack traces.

Step-by-step thought process: 1. "Constructors: `Exception(message, cause)` or `initCause(cause)` afterwards. Always preserve the original." 2. "Stack traces print causes via 'Caused by:' lines — gives you the full chain from low to high level." 3. "Anti-pattern: `try { ... } catch (SQLException e) { throw new ServiceException(\"db error\"); }` — original `SQLException` is LOST. Should be `throw new ServiceException(\"db error\", e);`." 4. "For multiple causes (try-with-resources, parallel ops), use `addSuppressed()` to attach them to the primary exception." 5. "Spring's `DataAccessException` hierarchy wraps native DB exceptions but preserves them as cause — you can dig into the `SQLException` details if needed."

Edge cases to mention: - The cause field is set-once — `initCause` throws if called twice. Use `Throwable(message, cause)` constructor instead - `getMessage()` returns only the immediate message; use a formatter like `ExceptionUtils.getRootCauseMessage` for chain summaries - In structured logging, log the entire exception object — most loggers (SLF4J + Logback) print the full chain

What NOT to say: - "Log and rethrow" — double-logs; either log OR rethrow with cause, not both - "Just create a new exception, forget the cause" — destroys the most useful debugging info

Common follow-up: "What's the difference between `getCause()` and `getSuppressed()`?"

Q99 · Checked exceptions — are they obsolete?

Asked at: Senior rounds · Razorpay · Cred

Difficulty: Medium · **Topic:** Exception handling

The question: "Some people say checked exceptions are obsolete. What's your view?"

The 30-second answer: Checked exceptions force callers to acknowledge failures at compile time — a good idea in theory. In practice they create boilerplate, leak through abstraction layers, and lead to catch-and-ignore. Modern Java (Spring, Stream API, `CompletableFuture`) leans heavily toward unchecked. I default to unchecked, use checked only when the caller can meaningfully recover and the recovery is local.

Step-by-step thought process: 1. "Original intent: force callers to handle expected failures. `FileNotFoundException` on open says 'this might not exist'." 2. "Problem 1: checked exceptions don't compose with lambdas. `stream.map(file -> new BufferedReader(new FileReader(file)))` doesn't compile because `FileReader` throws checked. You're forced to wrap or catch." 3. "Problem 2: in service layers, low-level checked exceptions either propagate everywhere (every method declares `throws SQLException`) or get wrapped repeatedly (boilerplate)." 4. "Problem 3: catch-and-ignore is rampant — developers add empty catch blocks just to compile, hiding real failures." 5. "My rule: throw unchecked for everything in business code. Reserve checked for narrow cases where the caller has a concrete recovery path AND the recovery is local to the caller."

Edge cases to mention: - `Callable` was created to allow checked exceptions in a runnable-like interface — predates the unchecked-everywhere trend - Lombok's `@SneakyThrows` lets you throw

checked exceptions without declaring them — handy but controversial - Project Loom and structured concurrency revive interest in checked exceptions for explicit failure handling

What NOT to say: - "Checked exceptions are objectively bad" — there's a real argument both ways - "Wrap everything in RuntimeException" — usually right, but loses type info

Common follow-up: "How does Spring's @Transactional handle checked vs unchecked exceptions for rollback?"

Q100 · Records — design rules and gotchas

Asked at: Most modern Java rounds · Razorpay · Swiggy

Difficulty: Medium · **Topic:** Java 17/21 modern features

The question: "What are records in Java? When should you use them and what are the limitations?"

The 30-second answer: Records (GA in Java 16) are immutable data carriers — `record Order(String id, BigDecimal amount) {}` generates the constructor, accessors (`id()` , not `getId()`), equals, hashCode, toString. Final by default. Can't extend other classes (always extends Record). Can implement interfaces and have additional methods.

Step-by-step thought process: 1. "Syntax: `record Order(String id, BigDecimal amount, Customer customer) {}` . The header declares the components." 2. "Generated: canonical constructor, accessor per component (`order.amount()` — no `get` prefix), `equals` and `hashCode` based on ALL components, `toString` printing all components." 3. "Compact constructor for validation: `public Order { if (amount.signum() < 0) throw new IllegalArgumentException(\"negative amount\"); }` . No arg list, no this assignments — just validation/normalization." 4. "Can implement interfaces, add static factories, add additional methods. Cannot extend another class (it implicitly extends `java.lang.Record`)." 5. "Cannot have instance fields beyond the components. Cannot be subclassed unless you mark the record abstract — wait, records are final. Use sealed interfaces with record implementations for hierarchical data."

Edge cases to mention: - Records with mutable components (List, Date) are NOT deeply immutable — copy in compact constructor and on access if true immutability matters - Serialization: records have a different protocol than regular classes — they serialize as a record stream, decoupled from class evolution rules - Pattern matching for records (Java 21) enables `if (obj instanceof Order(var id, var amount, var customer))` — destructuring

What NOT to say: - "Records are just classes with less boilerplate" — true at one level, but the semantic guarantees (immutability, value equality) are the bigger deal - "Use records for entities" — JPA's mutating nature fights records; use them for DTOs and value objects

Common follow-up: "How does record pattern matching work in Java 21?"

Q101 · Sealed classes — what problem do they solve?

Asked at: Most modern Java rounds · Razorpay · Walmart Labs

Difficulty: Medium · **Topic:** Java 17/21 modern features

The question: "What are sealed classes and what problem do they solve?"

The 30-second answer: Sealed classes (GA in Java 17) restrict which classes can extend them — you list the permitted subtypes. This enables exhaustive pattern matching, gives the compiler complete knowledge of a hierarchy, and prevents arbitrary extension. Useful for domain models with a closed set of variants (Order can be Paid, Pending, or Cancelled — nothing else).

Step-by-step thought process: 1. "Syntax: `sealed interface OrderStatus permits Paid, Pending, Cancelled {}` . Each permitted type must be in the same module (or same package if unnamed module)." 2. "Permitted subtypes must declare themselves: `final class Paid implements OrderStatus` , `non-sealed class Pending implements OrderStatus` , etc." 3. "Subtype modifiers: `final` (can't be extended further), `sealed` (continues the restriction), or `non-sealed` (opens up — anyone can extend)." 4. "Combined with pattern matching: `switch (status) { case Paid p -> ...; case Pending p -> ...; case Cancelled c -> ...; }` — compiler verifies exhaustiveness." 5. "Real-world example: I model `PaymentResult` as sealed — `Success, Declined, GatewayError`. Anywhere I handle results, the compiler tells me if I miss a case after adding a fourth variant."

Edge cases to mention: - `permits` clause is optional if all permitted subtypes are in the same file — compiler infers - Records work great as sealed subtypes — `sealed interface Shape permits Circle, Square` where `Circle` and `Square` are records - Sealed plus pattern matching is Java's answer to algebraic data types (sum types) from FP languages

What NOT to say: - "Sealed is just final" — final prevents ALL extension; sealed allows a controlled list - "Sealed is for performance" — primary win is type safety and exhaustiveness checking

Common follow-up: "How does sealed combine with switch pattern matching for exhaustiveness?"

Q102 · Pattern matching switch (Java 21)

Asked at: Modern Java rounds · Razorpay · Swiggy

Difficulty: Medium-Hard · **Topic:** Java 17/21 modern features

The question: "What did Java 21 add to switch with pattern matching?"

The 30-second answer: Switch now supports type patterns — `case Order o -> ...` matches type and binds the variable. Plus guard clauses with `when` — `case Order o when o.amount() > 1000 -> ...` . Plus record deconstruction — `case Order(var id, var amount, var customer) -> ...` . Plus null handling — `case null -> ...` . Combined with sealed types, you get compiler-enforced exhaustiveness.

Step-by-step thought process: 1. "Old switch: only constants — `int`, `String`, `enum`. New switch: any type via type patterns. `case Integer i -> ...`." 2. "Type pattern binds the variable in the arm — no cast needed. Replaces `if (x instanceof Order) { Order o = (Order) x; ... }` boilerplate." 3. "Guards refine the match: `case Order o when o.status() == PAID -> handlePaid(o);`. Multiple guards can target the same type." 4. "Record deconstruction: `case Order(String id, BigDecimal amt, Customer c) -> ...` binds all components." 5. "Null in switch: `case null -> handleNull();` lets you handle null explicitly. Previously switch would NPE."

Edge cases to mention: - For sealed types, the compiler verifies exhaustiveness — missing a permitted subtype is a compile error - Order of cases matters when patterns overlap — more specific patterns first - `case null, default -> ...` combines null and default arms

What NOT to say: - "Pattern matching is just instanceof in a switch" — also handles records, guards, exhaustiveness - "Available since Java 11" — type patterns in switch are GA in Java 21

Common follow-up: "How does exhaustiveness checking work with sealed types and switch?"

Q103 · Text blocks — when do they help?

Asked at: Most modern Java rounds

Difficulty: Easy · **Topic:** Java 17/21 modern features

The question: "What are text blocks and what problem do they solve?"

The 30-second answer: Text blocks (GA in Java 15) are triple-quoted multi-line strings. They preserve formatting without escape gymnastics — embed SQL, JSON, HTML naturally. The compiler strips incidental leading whitespace based on the closing `"""` position, so you can indent without polluting the content.

Step-by-step thought process: 1. "Syntax: opening `"""` on a line by itself (with optional whitespace), content on subsequent lines, closing `"""`." 2. "Incidental whitespace removal: the compiler finds the indentation of the closing delimiter and strips that much from every line." 3. "Concrete: `String sql = """\n SELECT id, amount FROM orders\n WHERE merchant_id = ?\n""";` — the four leading spaces are stripped." 4. "Escape sequences still work. `\\s` is space (useful at line ends), `\\` at end of line suppresses the newline." 5. "String interpolation is NOT in text blocks — use `String.format` or `.formatted()`. JEP 459 (templated strings) was withdrawn."

Edge cases to mention: - Trailing whitespace on each line is stripped — use `\\s` if you need a trailing space to be preserved - A text block is still just a String — concatenation, equality, formatting all work normally - Don't use for short one-liners; the syntax is heavier than worth it

What NOT to say: - "Text blocks support template variables" — they don't; template strings were withdrawn - "Available since Java 13" — preview in 13 and 14, GA in Java 15

Common follow-up: "How would you build a multi-line SQL query with parameters using text blocks?"

Q104 · Virtual threads — the basics

Asked at: Modern Java rounds · Razorpay · Swiggy · Cred

Difficulty: Medium-Hard · **Topic:** Java 17/21 modern features

The question: "What are virtual threads and when do they help?"

The 30-second answer: Virtual threads (GA in Java 21, JEP 444) are lightweight threads scheduled by the JVM, not the OS. Millions can run on a few platform threads. Designed for I/O-heavy concurrency — each request gets a dedicated thread that blocks on I/O cheaply. Don't help CPU-bound work. Don't pin to platform threads under synchronized in Java 24+ (JEP 491).

Step-by-step thought process: 1. "Platform thread = OS thread, ~1MB stack, expensive to spawn.

Virtual thread = ~few KB, scheduled on a small pool of carrier (platform) threads." 2. "Create:

`Thread.startVirtualThread(runnable);` or

`Executors.newVirtualThreadPerTaskExecutor()`. Spring Boot 3.2+ enables them with one property." 3. "When a virtual thread blocks on I/O (network, file, sleep), it unmounts from its carrier — the carrier picks up another virtual thread. Net effect: 'one thread per request' scales to millions." 4. "CPU-bound work doesn't benefit — only one carrier runs at a time. Use ForkJoinPool or platform thread pools for parallel computation." 5. "Pre-Java 24, `synchronized` blocks pinned a virtual thread to its carrier — couldn't unmount during I/O. Java 24 JEP 491 fixed this; `synchronized` no longer pins."

Edge cases to mention: - Virtual threads are NOT pooled in the traditional sense — create one per task, throw it away. ThreadLocal lifecycle changes accordingly - Don't make virtual threads daemon by default in production — explicit shutdown matters - Profiling tools, thread dumps work with virtual threads, but old monitoring may show overwhelming thread counts — adjust dashboards

What NOT to say: - "Virtual threads make everything faster" — true for I/O-heavy, irrelevant for CPU-bound - "They replace ExecutorService" — they complement it;

`newVirtualThreadPerTaskExecutor()` IS an ExecutorService

Common follow-up: "What does it mean for a virtual thread to be pinned to its carrier?"

Q105 · ZGC and the move to default-generational

Asked at: Senior backend rounds · Goldman · Walmart Labs

Difficulty: Hard · **Topic:** Java 17/21 modern features / JVM

The question: "What's ZGC and why did generational ZGC become the default?"

The 30-second answer: ZGC is a concurrent low-latency garbage collector — sub-millisecond pauses on heaps from MBs to TBs. Original ZGC treated the whole heap uniformly. Generational ZGC (JEP 439,

default in Java 23 via JEP 474) splits into young and old generations, exploiting the weak generational hypothesis — most objects die young — for much better throughput at the same pause goals.

Step-by-step thought process: 1. "ZGC is concurrent — marking and relocation happen alongside application threads. Pauses are ~sub-millisecond regardless of heap size." 2. "Generational ZGC adds the young/old split. Young gen collects frequently and cheaply because most short-lived objects die there. Old gen collects rarely." 3. "For a service with 1000 reqs/sec creating short-lived objects per request, generational ZGC drops both pause times and CPU spent on GC." 4. "Enable: `-XX:+UseZGC` (and `-XX:+ZGenerational` pre-Java 23 if you want generational explicitly). Java 23+: it's the default mode for ZGC." 5. "Pick ZGC when: latency-sensitive (real-time bidding, trading, low p99 APIs), large heaps (>4GB), can afford ~10-15% throughput hit vs Parallel GC."

Edge cases to mention: - ZGC and Shenandoah compete in the same niche — both concurrent, both low-pause. Shenandoah is RedHat-led, ZGC is Oracle-led - Monitoring: GC pauses are tiny but GC CPU usage may be high — check both - For heaps under ~4GB, G1 may be a better fit — overhead of ZGC isn't justified

What NOT to say: - "ZGC has zero pauses" — sub-millisecond, not zero - "Always use ZGC" — depends on your latency vs throughput trade-off

Common follow-up: "What's the trade-off between ZGC and G1 for a typical microservice?"

Q106 · ScopedValue vs ThreadLocal (Java 21+)

Asked at: Senior modern Java rounds · Razorpay · Cred

Difficulty: Hard · **Topic:** Java 17/21 modern features / Concurrency

The question: "What is ScopedValue and why is it preferred over ThreadLocal for virtual threads?"

The 30-second answer: ScopedValue (preview in Java 20, evolving) is an immutable, scoped alternative to ThreadLocal. Set via `ScopedValue.where(KEY, value).run(() -> ...)` — value is visible only inside the lambda. No `set()` / `remove()` issues, no lingering state in pooled threads. Designed for the virtual-thread era where ThreadLocal-per-thread doesn't scale to millions of threads.

Step-by-step thought process: 1. "Declare: `public static final ScopedValue<RequestContext> CONTEXT = ScopedValue.newInstance();`" 2. "Bind and run: `ScopedValue.where(CONTEXT, ctx).run(() -> handleRequest());`. Value is set for the duration of run." 3. "Read inside: `RequestContext c = CONTEXT.get();`. Throws if not bound." 4. "Immutable — no `set()` to change mid-scope. To rebind, nest another `where(...).run(...)`. This makes reasoning easier." 5. "Virtual thread context: millions of virtual threads each carrying ThreadLocal state would be huge memory pressure. ScopedValue shares structures more efficiently and avoids lifecycle bugs."

Edge cases to mention: - Still a preview API as of Java 21 — syntax may evolve in future versions - Works with structured concurrency: a parent's scoped values are inherited by child tasks - ThreadLocal still works for legacy code; ScopedValue is the recommended direction for new code

What NOT to say: - "ScopedValue replaces ThreadLocal everywhere today" — it's preview; mature only in Java 24+ - "ScopedValue is mutable" — explicitly immutable, that's the point

Common follow-up: "How does ScopedValue interact with structured concurrency?"

Q107 · JPA entity lifecycle — full picture

Asked at: Most Spring backend rounds · Razorpay · Walmart Labs

Difficulty: Medium · **Topic:** Hibernate / JPA

The question: "What are the JPA entity states and how does an entity transition between them?"

The 30-second answer: Four states: NEW (just instantiated, no DB row), MANAGED (in persistence context, changes tracked), DETACHED (was managed, persistence context closed), REMOVED (marked for delete, pending flush). Transitions: `persist()` NEW→MANAGED, `merge()` DETACHED→MANAGED, `remove()` MANAGED→REMOVED, end-of-transaction MANAGED→DETACHED.

Step-by-step thought process: 1. "NEW: `new Order()` . Object exists in JVM, no DB awareness. No identity yet (unless you set it manually)." 2. "MANAGED: `em.persist(order)` or query result. Persistence context tracks every change to the object's fields — dirty checking flushes them on commit." 3. "DETACHED: persistence context closed (transaction commit + EM close), or entity explicitly detached. Changes no longer tracked. To re-attach, use `merge()` which returns a managed copy." 4. "REMOVED: `em.remove(managed)` marks it for delete. Actual DELETE SQL runs at flush. Until flush, it's still in the persistence context." 5. "Dirty checking: at commit (or flush), JPA compares managed entities to their snapshots. Any field difference triggers UPDATE SQL. Why you don't call `save()` after mutating a managed entity."

Edge cases to mention: - `merge()` does NOT return the same instance — it returns a managed copy. `Order merged = em.merge(detached);` — work with `merged`, not `detached` - Detached entities lose lazy collections — accessing them throws `LazyInitializationException` - In Spring Data JPA, `save()` calls `persist` for new entities and `merge` for ones with an ID — convenient but masks the state difference

What NOT to say: - "Entities are always managed" — they're managed only inside a session and an attach - "save() is the universal verb" — JPA distinguishes persist, merge, remove

Common follow-up: "What's the difference between persist and merge?"

Q108 · Lazy vs eager loading — and the N+1 problem

Asked at: Most Spring backend rounds · Razorpay · Swiggy · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Hibernate / JPA

The question: "Explain lazy vs eager loading. What's the N+1 problem and how do you fix it?"

The 30-second answer: Lazy = related entities loaded on first access via a proxy (one query for parent, additional queries per child access). Eager = loaded immediately with the parent (single query with JOIN). N+1 is the classic lazy-loading trap: fetch N orders, then call `order.getCustomer()` in a loop — that's N additional queries. Fix with JOIN FETCH, entity graphs, or batch fetching.

Step-by-step thought process: 1. "`@ManyToOne` and `@OneToOne` are eager by default. `@OneToMany` and `@ManyToMany` are lazy by default." 2. "Lazy works via a Hibernate proxy. `order.getCustomer()` triggers a SELECT for that customer. Convenient — but in a loop, you trigger N queries." 3. "N+1: `SELECT * FROM orders` (1 query) + N calls to `getCustomer()` (N queries). Spot it by enabling SQL logging." 4. "Fix 1: JOIN FETCH in JPQL. `SELECT o FROM Order o JOIN FETCH o.customer WHERE ...`. Loads orders + customers in one query." 5. "Fix 2: `@EntityGraph(attributePaths = {\"customer\"})` on the repository method. Fix 3: `@BatchSize(size = 50)` on the association — Hibernate batches the lazy loads."

Edge cases to mention: - Default `@OneToOne` as eager is often wrong — almost always make it lazy unless you always need the related entity - `LazyInitializationException` — fetching a lazy field after the session closed. Use Open-Session-in-View (controversial) or fetch eagerly in repository methods - For lists, JOIN FETCH on a `@OneToMany` returns duplicate parents — use `SELECT DISTINCT` or `Set<>` collections

What NOT to say: - "Make everything eager to avoid N+1" — eagerly loads way too much, slow queries - "Hibernate caches it so N+1 doesn't matter" — first-level cache only saves within same session

Common follow-up: "How does an EntityGraph differ from JOIN FETCH?"

Q109 · First-level cache and dirty checking

Asked at: Most Spring backend rounds · Razorpay · Walmart Labs

Difficulty: Medium · **Topic:** Hibernate / JPA

The question: "What's the first-level cache in Hibernate and how does dirty checking use it?"

The 30-second answer: First-level cache (a.k.a. persistence context) is scoped to the EntityManager / Session. Every managed entity lives there as a snapshot. Dirty checking compares the current entity state to the snapshot at flush — any difference becomes an UPDATE. Disabled per-entity via `@Immutable` or via `setReadOnly()`.

Step-by-step thought process: 1. "Every entity loaded or persisted goes into the persistence context — keyed by entity class + primary key." 2. "Within one session, `em.find(Order.class, id)` twice

returns the SAME object (identity equality). Second call skips the DB." 3. "Hibernate stores a snapshot of each entity's fields at load time. At flush (commit or explicit `flush()`), it compares current vs snapshot — emits UPDATE for any field that changed." 4. "No need to call `save()` for mutations on a managed entity — the changes are detected automatically." 5. "Persistence context is per-EM, not shared. A second EM has its own snapshot — stale data possible if you don't refresh."

Edge cases to mention: - For batch jobs processing millions of rows, the persistence context grows unbounded — call `em.clear()` periodically or use `StatelessSession` to avoid heap pressure - `@DynamicUpdate` makes Hibernate emit UPDATE only for changed columns instead of all columns — useful for wide tables - Read-only optimization: `@Immutable` (entity-level) or `setReadOnly(true)` on a query — Hibernate skips snapshot and dirty checking

What NOT to say: - "First-level cache is shared across requests" — it's per-session/EM, request-scoped at best - "I always need to call save()" — managed entity mutations auto-flush

Common follow-up: "What's the difference between first-level and second-level cache?"

Q110 · @Transactional readonly and flush modes

Asked at: Senior Spring backend rounds · Razorpay · Cred

Difficulty: Medium-Hard · **Topic:** Hibernate / JPA

The question: "What does `@Transactional(readOnly=true)` actually do?"

The 30-second answer: `readOnly=true` does several things: hints the JDBC driver (some can route to read replicas), tells Hibernate to skip dirty checking and snapshots (memory + CPU savings), sets the flush mode to MANUAL so accidental updates don't get flushed. It's a perf optimization for query-only methods, not a security feature — you can still mutate, just nothing is flushed.

Step-by-step thought process: 1. "On the JDBC connection, Spring sets `setReadOnly(true)`. Many JDBC drivers ignore it; some (like PostgreSQL with the right config) route to a read replica." 2. "On the Hibernate session, flush mode becomes MANUAL — no automatic flush at commit. So even if you mutate managed entities, nothing is written to DB." 3. "Hibernate skips storing the original snapshot per entity (since dirty checking won't run) — substantial memory savings for queries returning many rows." 4. "NOT a security boundary. A read-only transaction can still call INSERT/UPDATE/DELETE if you bypass the persistence context (native query, `em.flush()` manually)." 5. "Apply `readOnly=true` to all your query/get methods. Don't apply at the class level if some methods mutate."

Edge cases to mention: - For pure read methods on a service, the perf gain on Hibernate's snapshot skipping is noticeable when returning thousands of entities - `readOnly + REQUIRES_NEW` gets you a true new read-only transaction — useful for fetching auxiliary data in the middle of a write flow - Connection routing to replica needs driver-specific config (e.g. AWS Aurora's ROLE-aware driver)

What NOT to say: - "readOnly prevents writes" — it doesn't enforce; it just avoids flushing - "Always read-only is faster" — true for reads; harmful if the method actually writes

Common follow-up: "How would you route read-only queries to a database replica?"

Q111 · REST HTTP verbs — semantics and idempotency

Asked at: Most backend rounds · Razorpay · Swiggy · Wells Fargo

Difficulty: Easy-Medium · **Topic:** REST / API design

The question: "What are the HTTP verb semantics, and which are idempotent?"

The 30-second answer: GET = read, safe + idempotent. PUT = replace whole resource, idempotent. PATCH = partial update, NOT necessarily idempotent. POST = create or non-idempotent action. DELETE = remove, idempotent (deleting twice still results in no resource). Safe = no server state change. Idempotent = same effect whether called once or N times.

Step-by-step thought process: 1. "GET is for reads — no side effects, safe, idempotent. Should be cacheable. Never put state-changing logic in a GET handler." 2. "PUT replaces the entire resource at a URL. Idempotent: `PUT /orders/123` with the same body twice has the same final state." 3. "PATCH applies a partial update. Idempotency depends on the patch semantics — JSON Merge Patch is idempotent, but a delta patch ("increment counter by 1") is not." 4. "POST is the non-idempotent verb. `POST /orders` typically creates a new order each call. Use idempotency keys in headers (Razorpay style) to make retries safe." 5. "DELETE is idempotent — first call removes the resource, subsequent calls return 404 or 200 but don't change state further."

Edge cases to mention: - Idempotency keys (an `Idempotency-Key` header) let you make POST safe to retry — server stores the result and replays it - HEAD is like GET but returns no body — useful for cache validation - OPTIONS is for CORS preflight and feature discovery — usually framework-handled

What NOT to say: - "POST is always for create and PUT is always for update" — too narrow; PUT can create too if the URL is known - "Idempotent means safe" — different concepts; GET is both, DELETE is idempotent but not safe

Common follow-up: "How would you implement idempotency for POST /payments?"

Q112 · REST status codes — pick the right one

Asked at: Most backend rounds

Difficulty: Easy-Medium · **Topic:** REST / API design

The question: "Walk me through the major HTTP status code categories and the common codes."

The 30-second answer: 2xx success: 200 OK, 201 Created (POST), 204 No Content (DELETE / update with no body). 3xx redirection: 301 permanent, 302 found, 304 Not Modified. 4xx client error: 400 bad request, 401 unauthenticated, 403 unauthorized, 404 not found, 409 conflict, 422 unprocessable entity,

429 rate limited. 5xx server error: 500 generic, 502 bad gateway, 503 service unavailable, 504 gateway timeout.

Step-by-step thought process: 1. "2xx — success. 200 for general success with body. 201 + Location header for resource creation. 204 for success with no body." 2. "3xx — caller should look elsewhere. 301/302 redirects, 304 'use your cached copy' for conditional GETs." 3. "4xx — caller's fault. 400 for malformed input. 401 for missing/bad auth. 403 for authenticated-but-not-allowed. 404 for missing. 409 for conflicts (e.g. duplicate). 422 for semantically wrong input. 429 for rate limited." 4. "5xx — server's fault. 500 for uncaught error. 502 when an upstream returned bad response. 503 when overloaded or maintenance. 504 when upstream timed out." 5. "Pick the most specific one. 422 vs 400 matters — 422 means 'I understand the syntax but the data is invalid'. 409 vs 400 matters — 409 means 'conflict with current state' (e.g. version mismatch)."

Edge cases to mention: - Don't return 200 with `{"status": "error"}` in body — use the actual error code. Frameworks (axios, browser caches, retries) react to status, not body - 401 vs 403 is THE most-confused pair — 401 = 'who are you?', 403 = 'I know you, you can't do this' - 429 should include a `Retry-After` header so clients know when to retry

What NOT to say: - "Just return 200 for everything and put status in the body" — fights the HTTP ecosystem - "5xx means the server crashed" — 5xx is any server-side failure, including known/handled ones

Common follow-up: "What status code would you return when a customer tries to delete an order that's already shipped?"

Q113 · API versioning strategies

Asked at: Most backend rounds · Razorpay · Swiggy

Difficulty: Medium · **Topic:** REST / API design

The question: "How would you version a REST API? What's the trade-off?"

The 30-second answer: Three common strategies: URL versioning (`/v1/orders`), header versioning (`Accept: application/vnd.api+json;version=1`), query parameter (`/orders?v=1`). URL is most explicit and discoverable but pollutes routes; header is cleanest but harder to debug; query param is rare in serious APIs. Pick URL versioning for public APIs (industry standard), header for internal microservices.

Step-by-step thought process: 1. "URL versioning: `/v1/orders` , `/v2/orders` . Pros: visible in logs and dashboards, easy to test in browser, clear in docs. Cons: a resource isn't really 'at' two URLs — REST purists object." 2. "Header versioning: `Accept: application/vnd.merchant.v1+json` or custom `API-Version: 1` . Pros: same URL, cleaner. Cons: invisible in URLs, harder to debug from a curl history." 3. "For most public APIs (Stripe, Razorpay, AWS), URL versioning wins on discoverability." 4. "Within a version, keep backward compatibility: add fields, don't remove; deprecate before removing;

document breaking changes loudly." 5. "Major version bumps for breaking changes only. Adding an optional field is NOT a breaking change."

Edge cases to mention: - For internal microservices, consider no versioning + strict contract testing — every consumer is upgraded together - Spring's `@RequestMapping(value = \"/orders\", headers = \"X-API-Version=2\")` supports header-based routing - Deprecation: respond with `Deprecation` and `Sunset` HTTP headers (RFC 8594, 8295) when an endpoint will go away

What NOT to say: - "Just never version — keep it backward compatible forever" — eventually you need to break things - "Bump version for every field addition" — that's not a breaking change

Common follow-up: *"How would you migrate clients from v1 to v2 without breaking anyone?"*

Q114 · Idempotency in payments — how to implement

Asked at: Razorpay · Cred · most fintech rounds

Difficulty: Medium-Hard · **Topic:** REST / API design

The question: *"How would you make POST /payments idempotent?"*

The 30-second answer: Accept an `Idempotency-Key` header (UUID). On first call: process the payment, store the (key → response, status) in a table with a TTL. On subsequent calls with the same key: return the stored response without reprocessing. If the first call is in flight, return 409 or block briefly. Index on key for fast lookup.

Step-by-step thought process: 1. *"Client generates a unique key per logical payment attempt (UUID). They send it in `Idempotency-Key` header on retries."* 2. *"Server stores (key, request_hash, response, status, created_at, expires_at) . Status is `PROCESSING` initially, then `COMPLETED` ."* 3. *"On retry: lookup by key. If `PROCESSING` and within timeout, wait or 409. If `COMPLETED`, return stored response — same body, same status code, even the same headers."* 4. *"Verify the request body matches what was originally sent (hash check). Mismatch = client is reusing a key incorrectly — 422 with clear error."* 5. *"TTL the idempotency records — typically 24 hours. Old keys aged out to prevent table bloat."*

Edge cases to mention: - Race condition on first request: two simultaneous calls with same key. Use a UNIQUE constraint on key column — second insert fails, the loser polls for the winner's result - For high throughput, store in Redis instead of the main DB — but persistence trade-off matters for payments - Stripe and Razorpay both use this exact pattern — read their docs for the exact contract

What NOT to say: - "Just check if the payment already exists" — race conditions; need an explicit idempotency record - "Idempotency is automatic for REST" — it isn't for POST; you have to build it

Common follow-up: *"What happens if two requests with the same idempotency key arrive at the same instant?"*

Q115 · HashMap.computeIfAbsent vs putIfAbsent

Asked at: Most Java mid-level rounds

Difficulty: Medium · **Topic:** Collections

The question: "What's the difference between computeIfAbsent and putIfAbsent in HashMap?"

The 30-second answer: `putIfAbsent(key, value)` puts the given value if absent. `computeIfAbsent(key, k -> compute(k))` invokes the function ONLY if absent — value is lazily computed. For expensive default values, use computeIfAbsent. Both return the existing value if present, or the new value if not.

Step-by-step thought process: 1. "`putIfAbsent(key, valueAlreadyComputed)` — you've already built the value, just want to insert if missing." 2. "`computeIfAbsent(key, k -> expensive(k))` — function runs only when key is absent. Save the cost when the entry already exists." 3. "Common use: building multi-maps. `map.computeIfAbsent(merchantId, id -> new ArrayList<>()).add(order);`. Creates the list only if needed." 4. "In ConcurrentHashMap, both are atomic. computeIfAbsent's function holds a lock on the bucket — keep it short, don't call other map operations inside." 5. "`compute(key, (k, v) -> ...)` is the variant for 'transform existing value or insert new'. `merge(key, value, mergeFn)` for accumulating."

Edge cases to mention: - ConcurrentHashMap's computeIfAbsent throws if the function tries to recurse into the same map — deadlock detection - In plain HashMap, mutating the map inside the function is allowed but discouraged - If the function returns null, no entry is added — useful for conditional insertion

What NOT to say: - "They're the same" — laziness matters for expensive defaults - "computeIfAbsent always runs the function" — it doesn't if the key is present

Common follow-up: "What happens if your computeIfAbsent function returns null?"

Q116 · StringBuilder vs StringBuffer

Asked at: TCS · Infosys · Wipro · most fresher and mid rounds

Difficulty: Easy · **Topic:** Strings

The question: "Difference between StringBuilder and StringBuffer?"

The 30-second answer: Both are mutable string builders. StringBuffer is synchronized (thread-safe but slower). StringBuilder is not synchronized (faster). For single-threaded use — which is 99% of cases — StringBuilder is the right choice. StringBuffer dates from Java 1.0 before there were better concurrency tools.

Step-by-step thought process: 1. "Both have `append`, `insert`, `delete`, `replace`, `reverse`, `toString`. Same API." 2. "StringBuffer (Java 1.0) — every method is `synchronized` on `this`. Thread-safe for single-builder access from multiple threads." 3. "StringBuilder (Java 5) — same API, no synchronization. Faster in single-threaded code. Use this by default." 4. "In modern code,

building a String concurrently from many threads is rare — you'd use a `ConcurrentMap` or join after each thread finishes. So `StringBuffer` is essentially obsolete." 5. "Both auto-grow an internal `char[]` (or `byte[]` since Java 9 compact strings). Initial capacity 16; doubles when full. Pre-size if you know the approximate result length."

Edge cases to mention: - Java 9+ uses compact strings — internal storage is `byte[]` for ASCII, `char[]` for non-ASCII. Halves memory for ASCII-heavy content - `String.join(", ", parts)` is often clearer than manual building - Modern JIT can sometimes optimize String concatenation into `StringBuilder` under the hood — but be explicit in loops

What NOT to say: - "Use `StringBuffer` for safety" — paying for synchronization you almost never need - "String concatenation is fine in loops" — creates $O(n)$ intermediate strings; use `StringBuilder`

Common follow-up: "What does Java do under the hood for `String x = a + b + c`?"

Q117 · final keyword — three contexts

Asked at: Most Java mid-level rounds

Difficulty: Easy · **Topic:** OOP

The question: "What does the `final` keyword do in Java? List the contexts where you can use it."

The 30-second answer: On a variable: can't be reassigned after initialization (reference is fixed, but contents of mutable object can change). On a method: can't be overridden by subclasses. On a class: can't be subclassed. Plus `final` parameters (can't be reassigned within the method). Critical for immutable classes and for lambda capture.

Step-by-step thought process: 1. "`final int x = 5;` — x can't be reassigned. For primitives, also means the value is constant." 2. "`final List<String> list = new ArrayList<>();` — the REFERENCE is fixed. You can still call `list.add(...)` because the object itself is mutable." 3. "`final void process()` — can't be overridden by subclasses. Useful for invariants you don't want subclasses to break." 4. "`final class Order` — can't be subclassed. Used for immutable classes like `String`, `Integer`, all wrappers." 5. "Lambdas/anonymous inner classes can only capture effectively-final local variables — Java enforces this so captured values don't change unexpectedly under the lambda."

Edge cases to mention: - `final` on parameters is rarely written but enforced by some style guides — adds clarity, no perf impact - `final` field initialization: must happen in constructor or inline; uninitialized at end of constructor is a compile error - `static final` for constants — convention is ALL_CAPS naming

What NOT to say: - "final makes the object immutable" — only the reference; object can still mutate - "final is just an optimization hint" — has compile-time semantics, not just a hint

Common follow-up: "Why must captured variables in a lambda be effectively final?"

Q118 · static keyword and static blocks

Asked at: Most Java mid-level rounds

Difficulty: Easy-Medium · **Topic:** OOP

The question: "Explain the static keyword. What's a static initialization block?"

The 30-second answer: `static` makes a member belong to the class, not instances — one copy shared. Static fields, methods, nested classes, and initialization blocks. Static blocks run once when the class is first loaded — useful for complex initialization of static fields. Order: static blocks run in source order during class loading.

Step-by-step thought process: 1. "`static int counter;` — one counter shared by all instances. Accessed via `ClassName.counter`." 2. "`static method()` — no `this` reference, can't access non-static members directly. Called via class name." 3. "`static class Inner` (nested class) — does NOT have an implicit reference to outer instance. Use `this`; non-static inner classes are usually a memory leak waiting to happen." 4. "`static { /* init code */ }` block — runs once, when class is first loaded by `ClassLoader`. Used to populate static fields that need logic to compute." 5. "Order: static blocks and static field initializers run in source order at class load time. Instance fields and instance blocks run for each new instance at construction."

Edge cases to mention: - Static fields aren't thread-safe by default — concurrent writes can race. Use `AtomicInteger` / `synchronized` / `final` - A class is loaded lazily, on first access (constructor call, static method call, static field access). Forcing load: `Class.forName(\"...\")` - Static fields hold strong references — common source of leaks in long-running apps (large caches kept by classloader)

What NOT to say: - "Static methods can be overridden" — they're hidden, not overridden — completely different dispatch - "Static blocks run every time you create an instance" — they run ONCE at class load

Common follow-up: "What's the difference between static and instance initialization blocks?"

Q119 · Generics — type erasure and its consequences

Asked at: Senior rounds · Razorpay · Goldman · Cred

Difficulty: Medium-Hard · **Topic:** OOP / Generics

The question: "What is type erasure in Java generics and what limits does it impose?"

The 30-second answer: Java generics are compile-time only — at runtime, `List<String>` and `List<Integer>` are both just `List`. The compiler inserts casts and checks; the JVM sees raw types. Consequences: can't do `new T[]`, can't check `instanceof List<String>`, can't have two methods that differ only in generic parameter, no primitive generics (need wrappers).

Step-by-step thought process: 1. "At compile, `List<String>` enforces type safety. At runtime, only `List` exists — the `<String>` is erased." 2. "Can't do `new T[10]` — JVM doesn't know what `T` is at runtime. Workaround: pass `Class<T>` to the constructor." 3. "Can't do `if (x instanceof`

`List<String>`) — erasure removes the type parameter. Only `instanceof List<?>` works." 4. "Method overload conflict: `void process(List<String>)` and `void process(List<Integer>)` are compile errors — same erasure." 5. "No `List<int>` — primitives can't be generic parameters. Use `List<Integer>` with autoboxing cost, or specialized `IntStream` / `int[]`."

Edge cases to mention: - Project Valhalla (in progress for Java 24+) may eventually bring primitive generics and reified types - `@SuppressWarnings(\"unchecked\")` is sometimes necessary when casting raw types — use sparingly with a comment why - `List.class` is `Class<List>` — you can't write `List<String>.class` for the same reason

What NOT to say: - "Java generics are like C++ templates" — C++ creates a new class per parameterization; Java erases - "Erasure is removed in Java 21" — still erased; Valhalla is the long-term project

Common follow-up: "How would you create a generic array workaround?"

Q120 · Wildcards — `? extends` vs `? super`

Asked at: Senior Java rounds · Razorpay · Wells Fargo

Difficulty: Medium-Hard · **Topic:** Generics

The question: "Explain PECS — Producer Extends, Consumer Super. Give a concrete example."

The 30-second answer: `List<? extends Number>` — you can READ Numbers from it (producer), can't add (you don't know if it's `List<Integer>` or `List<Double>`).

`List<? super Integer>` — you can ADD Integers (consumer), can only read as Object. PECS = Producer Extends, Consumer Super.

Step-by-step thought process: 1. "PECS is a mnemonic for choosing wildcards. If a collection PRODUCES values you'll read, use `? extends`. If it CONSUMES values you'll add, use `? super`." 2. "`Collections.copy(List<? super T> dest, List<? extends T> src)` — src is the producer (read T from it), dest is the consumer (write T to it). Textbook PECS." 3. "`List<? extends Number> nums = new ArrayList<Integer>();` . You can `Number n = nums.get(0);` . You CANNOT `nums.add(1);` — because the actual list might be `List<Double>` and 1 isn't a Double." 4. "`List<? super Integer> ints = new ArrayList<Number>();` . You CAN `ints.add(1);` because Integer is-a Number is-a Object. You read as `Object o = ints.get(0);` — that's the most specific known type." 5. "Without wildcards, `List<Number>` is INVARIANT — `List<Integer>` is NOT a `List<Number>` . Wildcards introduce variance into Java's generics."

Edge cases to mention: - `List<?>` is `List<? extends Object>` — read-only (well, can add null) - For method parameters, prefer the narrowest wildcard that still serves your use case — flexibility for callers - Self-referential bounds: `<T extends Comparable<T>>` is a common idiom for "must be comparable to its own type"

What NOT to say: - "Just use raw types, it's simpler" — defeats type safety - "Wildcards make code generic" — they make APIs flexible while preserving type safety

Common follow-up: "Why can't you add anything except null to a `List<? extends Number>`?"

Q121 · Stream.flatMap — the flatten operation

Asked at: Most Java mid-level rounds · Razorpay · Swiggy

Difficulty: Medium · **Topic:** Java 8 Functional

The question: "What does `Stream.flatMap` do and when is it useful?"

The 30-second answer: `flatMap` maps each element to a stream of values, then concatenates all those streams into one flat stream. Use it to flatten nested structures — `List<Customer>` where each has `List<Order>` becomes a flat `Stream<Order>` via `customers.stream().flatMap(c -> c.orders().stream())`.

Step-by-step thought process: 1. "`map(fn)` is 1-to-1 — each element becomes exactly one output. `flatMap(fn)` is 1-to-N — each element becomes 0 or more outputs." 2. "Function signature: `flatMap(T -> Stream<R>)`. The result streams are concatenated lazily." 3. "Classic example: split each sentence into words. `sentences.stream().flatMap(s -> Arrays.stream(s.split("\\s+")))`." 4. "For nested collections: `users.stream().flatMap(u -> u.getOrders().stream()).filter(o -> o.amount() > 1000)` — work on a flat stream of orders." 5. "Also exists on `Optional`: `optional.flatMap(value -> compute(value))` for chaining computations that themselves return `Optional` — avoids `Optional<Optional<T>>`."

Edge cases to mention: - For primitives, `flatMapToInt`, `flatMapToLong`, `flatMapToDouble` avoid boxing — use them when the inner streams are primitive - The inner stream from the function is closed automatically — usually you don't care, but matters for `Files.lines()` style streams - Empty inner stream is fine — that element contributes nothing to the output

What NOT to say: - "flatMap is just map with extra steps" — different cardinality semantics - "Use `flatMap` to filter" — wrong tool; filter is filter

Common follow-up: "How would you flatten a `List<Stream>` into a `Stream`?"

Q122 · Stream.peek — debugging or never?

Asked at: Most Java mid-level rounds

Difficulty: Easy-Medium · **Topic:** Java 8 Functional

The question: "What's `Stream.peek` for? Is it safe to use in production?"

The 30-second answer: peek lets you observe each element as it flows through the stream — typically for debugging or logging. It's intermediate, so it only runs when a terminal operation pulls elements through. Don't use peek for side effects in production code — the JVM may skip peek calls if the result is unused (e.g. `stream().peek(...).count()` with a SIZED source).

Step-by-step thought process: 1. "`stream.peek(System.out::println)` — prints each element as the stream is consumed. Returns the stream unchanged." 2. "Peek doesn't trigger consumption — it's intermediate. `stream().peek(...)` without a terminal doesn't run." 3. "Useful for debugging: `stream().filter(...).peek(this::debug).map(...).collect(...)` — see what survives the filter." 4. "Trap: `stream().peek(this::addToCache).count()` — for a SIZED source, the JVM knows the count without iterating. peek may not run. Don't use peek for state-changing side effects." 5. "For real side effects, use `forEach` (terminal). For per-element processing in a pipeline, use `map`."

Edge cases to mention: - Parallel streams: peek runs on the pool thread, not the calling thread — thread-safe code only - The Stream API spec explicitly says peek MAY be optimized away in some cases - Modern Java may skip peek entirely if downstream operations don't need the element — don't rely on it

What NOT to say: - "peek is the right way to add side effects" — explicitly warned against in the spec - "peek is exactly like forEach" — peek is intermediate; forEach is terminal

Common follow-up: "Why does peek sometimes not run when chained with count?"

Q123 · @PostConstruct vs constructor injection

Asked at: Most Spring rounds

Difficulty: Medium · **Topic:** Spring Core

The question: "What's the difference between initialization in the constructor and in a @PostConstruct method?"

The 30-second answer: Constructor runs before any injection happens — so field-injected dependencies are null. @PostConstruct runs AFTER all dependencies are injected — safe to use any injected bean. For constructor injection (preferred), the dependencies ARE available in the constructor.

Step-by-step thought process: 1. "Constructor runs at instantiation. With constructor injection, dependencies are passed in — fully usable inside." 2. "With field/setter injection, the constructor runs first, then Spring sets the fields. Using a field-injected dependency in the constructor — NullPointerException." 3. "`@PostConstruct` runs after all injection is complete, before the bean is published. Safe to use any dependency." 4. "Use @PostConstruct for: validation of injected state, populating caches from a repository at startup, starting background threads tied to the bean's lifecycle." 5. "Modern preference: constructor injection + perform setup in a method called from the constructor. Avoids @PostConstruct except when you genuinely need post-injection logic."

Edge cases to mention: - `@PostConstruct` is jakarta.annotation in Spring 6+ — was javax.annotation in earlier versions - `InitializingBean.afterPropertiesSet()` is an alternative — same lifecycle slot, no annotation - AOP proxies are wired AFTER `@PostConstruct` in some configurations — calling proxied methods on `this` inside `@PostConstruct` may bypass the proxy

What NOT to say: - "Constructor and `@PostConstruct` are the same" — different lifecycle slots, different access to dependencies - "Always use `@PostConstruct` for initialization" — constructor injection makes it usually unnecessary

Common follow-up: "What's the order between `@PostConstruct` and Spring's AOP proxy wiring?"

Q124 · @Bean vs @Component — when each?

Asked at: Most Spring rounds

Difficulty: Easy · **Topic:** Spring Core

The question: "What's the difference between defining a bean with `@Bean` and with `@Component`?"

The 30-second answer: `@Component` goes on the class — Spring instantiates it via reflection on the constructor. `@Bean` goes on a method inside a `@Configuration` class — YOU write the instantiation, can pass args, return any type. Use `@Component` for your own classes; `@Bean` for third-party classes you don't own, or when construction logic is complex.

Step-by-step thought process: 1. "`@Component` on your own class: Spring sees the annotation, calls the constructor (with injected args), registers the bean." 2. "`@Bean` on a method: `@Bean public RestTemplate restTemplate() { return new RestTemplate(); }` inside a `@Configuration` class. You control creation." 3. "`@Bean` wins for third-party classes — you can't annotate a class from a library, so `@Bean` is the only way to register them." 4. "`@Bean` wins for conditional or complex construction — `if (env.acceptsProfiles(\"cloud\")) return cloudClient(); else return localClient();` ." 5. "`@Component` is shorter for your own classes when constructor injection covers the needs."

Edge cases to mention: - Methods on `@Configuration` classes are wrapped by CGLIB — calling `beanA()` from `beanB()` returns the cached singleton instance, not a new one. Only happens for `@Configuration`, not `@Component` - `@Bean(initMethod=, destroyMethod=)` lets you wire up lifecycle methods without implementing JDK interfaces - `@Bean` methods can take parameters — Spring auto-injects them as bean references

What NOT to say: - "Use `@Bean` for primary beans, `@Component` for secondary" — both can be primary; that's a separate concern - "`@Bean` is slower" — both are evaluated once at startup; runtime cost is identical

Common follow-up: "Why do `@Configuration` class methods return the same instance when called from another `@Bean` method?"

Q125 · Spring Data JPA — what does it generate?

Asked at: Most Spring backend rounds · Razorpay · Swiggy · Walmart

Difficulty: Easy-Medium · **Topic:** Hibernate / JPA / Spring

The question: "How does Spring Data JPA generate implementations from repository interfaces?"

The 30-second answer: At startup, Spring scans for interfaces extending `JpaRepository<Entity, ID>`. For each, it creates a proxy at runtime that implements the methods. Method names are parsed by convention — `findByMerchantIdAndStatus(...)` becomes a SELECT generated automatically. Custom queries via `@Query` annotation.

Step-by-step thought process: 1. "Spring uses bean factory post-processors to scan for Repository interfaces. For each, it creates a JDK or CGLIB proxy backed by SimpleJpaRepository." 2. "Standard methods (`findById` , `findAll` , `save` , `delete`) are implemented in SimpleJpaRepository — uses EntityManager under the hood." 3. "Method-name queries: `findByMerchantIdAndStatusOrderByCreatedAtDesc(String, OrderStatus)` is parsed — generates `SELECT * FROM orders WHERE merchant_id = ? AND status = ? ORDER BY created_at DESC` ." 4. "@Query lets you write JPQL or native SQL: `@Query(\"SELECT o FROM Order o WHERE o.amount > :min\")` . Use `nativeQuery=true` for raw SQL." 5. "For complex queries, define a custom interface (`OrderRepositoryCustom`) with an implementation class (`OrderRepositoryImpl`) — Spring Data wires both together."

Edge cases to mention: - Method-name parsing has limits — anything beyond ~3 conditions or with OR logic gets unreadable. Use `@Query` or Specifications - `@Modifying` is required on `@Query` for UPDATE/DELETE — Spring otherwise treats them as SELECTs - For pagination, return `Page<T>` from the method — Spring generates COUNT + SELECT with LIMIT/OFFSET

What NOT to say: - "Spring Data uses bytecode generation at compile time" — actually runtime proxies - "Just write SQL for everything" — defeats the purpose of derived queries for simple cases

Common follow-up: "How does Spring Data JPA handle pagination and sorting?"

Q126 · @Async — how Spring runs async methods

Asked at: Most Spring backend rounds · Razorpay · Swiggy

Difficulty: Medium · **Topic:** Spring Core

The question: "How does Spring's @Async annotation work? What's the common gotcha?"

The 30-second answer: `@Async` on a public method makes the call return immediately — the actual execution runs on a TaskExecutor thread. Requires `@EnableAsync` on a config class. Common gotcha: self-invocation. Calling `this.asyncMethod()` from within the same bean bypasses the AOP proxy, so the method runs synchronously.

Step-by-step thought process: 1. "`@EnableAsync` activates the support. Configure the `TaskExecutor` (`ThreadPoolTaskExecutor`) — otherwise Spring uses a default that can OOM." 2. "`@Async` on a public method: caller gets back immediately. Return void or `CompletableFuture<T>` (or `Future<T>`). Spring runs the method on the executor." 3. "The mechanism is AOP. Spring wraps the bean in a proxy; the proxy intercepts the call and submits a `Runnable` to the executor." 4. "Self-invocation: if method A in bean X calls `this.B()` and B is `@Async`, the call bypasses the proxy — runs sync. Inject yourself via a `@Lazy` self-reference or restructure the code." 5. "Exception handling: void async methods can't propagate exceptions to the caller. Use a `@AsyncUncaughtExceptionHandler` for void; with `CompletableFuture`, exceptions flow through the future."

Edge cases to mention: - Don't use the default `SimpleAsyncTaskExecutor` in production — it creates a new thread per task with no pool - `@Async` on private methods is ignored — AOP requires public - For Spring Boot 3.2+ with virtual threads, set `spring.threads.virtual.enabled=true` — async tasks use virtual threads

What NOT to say: - "Async makes my code faster automatically" — it shifts work to another thread; doesn't make the work faster - "`@Async` works on private methods" — silently doesn't

Common follow-up: "How do you handle exceptions from a void `@Async` method?"

Q127 · Reflection — when is it appropriate?

Asked at: Senior Java rounds

Difficulty: Medium-Hard · **Topic:** Reflection

The question: "When would you actually use reflection in real code?"

The 30-second answer: Rarely in app code; commonly in frameworks. Use cases: dynamic instantiation when class is known only at runtime (plugins, deserialization), reading annotations (Spring scans for `@Component`, Jackson reads `@JsonProperty`), generic-purpose tooling (test runners, ORMs). For app code, prefer interfaces and DI over reflection.

Step-by-step thought process: 1. "Reflection lets you inspect classes, fields, methods at runtime, and invoke them dynamically. Powerful but slow and breaks encapsulation." 2. "Frameworks use it everywhere — Spring's DI, Hibernate's entity mapping, Jackson's serialization. As a framework author, reflection is essential." 3. "App-code use cases: plugin loading (`Class.forName(name).getConstructor().newInstance()`), reading annotations (`field.getAnnotation(JsonProperty.class)`)." 4. "Performance: reflective call is 10-100x slower than direct call. JIT can sometimes optimize but don't assume. Cache `Method` objects if hot." 5. "Java 9+ module system restricts reflective access to non-exported packages — `setAccessible(true)` may fail without explicit `--add-opens`."

Edge cases to mention: - `MethodHandles` (since Java 7) is the modern alternative — similar power, much better performance, JIT-friendly - `VarHandle` (Java 9) for atomic operations on fields without

writing AtomicXxx wrappers - Avoid `setAccessible(true)` if you can — fragile across JDK upgrades

What NOT to say: - "Reflection is always too slow to use" — fine for non-hot paths, frameworks use it everywhere - "Reflection is the way to call private methods" — it works but indicates a design smell

Common follow-up: "What's the difference between `Method.invoke()` and `MethodHandle.invokeExact()`?"

Q128 · Default methods in interfaces

Asked at: Most modern Java rounds

Difficulty: Easy-Medium · **Topic:** OOP / Java 8

The question: "What problem do default methods in interfaces solve? What are the multiple-inheritance gotchas?"

The 30-second answer: Default methods (Java 8) let you add methods to interfaces without breaking existing implementations. Used to extend `Collection` with `stream()`, `forEach()`, etc. Multiple-inheritance gotcha: if a class inherits the same default method from two interfaces, compile error — must override and explicitly call `InterfaceA.super.method()`.

Step-by-step thought process: 1. "Pre-Java 8: adding a method to an interface broke every existing implementation. Default methods let you ship a default implementation." 2. "`interface Collection { default Stream<E> stream() { ... } }`" — every existing implementer gets `stream()` for free." 3. "Diamond problem: if class implements two interfaces with the same default method, the compiler can't choose — must override explicitly: `void m() { InterfaceA.super.m(); }` ." 4. "Default methods give you behavior sharing without inheritance — supports composition-style design with interfaces." 5. "`static` methods in interfaces (also Java 8) are factory methods (e.g. `List.of(...)`) — not inherited by implementers, called via interface name."

Edge cases to mention: - `private` methods in interfaces (Java 9) — for helper methods used by default methods, don't leak to implementers - Class-defined methods ALWAYS win over interface default methods (class > interface) - Functional interfaces can have default methods — but only one abstract method

What NOT to say: - "Default methods make Java multi-inheritance" — partial; only behavior, not state - "Always provide defaults" — sometimes you want implementers to implement explicitly

Common follow-up: "How does a class resolve a method when superclass and interface both define it?"

Q129 · Java memory model — happens-before in 30 seconds

Asked at: Senior backend rounds · Razorpay · Goldman

Difficulty: Hard · **Topic:** Concurrency / JVM

The question: "What is the happens-before relationship in the Java Memory Model?"

The 30-second answer: A formal relation that defines when one action's effects are visible to another. If action A happens-before action B, B sees A's writes. Established by: program order (within a thread), synchronized block exit/enter (same lock), volatile write/read (same variable), Thread.start before any action in started thread, all actions in a thread before another thread joins it.

Step-by-step thought process: 1. "Without happens-before, the JVM can reorder reads and writes for performance — observable to other threads as 'impossible' interleavings." 2. "Within a single thread, actions appear to execute in program order — happens-before is intra-thread by default." 3. "Synchronized: unlocking a monitor happens-before any later locking of the same monitor. So writes inside the first synchronized block are visible inside the second." 4. "Volatile: a write to a volatile field happens-before any later read of the same field. Two volatile writes-and-reads chain visibility." 5. "Thread lifecycle: `Thread.start()` happens-before anything the started thread does. Anything the thread does happens-before `Thread.join()` returns."

Edge cases to mention: - Final fields: writes inside the constructor happen-before any other thread sees the constructed reference — if the reference is published safely - `Atomic*` operations and many `java.util.concurrent` classes provide happens-before — that's why you don't need to think about it constantly when using them - A `ConcurrentHashMap.put` happens-before subsequent `get` for the same key in another thread

What NOT to say: - "Synchronized makes everything visible" — only between blocks on the same lock - "Volatile is just visibility, nothing more" — also has happens-before with subsequent reads

Common follow-up: "Why does volatile guarantee visibility but synchronized also covers atomicity?"

Q130 · "What would you change in Java if you could?"

Asked at: Closing question, mid-to-senior rounds

Difficulty: Soft / signal-question · **Topic:** Self-awareness

The question: "If you could change one thing about Java, what would it be?"

The 30-second answer: This is a signal question testing if you've engaged with the language critically. Pick something specific and grounded — not "Java is too verbose." Strong picks: checked exceptions (or their interaction with lambdas), null references (long-standing pain), erasure preventing reified generics, default constructor verbosity vs records.

Step-by-step thought process: 1. "Signal test: are you a thoughtful engineer or someone who never reflects? Wrong: 'Java is perfect.' Right: pick a specific friction and discuss the trade-off." 2. "Strong pick — null: 'I'd push harder for Optional-by-default in APIs or null-safety in the type system like Kotlin. NullPointerException is still the most expensive runtime bug.'" 3. "Strong pick — checked exceptions in lambdas: 'I'd revisit how checked exceptions compose with lambdas — currently you wrap every checked-throwing function manually.'" 4. "Strong pick — erasure: 'Reified generics (a la Valhalla) would

eliminate a whole class of workarounds. The fact that you can't do `new T[]` is a daily annoyance.'" 5. "Show curiosity, not bitterness: 'I know Valhalla is working on value types and reified generics — exciting direction.'"

Edge cases to mention: - Tie it to your actual experience: 'In a payment service, null on a deeply nested field bit us hard last quarter — I'd love compile-time null safety.' - Mention what's GETTING fixed: 'Pattern matching closed a real gap; sealed classes solve hierarchy problems we used to fake with enum-like patterns.' - Don't pick something objectively fixed: 'I wish there were lambdas' (added in Java 8) is a no-go

What NOT to say: - "Nothing, Java is perfect" — signals you don't think critically - "Java is too verbose" — vague, says nothing - "I'd add async/await" — Java solved I/O concurrency differently with virtual threads; shows you don't know

Common follow-up: "What's your take on Project Valhalla?"

Tier B — Asked in 25-50% of Java interviews (70 Qs)

These are the depth-layer questions — they don't show up in every interview, but when they do, getting them right is the difference between an "okay" round and a "this candidate is senior" signal. Skim Tier B the week of your interview; focus harder if you're targeting senior or staff-level roles, or if the JD lists a specific area (Spring Security, JVM tuning, JPA performance) in detail.

Q131 · Designing a custom ThreadPoolExecutor — what knobs matter?

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · JP Morgan India

Difficulty: Medium-Hard · **Topic:** Concurrency advanced

The question: "You can't use `Executors.newFixedThreadPool()` in production. Build me a thread pool from scratch — what parameters do you pick and why?"

The 30-second answer: Seven knobs — `corePoolSize`, `maximumPoolSize`, `keepAliveTime`, `BlockingQueue` choice, `ThreadFactory` (named threads for logs), `RejectedExecutionHandler`, and `allowCoreThreadTimeOut`. The queue choice is the highest-impact decision — `LinkedBlockingQueue` (unbounded) makes `maximumPoolSize` meaningless, `SynchronousQueue` forces immediate handoff.

Step-by-step thought process: 1. "`Executors.newFixedThreadPool` uses an unbounded `LinkedBlockingQueue` — under load, tasks pile up and you OOM before threads spawn. That's why we avoid it." 2. "For a payments-callback worker, I'd use `new ThreadPoolExecutor(8, 32, 60, SECONDS, new ArrayBlockingQueue<>(500), factory, CallerRunsPolicy)` — bounded queue, capped threads, and back-pressure when full." 3. "`ThreadFactory` must name threads — `razorpay-webhook-worker-1`, `2`, `3`. Default thread names like `pool-3-thread-7` make production thread dumps unreadable." 4. "`RejectedExecutionHandler: AbortPolicy` throws (good for

fail-fast APIs), `CallerRunsPolicy` makes the submitter run it (natural back-pressure), `DiscardOldest` for telemetry-style workloads where stale events don't matter."

Edge cases to mention: - A pool with `LinkedBlockingQueue` never grows past `corePoolSize` — `maximumPoolSize` is dead config unless the queue is bounded or `SynchronousQueue` - `allowCoreThreadTimeOut(true)` lets even core threads die after `keepAlive` — useful for low-traffic services - For Java 21+, virtual threads via `Executors.newVirtualThreadPerTaskExecutor()` replace most blocking-I/O pools entirely

What NOT to say: - "I'd use `Executors.newCachedThreadPool`" — unbounded thread creation, will exhaust the OS process limit - "Pick `maximumPoolSize` = number of cores" — wrong for I/O-bound work; cores matter for CPU-bound only

Common follow-up: "What's the difference between `submit()` and `execute()` on this pool?"

Q132 · ForkJoinPool work-stealing — how does it actually work?

Asked at: Senior rounds at Flipkart · Walmart Labs · Goldman · Morgan Stanley

Difficulty: Hard · **Topic:** Concurrency advanced

The question: "Explain `ForkJoinPool`'s work-stealing algorithm. Why is it different from a regular thread pool?"

The 30-second answer: Each worker thread has its own deque. Tasks are pushed/popped from the LIFO end (cache-friendly). When a worker's deque is empty, it steals from the FIFO end of a *random* victim's deque — this minimizes contention and keeps cache-hot work local.

Step-by-step thought process: 1. "Regular pools have one shared queue — every worker contends for the head. With many short tasks, that lock becomes a bottleneck." 2. "`ForkJoinPool` gives each worker its own deque. Local work — push/pop from the top, LIFO, no synchronization on the fast path." 3. "When a worker runs dry, it picks a random victim and steals from the bottom of that deque, FIFO order — stealing happens on the opposite end from local work, minimizing CAS contention." 4. "`fork()` pushes onto your own deque, `join()` either runs the task (if still on your deque) or helps the stealer — workers never just block waiting."

Edge cases to mention: - `ForkJoinPool.commonPool()` is shared across the JVM — parallel streams use it, so blocking inside a parallel stream starves every other parallel stream - Submitting blocking I/O to a FJP is an anti-pattern — use `ManagedBlocker` to inform the pool a thread will block, so it can spawn a compensator - Parallelism level defaults to `Runtime.availableProcessors()` - `1` — explicit constructor takes a parallelism arg

What NOT to say: - "It's just a faster thread pool" — misses the algorithmic point; interviewer wants the deque-per-worker detail - "Work-stealing means threads share queues" — wrong direction; the *point* is they don't share, they only steal occasionally

Common follow-up: "What's the right pool for blocking I/O — `ForkJoinPool`, `ThreadPoolExecutor`, or `virtual threads`?"

Q133 · Happens-before relationship — what does it formally guarantee?

Asked at: Senior rounds at Razorpay · Cred · Flipkart · Goldman

Difficulty: Hard · **Topic:** Concurrency advanced

The question: "What does 'happens-before' mean in the Java Memory Model? Give me three concrete examples."

The 30-second answer: Happens-before is a partial ordering that says: if A happens-before B, then the effects of A (writes, side effects) are visible to B. Without it, the JVM and CPU can reorder operations freely. Three examples — monitor unlock-then-lock on the same lock, volatile write-then-read of the same field, `Thread.start()` before any action in the new thread.

Step-by-step thought process: 1. "The JMM allows aggressive reordering — compiler hoists loads, CPU executes out-of-order. Happens-before is the one guarantee about what writes another thread is allowed to see." 2. "Monitor rule — `synchronized` exit happens-before next `synchronized` entry on the same object. That's why writes inside one synchronized block are visible to the next thread that enters." 3. "Volatile rule — write to a volatile field happens-before every subsequent read of that field by any thread." 4. "Thread rules — `Thread.start()` happens-before any action in the new thread; final actions in a thread happen-before `Thread.join()` returns." 5. "Transitivity — if A happens-before B and B happens-before C, then A happens-before C. This is how you reason about chains across multiple locks/volatiles."

Edge cases to mention: - `final` fields have their own rule — properly constructed objects (no `this` leak in constructor) guarantee final-field visibility without sync - Double-checked locking *requires* `volatile` on the instance field — without it, partially-constructed object is visible to a second thread - Happens-before doesn't imply real-time ordering — it's about visibility, not wall-clock sequence

What NOT to say: - "It means A runs before B" — wrong, that's real-time; happens-before is about *visibility* - "Synchronized always gives happens-before" — only on the *same lock*, not across different locks

Common follow-up: "Why does `volatile` not solve the read-modify-write race for a counter?"

Q134 · Volatile vs synchronized — when each fails

Asked at: Senior rounds at Swiggy · PhonePe · Razorpay · TCS Digital

Difficulty: Medium-Hard · **Topic:** Concurrency advanced

The question: "Give me a case where `volatile` is enough and a case where it isn't."

The 30-second answer: `volatile` guarantees visibility of writes and prevents reordering across the volatile access — enough for single-write flags (`boolean shutdown`). It is NOT enough for compound operations like `counter++` because increment is read-modify-write — two threads can read the same value, increment, and write back losing one update.

Step-by-step thought process: 1. " `volatile boolean running = true` works for a worker loop — one thread writes false to stop, others see it on next read. No race because there's only one writer of the flag." 2. " `volatile int counter; counter++` does NOT work — that's three operations (load, add, store). Two threads can read 5, both compute 6, both store 6 — one increment lost." 3. "For counters use `AtomicInteger` (uses CAS), or `LongAdder` (better under contention — striped counters merged on read)." 4. " `synchronized` gives both visibility AND atomicity — but at the cost of blocking and a happens-before edge on every entry/exit."

Edge cases to mention: - `volatile` arrays — the *reference* is volatile, not the elements. Use `AtomicReferenceArray` for element-level guarantees - Double-checked locking pattern *needs* `volatile` on the instance — otherwise the partially-constructed object publish race kicks in - `volatile long` and `volatile double` — pre-Java 5, 64-bit reads weren't atomic on 32-bit JVMs; volatile fixes that

What NOT to say: - "Volatile is just synchronized but lighter" — wrong; they solve different problems - "Atomic is always faster than synchronized" — under heavy contention, CAS retry loops can be slower than a fair lock

Common follow-up: "How does `AtomicInteger` achieve thread-safety without locks?"

Q135 · Lock-free data structures — how does `ConcurrentLinkedQueue` work?

Asked at: Staff+ rounds at Cred · Flipkart · Goldman · Walmart Labs

Difficulty: Hard · **Topic:** Concurrency advanced

The question: " `ConcurrentLinkedQueue` claims to be non-blocking. How? What algorithm does it use?"

The 30-second answer: It uses the Michael-Scott non-blocking queue algorithm — head and tail pointers updated via CAS (compare-and-swap), not locks. Enqueue: CAS the tail's next pointer to the new node, then CAS the tail pointer forward. Dequeue: CAS the head pointer forward. Readers can see a stale tail briefly — they help advance it.

Step-by-step thought process: 1. "The structure is a singly-linked list with head and tail references, both `volatile`. No locks anywhere on the fast path." 2. "Enqueue is two steps — CAS `tail.next` from null to the new node, then CAS `tail` to point at the new node. The second CAS can fail; another thread will help by retrying." 3. "That's the 'helping' pattern — non-blocking algorithms often have threads cooperate to finish each other's in-progress operations, so no thread can stall the others." 4. " `size()` is $O(n)$ — it traverses the list — because maintaining a counter would require contention. Don't call `size()` in hot paths."

Edge cases to mention: - The ABA problem — if a node is removed and re-added, a CAS comparing the reference returns true even though the queue changed. JDK uses `AtomicMarkableReference` / `AtomicStampedReference` to add a counter - `ConcurrentLinkedQueue` is unbounded — if producers outpace consumers, memory grows; use `LinkedBlockingQueue` with a capacity for back-pressure - Memory consistency — operations are happens-before consistent across enqueue/dequeue, so handoff semantics are safe

What NOT to say: - "It uses fine-grained locks per node" — wrong; that would be a different data structure (lock-coupling) - "It's the same as ArrayDeque" — wrong; ArrayDeque is single-threaded

Common follow-up: "What's false sharing and how would it affect this queue?"

Q136 · False sharing — what is it and how do you fix it?

Asked at: Staff rounds at Flipkart · Cred · Goldman · Morgan Stanley

Difficulty: Hard · **Topic:** Concurrency advanced

The question: "Explain false sharing. Why does it matter? How does Java solve it?"

The 30-second answer: CPUs cache memory in 64-byte cache lines. If two threads modify two different variables that happen to sit in the same cache line, each modification invalidates the other CPU's cache — so you get cache-line ping-pong even though there's no logical sharing. Fix: padding or `@Contended` annotation.

Step-by-step thought process: 1. "Modern CPUs don't fetch one byte at a time — they fetch a cache line, typically 64 bytes. The MESI cache coherence protocol invalidates the line on any write." 2. "Two `long` variables (8 bytes each) sitting next to each other in a class share a cache line. Thread A writes one, Thread B writes the other — both lines invalidate constantly. Performance falls off a cliff." 3. "Java 8 introduced `@jdk.internal.vm.annotation.Contended` — JVM pads the field with 128 bytes on each side. Used inside `LongAdder`, `Striped64`, `ConcurrentHashMap`'s counter cells." 4. "For application code you can't use the internal annotation directly without `-XX:-RestrictContended`. Manual padding (7 dummy longs) is the workaround."

Edge cases to mention: - 128 bytes of padding (not 64) — some CPUs prefetch adjacent cache lines, so two-line padding is safer - `LongAdder` beats `AtomicLong` under contention precisely because of `@Contended` padding on its striped cells - JOL (Java Object Layout) library can show actual field offsets — use it to diagnose suspected false sharing

What NOT to say: - "It's a Java bug" — no, it's a CPU-level effect; Java only exposes it - "Use `synchronized` to fix it" — opposite of useful; sync makes contention explicit instead of hidden

Common follow-up: "Walk me through LongAdder vs AtomicLong — when do you prefer which?"

Q137 · ReadWriteLock vs StampedLock — when do you upgrade?

Asked at: Senior rounds at Razorpay · Swiggy · Goldman

Difficulty: Medium-Hard · **Topic:** Concurrency advanced

The question: "You have a cache that's read 100x more than written. Walk me through the lock options."

The 30-second answer: `ReentrantReadWriteLock` lets multiple readers in concurrently and locks out readers when a writer wants in — but readers still acquire a real lock (CAS on a counter).

`StampedLock` (Java 8+) adds an optimistic read mode — no lock, just a version stamp you validate after reading.

Step-by-step thought process: 1. "`ReentrantReadWriteLock` — readers share, writer is exclusive. Fair vs unfair mode controls writer starvation. Downside: read lock still does a CAS, which hurts under massive read contention." 2. "`StampedLock` — `tryOptimisticRead()` returns a stamp without acquiring anything. You read your fields, then call `validate(stamp)` — if anyone wrote, `validate` returns false and you fall back to a real read lock." 3. "That makes pure reads almost free under low write contention — typical web cache pattern." 4. "StampedLock is NOT reentrant and has no Condition variables — it's a lower-level primitive. Don't use it when you need either."

Edge cases to mention: - Optimistic read returns 0 if a write is in progress — must fall back to `readLock()` - Lock upgrade (read → write) requires `tryConvertToWriteLock(stamp)` — returns 0 on failure; never just acquire write after read or you deadlock - `ReentrantReadWriteLock` reader can be starved by a steady stream of writers in fair mode — measure before assuming fairness is free

What NOT to say: - "ReadWriteLock is always faster than synchronized" — false under heavy write contention; the read/write tracking has overhead - "Just use ConcurrentHashMap" — fine for maps, but ReadWriteLock generalizes to any shared structure

Common follow-up: "What's the deadlock risk with lock upgrade?"

Q138 · CompletableFuture — composing async pipelines

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Concurrency advanced

The question: "I need to call three downstream services in parallel and combine the results. Show me the `CompletableFuture` chain."

The 30-second answer: `allOf(f1, f2, f3).thenApply(v -> combine(f1.join(), f2.join(), f3.join()))` — `allOf` returns a `CompletableFuture<Void>` that completes when all three complete; then you join each (safe because they're already done) and combine. Use `supplyAsync(..., executor)` to feed each one with your own pool.

Step-by-step thought process: 1. "Default executor is `ForkJoinPool.commonPool()` — fine for CPU work, bad for blocking I/O. Pass your own `Executor` to every `Async` method." 2. "For three independent downstream calls:

```
CompletableFuture<PaymentStatus> p = supplyAsync(() -> paymentApi.fetch(id), pool);
CompletableFuture<UserProfile> u = supplyAsync(() -> userApi.fetch(id), pool);
```

— etc." 3. "Combine with `allOf` — `allOf(p, u, o).thenApply(v -> new OrderView(p.join(), u.join(), o.join()))` . `join()` won't block here because `allOf` only completes when each underlying is done." 4. "For exception handling: `.exceptionally(ex -> fallback)` or `.handle((result, ex) -> ...)` — `handle` gets both, `exceptionally` only the failure path."

Edge cases to mention: - `thenApply` vs `thenApplyAsync` — the first runs on whatever thread completed the previous stage (could be the caller); `Async` re-submits to your executor - `join()` vs `get()` — `join` throws unchecked `CompletionException`, `get` throws checked; prefer `join` in pipelines - `orTimeout(2, SECONDS)` (Java 9+) cancels the future after the timeout instead of forever-blocking — critical for production

What NOT to say: - "Just use `Thread.sleep` between calls" — sequential, defeats the point - "Future is the same as `CompletableFuture`" — Future has no chaining, no completion handlers, no combiner methods

Common follow-up: "What's `thenCompose` and when do you use it over `thenApply`?"

Q139 · Virtual threads pinning — what causes it and what's the fix?

Asked at: Senior rounds at Razorpay · Swiggy · Cred · Walmart Labs

Difficulty: Hard · **Topic:** Concurrency advanced

The question: "Virtual threads in Java 21 — what does 'pinning' mean and when does it happen?"

The 30-second answer: A virtual thread normally parks itself when it blocks, freeing its carrier (platform thread) to run another virtual thread. Pinning means the virtual thread *can't* unmount and instead blocks the carrier — happens inside `synchronized` blocks and JNI calls in Java 21. Fix: replace `synchronized` with `ReentrantLock`. Java 24 (JEP 491) removes the `synchronized` pinning entirely.

Step-by-step thought process: 1. "Virtual threads are M:N — many virtual threads multiplexed over a small pool of platform carriers. Mounting/unmounting on blocking is the whole point." 2. "Inside `synchronized`, the JVM tracks the monitor on the native stack of the carrier — if the virtual thread unmounted, the monitor would be lost. So Java 21 just pins." 3. "In Java 21-23, replace hot `synchronized` blocks with `ReentrantLock` — its state lives on the heap, no native stack dependency, so the virtual thread can unmount and another virtual thread runs on the carrier." 4. "From Java 24 (JEP 491) onwards this is fixed — `synchronized` no longer pins. So this question's importance fades over time, but for any service still on Java 21 LTS, it matters."

Edge cases to mention: - Detect pinning with `-Djdk.tracePinnedThreads=full` — logs the stack trace every time a virtual thread pins - JNI / native code pins too — unavoidable when calling C libraries; usually rare enough to ignore - `Object.wait()` inside synchronized still pins until Java 24

What NOT to say: - "Virtual threads replace ThreadPoolExecutor everywhere" — not for CPU-bound work; carriers are still limited to core count - "synchronized is deprecated in Java 21" — no, just imperfect with virtual threads on that version

Common follow-up: *"How would you migrate a JDBC-heavy service from platform threads to virtual threads?"*

Q140 · Reentrant lock fairness — when to set it true?

Asked at: Senior rounds at JP Morgan India · Goldman · Wells Fargo

Difficulty: Medium · **Topic:** Concurrency advanced

The question: " `ReentrantLock` has a fairness flag — what does it do and when do you turn it on?"

The 30-second answer: Fair mode grants the lock to threads in FIFO order of arrival — no thread starves. Unfair (default) lets a thread that's about to release the lock barge ahead of waiting threads — much higher throughput but the unlucky waiter can wait a long time. Use fair only when you genuinely need bounded wait time.

Step-by-step thought process: 1. "Default is unfair — it's barge-friendly. Total throughput is much higher because the just-released-the-lock thread is hot in cache, so back-to-back acquires are cheap." 2. "Fair mode forces queueing — every acquire goes through the wait queue. Throughput can be 5-10x lower, but the longest wait is bounded by N other waiters." 3. "Reach for fair when you have a constrained shared resource (e.g., 10 DB connections, 100 threads competing) and you need to prevent starvation that breaks SLAs." 4. "Don't use fair as a default — most code wants throughput, not fairness."

Edge cases to mention: - `tryLock()` (without timeout) barges even in fair mode — it doesn't honour the queue - `Semaphore` has the same fairness flag with the same semantics - `ReentrantReadWriteLock` fairness applies to both reader and writer queues — fair readers can be starved by a steady writer

What NOT to say: - "Always use fair mode for safety" — performance hit is real; only use when starvation is a known problem - "Fair means random" — no, fair means FIFO

Common follow-up: *"Compare ReentrantLock and synchronized — when do you reach for which?"*

Q141 · G1 vs ZGC vs Shenandoah — pick a collector

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Walmart Labs · Cred

Difficulty: Hard · **Topic:** JVM tuning + GC

The question: "You have a 32GB heap service with strict p99 latency. Which GC and why?"

The 30-second answer: For 32GB heap with strict tail latency, **ZGC** (default since Java 15 for tuning targets, generational since Java 23) — sub-millisecond pause times regardless of heap size. G1 is the safe default at smaller heaps (4-16GB) with mixed throughput/latency goals. Shenandoah (Red Hat / OpenJDK) is similar to ZGC in goals, used in RHEL ecosystems.

Step-by-step thought process: 1. "G1 (default since Java 9) — region-based, divides heap into ~2048 regions, collects highest-garbage regions first. Stop-the-world young GC, concurrent mixed/old. Pause target tunable via `-XX:MaxGCPauseMillis`. Good up to ~16GB heaps." 2. "ZGC (production since 15) — colored pointers, load barriers, fully concurrent. Pause times under 1ms even at 100GB+ heaps. Generational ZGC (default in 23) added young/old separation for throughput." 3. "Shenandoah — similar story, concurrent compaction via Brooks pointers / forwarding. Used heavily in JBoss/WildFly stacks." 4. "Trade-offs — ZGC has higher CPU overhead (load barriers on every reference read) and higher heap footprint. For a 4GB latency-tolerant service, G1 is cheaper."

Edge cases to mention: - Parallel GC is still default for `-XX:+UseParallelGC` workloads optimizing pure throughput (batch jobs) - Epsilon GC (`-XX:+UseEpsilonGC`) is no-op — allocates only, never collects. Used for short-lived benchmarks and OOM testing - Don't switch GCs blindly — profile, look at GC logs, measure p99 before tuning

What NOT to say: - "ZGC is always best" — overhead and complexity aren't free - "G1 is deprecated" — no, it's the JVM default for most heaps

Common follow-up: "Show me how you'd read a G1 GC log to find a problem."

Q142 · Reading a GC log — what to look for first?

Asked at: Senior rounds at Swiggy · Razorpay · Goldman · Morgan Stanley

Difficulty: Medium-Hard · **Topic:** JVM tuning + GC

The question: "Here's a GC log. Walk me through what you'd look at first."

The 30-second answer: Four checks — total pause time as a percentage of wall time (>5% is bad), individual long pauses (anything over your latency budget), full GC frequency (full GC in G1 is a failure mode), and allocation rate (high allocation rate causes high young GC frequency).

Step-by-step thought process: 1. "Enable structured logs — `-Xlog:gc*:file=gc.log:time,uptime,level,tags`. Don't try to read raw lines; pipe through `gceasy.io` or `GCViewer` for the summary." 2. "First number — % time in GC. Healthy is under 2%; 5%+ is degraded; 10%+ is broken. This tells you whether you have a GC problem at all." 3. "Look at p99 / p99.9 pause durations — if your SLA says p99 = 50ms and you have a 200ms G1 mixed pause, you've found the culprit." 4. "Full GC frequency — in G1, a full GC means concurrent cycle failed (humongous objects, evacuation failure). Look at the cause string: `to-space exhausted`, `humongous allocation`,

`metadata GC threshold` . " 5. "Allocation rate — if young gen fills every 200ms, you have a high allocation pressure. Profile with JFR to find the allocation hot spots."

Edge cases to mention: - Humongous allocations (>50% of region size in G1) skip young gen and go straight to old — fix by reducing object size or upping region size - Time-to-safepoint can be huge even when GC itself is fast — measured as `Total time for which application threads were stopped` minus actual GC work - `Concurrent Mark Cycle` aborts in G1 mean the concurrent phase didn't finish in time — usually allocation rate too high

What NOT to say: - "I'd just increase the heap" — bigger heap = longer pauses for non-ZGC collectors; that's the wrong knob most of the time - "Restart the JVM" — addresses symptom, not the leak/pressure

Common follow-up: "What's a humongous object in G1 and why does it matter?"

Q143 · Heap dump analysis with MAT — finding the leak

Asked at: Senior rounds at Razorpay · Swiggy · TCS Digital · JP Morgan India

Difficulty: Medium-Hard · **Topic:** JVM tuning + GC

The question: "Your service OOMs every 12 hours. Walk me through your investigation."

The 30-second answer: Capture a heap dump with `jmap -dump:live,format=b,file=heap.hprof <pid>` or auto-on-OOM via `-XX:+HeapDumpOnOutOfMemoryError` . Open in Eclipse MAT. Run Leak Suspects report — it identifies dominator trees holding the most retained heap. Drill into the GC roots for the top suspect.

Step-by-step thought process: 1. "Always enable `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/log/dumps` in production. When the OOM happens, you have the evidence." 2. "Open the .hprof in MAT (Memory Analyzer Tool). Run Leak Suspects Report — gives you a list of objects that dominate the retained heap." 3. "Drill into the top suspect — view the dominator tree, look at retained size (not shallow size). A 100MB `ConcurrentHashMap` is a leak; a 100MB `byte[]` is probably a cache." 4. "Right-click → Path to GC Roots → exclude weak/soft references. That gives you the chain of strong references keeping the object alive — usually points at a static field, a `ThreadLocal`, or a listener registration."

Edge cases to mention: - Common Java leak patterns — unbounded caches, `ThreadLocal` not removed in pooled threads, listener/observer registrations not removed, classloader leaks in app servers - Heap dumps are sensitive (contain user data, session tokens) — encrypt at rest, restrict access - `jcmd <pid> GC.heap_dump <path>` works without going through jmap, often more reliable in containerized environments

What NOT to say: - "I'd add more heap" — masks the leak, eventually OOMs again - "I'd just restart the service" — same; the leak is real, find it

Common follow-up: "What's the difference between shallow heap, retained heap, and dominator tree?"

Q144 · jcmd, jstat, jmap — when to reach for which?

Asked at: Senior rounds at Swiggy · Razorpay · Goldman · Walmart Labs

Difficulty: Medium · **Topic:** JVM tuning + GC

The question: "You're SSH'd into a prod box with a misbehaving Java process. What's your toolkit?"

The 30-second answer: `jcmd` is the modern Swiss-army — it can do everything `jmap/jstack/jinfo` used to do. Reach for `jstat` for cheap GC stats over time. Avoid `jmap -histo:live` in prod — it triggers a full GC.

Step-by-step thought process: 1. "`jcmd <pid> help` — lists every diagnostic command the JVM exposes. The modern recommendation is to use `jcmd` over the older tools." 2. "`jcmd <pid> GC.heap_info` — quick view of generation sizes and usage. `jcmd <pid> Thread.print` — thread dump, same as `jstack`." 3. "`jstat -gcutil <pid> 1000` — prints young/old gen utilization every second. Lightweight, doesn't pause the JVM. Use it to spot allocation pressure live." 4. "`jcmd <pid> GC.heap_dump /tmp/heap.hprof` — full heap dump. Heavy operation, pauses the JVM for seconds — schedule a maintenance window or do it on a canary instance."

Edge cases to mention: - `jmap -histo:live` does a full GC first to get accurate live counts — never run on a prod node serving traffic - JFR (`jcmd <pid> JFR.start`) is the modern profiling answer — low overhead, captures allocation, locks, I/O, GC together - Native memory tracking — `jcmd <pid> VM.native_memory_summary` shows JVM's view of native allocations (class metadata, code cache, GC overhead)

What NOT to say: - "I'd attach VisualVM" — fine for dev, but prod boxes rarely have a GUI session available - "Just enable verbose:gc" — that's a JVM flag, not a runtime diagnostic; needs restart

Common follow-up: "You see 95% old-gen utilization in `jstat`. What do you do next?"

Q145 · Native memory tracking — what's outside the heap?

Asked at: Senior rounds at Cred · Goldman · Walmart Labs · JP Morgan India

Difficulty: Hard · **Topic:** JVM tuning + GC

The question: "My JVM has `-Xmx4g` but the process is using 7GB RSS. Where's the extra memory?"

The 30-second answer: Java heap is one of many memory areas. Outside the heap: thread stacks (1MB each by default × N threads), Metaspace (class metadata), code cache (JIT-compiled native code), direct byte buffers, JNI allocations, GC overhead, and Compressed Class Space.

Step-by-step thought process: 1. "Enable NMT — `-XX:NativeMemoryTracking=summary` at startup. Then `jcmd <pid> VM.native_memory_summary` shows the breakdown." 2. "Common culprits — thread stacks (1000 threads × 1MB = 1GB), direct buffers (often misused for I/O, never freed),

Metaspace ballooning from dynamic class generation, code cache filling up from heavy JIT." 3. "Direct buffers (allocated via `ByteBuffer.allocateDirect`) live outside the heap, freed by phantom-reference cleaner threads — slow GC means slow cleanup means real memory pressure." 4. "Containers add a wrinkle — if your container limit is 4GB but Xmx is 4GB, you have zero headroom for the non-heap regions and the OOM-killer gets you."

Edge cases to mention: - For containers, set `-XX:MaxRAMPercentage=75` instead of `-Xmx` — JVM uses 75% of container limit for heap, leaves room for non-heap - Metaspace doesn't have a default cap until you set `-XX:MaxMetaspaceSize` — Hibernate proxies and bytecode-heavy apps can fill it - JNI memory is invisible to JVM tools — you need OS-level tools (pmap, gdb) to chase JNI leaks

What NOT to say: - "The JVM is leaking" — usually it's not; you're under-counting non-heap - "Increase Xmx" — doesn't help; the extra memory isn't in the heap

Common follow-up: "What's Compressed Oops and when does it stop working?"

Q146 · JIT compilation tiers — interpreter, C1, C2, Graal

Asked at: Staff rounds at Flipkart · Cred · Goldman · Walmart Labs

Difficulty: Hard · **Topic:** JVM tuning + GC

The question: "Explain the JVM's tiered compilation. Why does a method get slower before it gets faster?"

The 30-second answer: HotSpot starts every method in the bytecode interpreter (Tier 0). Hot methods promote to C1 (fast compile, light optimization — Tier 1-3), then to C2 (slower compile, aggressive optimization — Tier 4). Methods can deoptimize and bounce back to the interpreter if assumptions break (e.g., a new class loaded that invalidates a type check).

Step-by-step thought process: 1. "Interpreter — every JVM starts here. Bytecode interpreted instruction by instruction. Slow but no compile time." 2. "C1 (client compiler) — fast compile, basic optimization. Used as the warmup compiler. Captures profiling info: which branches taken, which types observed." 3. "C2 (server compiler) — slow compile, aggressive optimization. Inlining, escape analysis, vectorization, loop unrolling. The hot path." 4. "Deoptimization — C2's optimizations are speculative. If profiling said 'only `ConcreteImpl` here' and a new subclass shows up, the compiled code is invalidated and execution bounces back to the interpreter while C2 recompiles." 5. "GraalVM JIT — a Java-written C2 replacement with better optimization for polymorphic and lambda-heavy code. Default in Java 17+ via `-XX:+UseJVMCICompiler`."

Edge cases to mention: - Profile-guided benchmarks need warmup — JMH (Java Microbenchmark Harness) handles this; ad-hoc timing usually measures the wrong thing - AOT (ahead-of-time) compilation via `jaotc` or GraalVM native image trades startup for peak throughput - Inlining is the biggest C2 win — methods over `-XX:MaxInlineSize` (default 35 bytes) won't inline; refactor hot small methods accordingly

What NOT to say: - "Java is compiled" — partly true; the bytecode is, but execution is mostly JIT'd machine code - "C2 is always used" — only for hot methods; cold paths stay interpreted forever

Common follow-up: "What's escape analysis and what optimization does it enable?"

Q147 · Metaspace OOM — what causes it and how to diagnose

Asked at: Senior rounds at Razorpay · Cognizant · TCS Digital · Wells Fargo

Difficulty: Medium-Hard · **Topic:** JVM tuning + GC

The question: " `java.lang.OutOfMemoryError: Metaspace` . Diagnose. "

The 30-second answer: Metaspace holds class metadata, replaced PermGen in Java 8. OOM happens when too many classes are loaded — usually from dynamic class generation (CGLIB proxies, ByteBuddy, JSP recompilation, classloader leaks in app servers). Cap with `-XX:MaxMetaspaceSize` , diagnose with `jcmd <pid> GC.class_stats` .

Step-by-step thought process: 1. "PermGen → Metaspace transition in Java 8 — Metaspace is in native memory by default, sizes dynamically. The OOM means the JVM hit `MaxMetaspaceSize` (default unlimited, but containers usually cap it)." 2. "Top culprits — CGLIB / ByteBuddy proxies generating new classes per call instead of caching, Groovy / Kotlin scripting reloading, web containers reloading apps and leaking the old classloader." 3. "Diagnose with `-XX:+UnlockDiagnosticVMOptions -XX:+UnlockExperimentalVMOptions` then `jcmd <pid> GC.class_histogram` — shows class count and instance count." 4. "Look for thousands of classes with synthetic names like `Foo$$EnhancerByCGLIB$$abc123` — sign of proxy explosion."

Edge cases to mention: - Classloader leak — a single retained reference to a Class object pins the entire classloader and every class it loaded. Common in static maps holding user-provided classes - `-XX:MetaspaceSize` is the *initial* size that triggers a class-unload cycle; `-XX:MaxMetaspaceSize` is the hard cap - Hibernate's session factory builds enhanced entity classes — bytecode enhancement can balloon Metaspace if entity hierarchy is large

What NOT to say: - "Add more Metaspace" — same symptom-not-cause trap as heap OOM - "PermGen issue" — outdated; PermGen doesn't exist post-Java 7

Common follow-up: "What's a classloader leak and how do you find one?"

Q148 · String deduplication — what is it and when does it help?

Asked at: Senior rounds at Flipkart · Cred · Goldman

Difficulty: Medium · **Topic:** JVM tuning + GC

The question: "What does `-XX:+UseStringDeduplication` do? When is it worth turning on?"

The 30-second answer: G1 (and ZGC) can detect String objects whose internal `byte[]` (the character data) has identical contents and replace them with a shared array — leaving the String objects distinct but sharing their backing storage. Helps apps that hold many duplicate strings in memory (JSON parsing, ORM, logs).

Step-by-step thought process: 1. "Strings hold a `byte[]` (since Java 9 with compact strings) for their characters. Many identical strings means many identical byte arrays — pure waste." 2. " **-XX: +UseStringDeduplication** (G1 since Java 8, ZGC since 18) — during young GC, scan promoted Strings. If two Strings have the same content hash, point both at the same `byte[]` ." 3. "Sweet spot — JSON-heavy services where many entities share enum-like values (`status = 'PAID'` , `currency = 'INR'`). 20-40% String memory savings reported in production benchmarks." 4. "Not free — GC does extra work hashing and comparing. Measure first."

Edge cases to mention: - Only deduplicates after a few promotions (avoids deduping short-lived strings) - String interning (`String.intern()`) is a different mechanism — explicit, app-controlled, uses the String Pool table - For massive duplicate sets, consider a String cache (`Map<String, String>`) or pre-interning known values

What NOT to say: - "It interns every string" — wrong; that's `String.intern()` - "Doubles GC pause times" — usually under 5% overhead; measure before claiming

Common follow-up: "What's the difference between `String.intern()` and string deduplication?"

Q149 · Escape analysis and scalar replacement

Asked at: Staff rounds at Flipkart · Goldman · Walmart Labs · Cred

Difficulty: Hard · **Topic:** JVM tuning + GC

The question: "Why doesn't `new Order()` always allocate on the heap?"

The 30-second answer: C2 performs escape analysis — if it can prove an object never escapes its allocating method (no field assignment, no return, no thread publication), it eliminates the heap allocation via *scalar replacement* — the object's fields become local variables in registers or stack slots.

Step-by-step thought process: 1. "Escape analysis classifies allocations — no escape (stays in method), arg escape (escapes only to called methods), global escape (stored in field, returned, published)." 2. "For no escape, C2 doesn't actually allocate the object at all. Its fields become scalar locals — like inlining the constructor and accessing the fields directly." 3. "This means lots of intermediate objects (Optional wraps, Stream lambdas, small builders) cost nothing in steady state — they're optimized away." 4. "Allocation profiling with JFR can mislead — it counts objects the JIT will later eliminate. Interpret with care."

Edge cases to mention: - Synchronized blocks on a non-escaping object are eliminated entirely (**-XX: +EliminateLocks**) - Boxing — autoboxing primitive into wrappers is the #1 escape-killer; prefer

primitive types in hot loops - **Optional** is often elided by escape analysis when used purely within a method — return values are a different story

What NOT to say: - "Every **new** allocates on the heap" — outdated since C2 escape analysis - "Stack allocation" — Java doesn't actually allocate on the stack; it does scalar replacement

Common follow-up: *"Why does autoboxing kill performance even though escape analysis is so good?"*

Q150 · JFR — recording flight events in production

Asked at: Senior rounds at Razorpay · Walmart Labs · JP Morgan India

Difficulty: Medium-Hard · **Topic:** JVM tuning + GC

The question: *"What's JFR and why is it the modern profiling answer?"*

The 30-second answer: Java Flight Recorder is a low-overhead ($\approx 1\%$) JVM-built-in profiler that records GC, allocations, locks, I/O, exceptions, method profiling as events. Open the recording in JDK Mission Control (JMC). Always-on in prod with rolling buffers is a legitimate strategy.

Step-by-step thought process: 1. "Trigger via `jcmd <pid> JFR.start duration=60s filename=/tmp/run.jfr` for a one-shot, or

`-XX:StartFlightRecording=duration=60s,filename=...` at JVM startup." 2. "Continuous recording —

`-XX:StartFlightRecording=disk=true,maxsize=200M,maxage=2h,settings=profile`. Buffer stays on disk, automatically rolls. When something goes wrong, the data is already there." 3. "Open in JMC — flame graphs, allocation sites, lock contention, GC summary, I/O latency. Goes way beyond what `jstat` / `jstack` can show." 4. "JFR overhead — under 1% with default settings. Under 3% with **profile** settings. Safe to keep on permanently."

Edge cases to mention: - JFR was Oracle-only before Java 11 — now in OpenJDK - Event streaming API (Java 14+) lets you consume events in real-time without writing to disk — useful for in-process observability - Combine JFR + async-profiler for the best flame graphs — JFR for system-wide, async-profiler for sample-based CPU profiling

What NOT to say: - "Use VisualVM in production" — interactive sampling is high overhead and fragile - "JFR is for Oracle JDK only" — no, OpenJDK ships it since 11

Common follow-up: *"How would you set up always-on JFR in a Kubernetes pod?"*

Q151 · BeanPostProcessor — what does it actually do?

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Spring deep

The question: "What's a `BeanPostProcessor` ? Walk me through what runs when."

The 30-second answer: `BeanPostProcessor` has two callbacks —

`postProcessBeforeInitialization` (after dependency injection, before `@PostConstruct` / `InitializingBean.afterPropertiesSet` / `init-method`) and `postProcessAfterInitialization` (after init). Spring uses this hook to wrap beans in AOP proxies, apply annotations like `@Autowired` / `@Value` , and register lifecycle adapters.

Step-by-step thought process: 1. "Order of bean lifecycle — instantiate, populate properties (DI), `setBeanName` / `setBeanFactory` / `setApplicationContext` , `postProcessBeforeInitialization` , `@PostConstruct` , `afterPropertiesSet` , custom init, `postProcessAfterInitialization` , ready." 2. " `AbstractAutoProxyCreator` is a `BeanPostProcessor` — it's what wraps your `@Transactional` and `@Async` beans in CGLIB / JDK proxies during `postProcessAfterInitialization` ." 3. "To write your own — implement the interface, return a (possibly wrapped) bean from after-init. Use it for cross-cutting bean modification, custom annotations, or metric instrumentation." 4. " `BeanFactoryPostProcessor` is different — runs earlier, modifies bean definitions (not instances). Used by `PropertySourcesPlaceholderConfigurer` to resolve `${}` placeholders before any bean is created."

Edge cases to mention: - Order matters — implement `Ordered` or `PriorityOrdered` . Wrong order can apply AOP before `@Autowired` injection - `BeanPostProcessors` themselves aren't post-processed — they're created early and untouched - Use `@Lazy` carefully — lazy beans skip post-processing until first access, which can hide order bugs

What NOT to say: - "It's just a lifecycle callback" — undersells it; this is how AOP, `@Async` , `@Cacheable` , etc. all work - "Same as `@PostConstruct` " — no, BPP runs around `@PostConstruct`

Common follow-up: "How does Spring decide between JDK dynamic proxies and CGLIB?"

Q152 · BeanFactory vs ApplicationContext

Asked at: Senior rounds at TCS · Infosys · Cognizant · Wipro

Difficulty: Medium · **Topic:** Spring deep

The question: "What's the difference between `BeanFactory` and `ApplicationContext` ?"

The 30-second answer: `BeanFactory` is the minimal DI container — just lazy bean instantiation. `ApplicationContext` extends it with enterprise features: eager singleton instantiation, event publishing, internationalization (MessageSource), resource loading, environment / profile abstraction, and parent context chaining.

Step-by-step thought process: 1. " `BeanFactory` — the foundational interface. Lazy by default; beans only created on first `getBean()` . Useful for memory-constrained scenarios." 2.

" `ApplicationContext` — what Spring Boot wires up. Eagerly creates singletons at startup (so missing-bean errors surface immediately, not on first request). Publishes lifecycle events (`ContextRefreshedEvent` , etc.)." 3. " `ApplicationContext` adds — `MessageSource` (i18n), `ResourceLoader` (classpath / file URLs), `ApplicationEventPublisher` , `Environment` for properties and profiles." 4. "In modern Spring you never touch `BeanFactory` directly — you depend on `ApplicationContext` or, more typically, just inject the beans you need."

Edge cases to mention: - `BeanFactory#getBean` lazy is theoretical — in practice `ApplicationContext` eager init prevents subtle startup bugs - `ApplicationContext` is itself a `BeanFactory` — interface hierarchy gives full backward compatibility - `WebApplicationContext` extends `ApplicationContext` with `ServletContext` access for web modules

What NOT to say: - "They're basically the same" — true at the basic DI level, false at every enterprise feature - "BeanFactory is faster" — slower in steady state because startup-time eager init catches bugs early

Common follow-up: "What does `ContextRefreshedEvent` mean and when does it fire?"

Q153 · AOP weaving — compile-time, load-time, runtime

Asked at: Senior rounds at Swiggy · Razorpay · JP Morgan India · Walmart Labs

Difficulty: Hard · **Topic:** Spring deep

The question: "Spring AOP is one form of weaving. What are the others?"

The 30-second answer: Three weaving modes — *runtime proxy weaving* (Spring AOP — JDK or CGLIB proxies wrapping beans), *load-time weaving* (AspectJ LTW — bytecode rewritten by a Java agent at class load), and *compile-time weaving* (AspectJ AJC — bytecode modified by a custom compiler before runtime).

Step-by-step thought process: 1. "Spring AOP — proxy-based. `@Transactional` works because Spring wraps your bean in a proxy that intercepts methods. Limitations: only public methods, only external invocations (a self-call inside the same class bypasses the proxy)." 2. "AspectJ load-time weaving — `-javaagent:aspectjweaver.jar` rewrites classes as they load. No proxies, advice woven directly. Catches self-calls and private methods, but adds a JVM agent." 3. "Compile-time weaving — uses the AspectJ compiler (`ajc`) instead of `javac` . Produces classes with the advice baked in. Fastest at runtime, most invasive to build." 4. "Pick: Spring AOP for 90% of cases (transactions, security, logging on Spring beans). AspectJ when you need to advise non-Spring code or self-calls."

Edge cases to mention: - Self-invocation gotcha — `this.transactionalMethod()` from within the same class bypasses Spring AOP. Inject a self-reference or use `AopContext.currentProxy()` - `@EnableAspectJAutoProxy(proxyTargetClass = true)` forces CGLIB even for interface-based beans - AspectJ pointcut expressions are richer than Spring AOP's — full Java type matching, including private/static

What NOT to say: - "AspectJ is just like Spring AOP" — fundamentally different mechanism - "AOP is dead, use interceptors" — interceptors are AOP under a different name

Common follow-up: "Why does `@Transactional` not work on a self-called method?"

Q154 · Custom annotations + meta-annotations

Asked at: Senior rounds at Razorpay · Cred · Swiggy

Difficulty: Medium-Hard · **Topic:** Spring deep

The question: "Build me a `@RateLimited` annotation that's enforced by Spring."

The 30-second answer: Define the annotation with `@Retention(RUNTIME) @Target(METHOD)`. Write an `@Aspect` with a `@Around` advice targeting `@annotation(com.example.RateLimited)`. Inside the advice, look up the rate limiter bucket for the key (method name + caller IP), allow or reject. Stack with `@AliasFor` and other Spring meta-annotation tricks.

Step-by-step thought process: 1. "Annotation declaration — `@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD) public @interface RateLimited { int permitsPerSecond() default 10; }`" 2. "Aspect — `@Aspect @Component class RateLimitAspect { @Around("@annotation(rateLimited)") public Object check(ProceedingJoinPoint pjp, RateLimited rateLimited) throws Throwable { ... } }`. Spring passes the matched annotation in for free." 3. "Meta-annotation pattern — compose your own annotation that itself is annotated with `@RateLimited` + `@Transactional`. Spring's annotation discovery follows meta-annotations one level deep by default." 4. "`@AliasFor` — lets a custom annotation alias an attribute from a meta-annotation. Used in Spring's own `@GetMapping` aliasing `@RequestMapping(method = GET)`."

Edge cases to mention: - `RetentionPolicy.SOURCE` (Lombok), `CLASS` (default; in bytecode but not loaded), `RUNTIME` (reflectable). Spring needs `RUNTIME` - `@Inherited` only works for class-level annotations and only at one level - Aspect must be a Spring bean — otherwise it's just a class with no proxying

What NOT to say: - "I'd write a HandlerInterceptor" — works for controllers only, misses service-layer calls - "Annotations can do this by themselves" — no, you always need a runtime processor

Common follow-up: "What's the difference between `RetentionPolicy.SOURCE`, `CLASS`, and `RUNTIME`?"

Q155 · Spring AOT and Native Image

Asked at: Senior rounds at Swiggy · Razorpay · Cred · Walmart Labs

Difficulty: Hard · **Topic:** Spring deep

The question: "Spring Boot 3 added AOT and native image support. What problem does it solve?"

The 30-second answer: AOT (ahead-of-time) processing analyzes your beans at build time and generates static configuration — no reflection, no classpath scanning at runtime. Combined with GraalVM Native Image, this gives sub-100ms startup and 5-10x lower memory. Trade-off: build time goes up significantly and dynamic features (runtime classpath scan, JDK proxies for unknown interfaces) need explicit hints.

Step-by-step thought process: 1. "Standard Spring Boot startup — classpath scan, reflection-based bean instantiation, dynamic proxy generation. Takes seconds, loads metadata for every annotation." 2. "Spring Boot 3 + Spring Framework 6 — AOT processor runs at build time. Generates Java source files with the bean factory definitions, reflection hints, proxy classes. Runtime startup skips all the scanning." 3. "GraalVM Native Image — compiles your AOT-processed app to a static binary. Cold start under 100ms, RSS under 100MB. Critical for serverless / FaaS / scale-to-zero containers." 4. "Caveats — anything not in the AOT hints (`@RegisterReflectionForBinding` , `@ImportRuntimeHints`) fails at runtime. Spring covers most common cases; bring-your-own libraries may need work."

Edge cases to mention: - AOT in JVM mode (without native) still helps — about 20% startup improvement, no GraalVM needed - `spring-boot-starter-parent` 3.x integrates AOT into the build automatically — `mvn -Pnative package` - Profiles, conditional beans, and `@ConditionalOnProperty` are resolved at AOT time — you can't change `application.properties` at runtime to flip them on/off in native mode

What NOT to say: - "Native image is just a faster JVM" — different runtime, different limitations - "Use AOT to make my app faster at peak" — AOT helps startup; peak throughput often slightly lower than C2-optimized JVM

Common follow-up: "What's a reflection hint and when do I need one?"

Q156 · Conditional beans — @ConditionalOnProperty, @ConditionalOnClass

Asked at: Senior rounds at Razorpay · Swiggy · Walmart Labs

Difficulty: Medium · **Topic:** Spring deep

The question: "How do Spring Boot starters decide whether to auto-configure a feature?"

The 30-second answer: `@ConditionalOnClass` (a class is on the classpath), `@ConditionalOnMissingBean` (user didn't define their own), `@ConditionalOnProperty` (a property is set), `@ConditionalOnBean` (some other bean is registered). Auto-configurations are conditional declarations — they trigger or skip based on classpath + user config.

Step-by-step thought process: 1. "`spring-boot-starter-data-redis` adds Lettuce to the classpath. `RedisAutoConfiguration` has `@ConditionalOnClass(RedisClient.class)` — it activates because Lettuce is present." 2.

"`@ConditionalOnMissingBean(StringRedisTemplate.class)` — only registers if you haven't defined your own. This is the override-default pattern." 3. "`@ConditionalOnProperty(name = \"feature.x.enabled\", havingValue = \"true\")` — feature flags through config. Useful for canary releases." 4. "Use `--debug` flag at startup — Spring Boot prints the conditional report showing why each auto-config was applied or skipped."

Edge cases to mention: - `@ConditionalOnClass(name = "...")` uses string for cases where the class genuinely isn't on the classpath (so you can't import it for the annotation parameter) - Custom `Condition` classes for complex multi-property logic - `@Profile` is a special case of conditional — activates only when a profile is active

What NOT to say: - "Auto-configuration is magic" — every condition is explicit and inspectable via `--debug` - "It's the same as `@Profile`" — profiles are one mechanism; conditionals are more general

Common follow-up: "How do you write your own custom conditional annotation?"

Q157 · Spring transaction propagation — REQUIRED vs REQUIRES_NEW vs NESTED

Asked at: Senior rounds at Razorpay · PhonePe · TCS Digital · Wells Fargo

Difficulty: Hard · **Topic:** Spring deep

The question: "`@Transactional(propagation = REQUIRES_NEW)` inside a `@Transactional` method — what actually happens?"

The 30-second answer: REQUIRED (default) joins an existing transaction or starts one. REQUIRES_NEW suspends any existing transaction and starts a brand-new one — even if the inner fails, the outer can still commit (and vice versa). NESTED uses a savepoint inside the current transaction — inner rollback rolls back to the savepoint without aborting the outer.

Step-by-step thought process: 1. "REQUIRED — the workhorse default. If the caller has a transaction, join it; otherwise start one. Rollback of inner rolls back the whole thing." 2. "REQUIRES_NEW — suspends the outer (saves it on the connection, gets a new connection from pool). Inner runs as a fully independent transaction. Use case: audit log that must persist even if business transaction rolls back." 3. "NESTED — single connection, savepoint-based. Inner can be rolled back without aborting the outer. Less common; only some DBs / JDBC drivers support savepoints reliably." 4. "NEVER — throws if a transaction exists. MANDATORY — throws if one doesn't. SUPPORTS — runs in the existing one if present, else non-transactional."

Edge cases to mention: - REQUIRES_NEW needs a fresh connection — under heavy load with a small connection pool, you can deadlock holding two connections per thread - Self-invocation bypasses

propagation — `this.innerMethod()` doesn't trigger the proxy - Read-only optimizations — `@Transactional(readOnly = true)` lets Hibernate skip dirty checking, JDBC drivers may use read-only connections

What NOT to say: - "REQUIRES_NEW pauses the outer" — close enough but say *suspends*; it's a specific technical term Spring uses - "Just use REQUIRED everywhere" — works most of the time but misses the audit/notify use cases

Common follow-up: "What's the connection pool risk with REQUIRES_NEW?"

Q158 · Spring profiles, active vs default vs include

Asked at: Senior rounds at TCS Digital · Cognizant · Razorpay

Difficulty: Medium · **Topic:** Spring deep

The question: "Walk me through how Spring profile resolution works at startup."

The 30-second answer: `spring.profiles.active` is the explicit list (comma-separated). If empty, Spring uses `spring.profiles.default` (which itself defaults to `default`). Profiles can include others via `spring.profiles.include`. `application-<profile>.yaml` files load only when the matching profile is active, with property override priority over `application.yaml`.

Step-by-step thought process: 1. "Set via `--spring.profiles.active=prod,asia` on CLI, `SPRING_PROFILES_ACTIVE=prod` env var, or `spring.profiles.active=prod` in `application.yaml`." 2. " `application-prod.yaml` is auto-loaded when `prod` is active. Properties there override `application.yaml`. Multiple active profiles: later-listed profiles override earlier." 3. " `@Profile("!prod")` on a bean — only registers when `prod` is NOT active. Useful for dev-only stubs." 4. "Common pattern — `prod`, `stage`, `dev`, `test` for environments; `aws`, `gcp` for cloud-specific overrides; `mysql`, `postgres` for DB swapping."

Edge cases to mention: - Profile group (`spring.profiles.group.prod=asia,observability`) — activating `prod` activates the group transitively - Profiles in Spring Boot 2.4+ use the new "config data" API — `spring.config.activate.on-profile` inside the yaml itself - Test profiles via `@ActiveProfiles("test")` on the test class

What NOT to say: - "Profiles are like environments" — close but they're more flexible (multiple active at once) - "You can change profile at runtime" — no, profile resolution happens at context startup only

Common follow-up: "How does the Spring Boot config data API differ from the older property source approach?"

Q159 · JWT vs session — when each makes sense

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Spring Security

The question: "Should I use JWT or session-based auth for my Spring Boot app?"

The 30-second answer: Session = state in your server (or Redis). JWT = state in the token itself, server stateless. Session is simpler to invalidate (delete from store) but requires sticky session or shared session store. JWT scales horizontally with zero infra but invalidation requires either short expiry + refresh tokens or a blocklist.

Step-by-step thought process: 1. "Session cookie — server stores user state, hands client an opaque session ID, validates every request. Logout = delete from session store. Easy invalidation, requires shared state across nodes." 2. "JWT — server signs a token containing user claims (`sub` , `roles` , `exp`). Validates signature on every request. Stateless on the server. Hard to revoke: token is valid until `exp` , no central record." 3. "For a single-app monolith with sticky sessions — session is simpler. For microservices where many services validate the token — JWT avoids the shared session lookup." 4. "Practical JWT pattern — short-lived access token (15 min), long-lived refresh token (stored as `HttpOnly` cookie). Revocation list for refresh tokens only."

Edge cases to mention: - Don't put PII or large data in JWT — every request carries the full payload - Storage on client — `localStorage` is XSS-exposed; `HttpOnly` Secure SameSite cookie is safer - Symmetric (HS256) vs asymmetric (RS256) signing — services that only validate need only the public key, never the signing secret

What NOT to say: - "JWT is always better because stateless" — ignores the revocation cost - "Just use OAuth" — OAuth is the framework, JWT/session is the token mechanism

Common follow-up: "How do you handle JWT revocation?"

Q160 · OAuth2 grant types — pick one for a mobile app

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Spring Security

The question: "Walk me through OAuth2 grant types. Which one for a mobile app?"

The 30-second answer: Authorization Code (with PKCE) for any public client (mobile, SPA) — exchange the code for tokens with proof-of-possession. Client Credentials for server-to-server (no user). Resource Owner Password (deprecated) and Implicit (deprecated) — don't use. Device Code for TV / CLI flows. Refresh Token for renewal.

Step-by-step thought process: 1. "Authorization Code + PKCE — modern default for any client that can't keep a secret (mobile apps, SPAs). User redirects to auth server, app exchanges short-lived code for tokens, PKCE proof binds the exchange to the original requestor." 2. "Client Credentials — service A calls

service B. No user. Service has a client ID + secret, exchanges them for an access token. Used internally between microservices." 3. "Refresh Token — separate grant to renew expiring access tokens without re-prompting the user. Should be one-time-use + rotation in modern implementations." 4. "Implicit and Password — both deprecated. Implicit because access token in URL fragment is dangerous; Password because it requires sharing real credentials with the client app."

Edge cases to mention: - PKCE (`S256` code challenge) is now recommended for *all* Auth Code flows, not just public clients - Token endpoints should reject Authorization headers without client auth when expected; many services miss this - Spring Authorization Server (replaces Spring Security OAuth which was retired) is the modern Java implementation

What NOT to say: - "Use Implicit for mobile apps" — outdated and unsafe - "OAuth and authentication are the same" — OAuth is authorization; OpenID Connect on top provides authentication

Common follow-up: "What does PKCE actually protect against?"

Q161 · Method-level security — @PreAuthorize vs @Secured

Asked at: Senior rounds at Razorpay · TCS Digital · Wells Fargo

Difficulty: Medium · **Topic:** Spring Security

The question: "Compare `@PreAuthorize`, `@Secured`, and `@RolesAllowed`."

The 30-second answer: `@Secured("ROLE_ADMIN")` is the simplest — checks roles by name. `@PreAuthorize("hasRole('ADMIN') and #order.userId == authentication.principal.id")` evaluates SpEL expressions, including method-argument references. `@RolesAllowed` is the JSR-250 standard equivalent of `@Secured`. Enable with `@EnableMethodSecurity` (Spring Security 6+).

Step-by-step thought process: 1. "`@EnableMethodSecurity(prePostEnabled = true)` — turns on the SpEL-based annotations. Use it instead of older `@EnableGlobalMethodSecurity`." 2. "`@PreAuthorize` for everything new — SpEL gives you access to method args (`#orderId`), the authenticated principal, and Spring Security helpers (`hasRole`, `hasAuthority`, `isAuthenticated`)." 3. "`@PostAuthorize("returnObject.userId == authentication.principal.id")` — runs after method returns; you can reject access based on what was returned." 4. "`@PreFilter` / `@PostFilter` — filter elements of a collection in/out based on per-element SpEL — niche but powerful."

Edge cases to mention: - Spring Security uses AOP — same proxy / self-invocation gotcha as `@Transactional` - `@PreAuthorize` on interfaces vs implementations — Spring scans both, prefer the interface for cleanliness - Custom `PermissionEvaluator` for object-level permissions (`hasPermission(#order, 'WRITE')`)

What NOT to say: - "@Secured is deprecated" — still supported, just older; @PreAuthorize is preferred for new code - "Security annotations are slower than filter chains" — both go through filter chain; method-level adds an extra AOP layer

Common follow-up: "How do you write a custom PermissionEvaluator?"

Q162 · CSRF — what it is and when Spring Security needs it

Asked at: Senior rounds at Razorpay · TCS Digital · Wipro · Cognizant

Difficulty: Medium · **Topic:** Spring Security

The question: "What is CSRF and when can I disable Spring's CSRF protection?"

The 30-second answer: CSRF (Cross-Site Request Forgery) — a malicious site tricks your authenticated browser into submitting a request to a different site (yours). Spring Security defaults to CSRF protection via a synchronizer token. Safe to disable only for stateless APIs that don't use cookies for auth — pure JWT-bearer APIs.

Step-by-step thought process: 1. "The attack — user logs into bank.com, gets a session cookie. Then visits attacker.com, which has a hidden form submitting to bank.com/transfer. Browser automatically attaches the cookie. Attack succeeds." 2. "Defense — CSRF token. Server generates a random token per session, embeds in forms / sends as header. Server verifies the token on state-changing requests. Attacker can't read the token (same-origin policy)." 3. "Spring Security — CSRF on by default. For JWT-bearer APIs (no cookies), CSRF isn't possible — attacker can't fabricate the Authorization header. Disable with `http.csrf(csrf -> csrf.disable())` ." 4. "If you use cookies for auth and don't want a token, set the cookie to `SameSite=Lax` or `Strict` — modern browsers block cross-site requests carrying the cookie."

Edge cases to mention: - `CookieCsrfTokenRepository.withHttpOnlyFalse()` — token in a JS-readable cookie, app reads and adds as header. Pattern for SPAs - GET requests are exempt from CSRF (they should be idempotent) — but if your GET endpoint mutates state, that's the real bug - Double-submit cookie pattern is an alternative to synchronizer tokens

What NOT to say: - "Always disable CSRF for APIs" — only safe for non-cookie auth - "CSRF and XSS are the same" — completely different attacks

Common follow-up: "What's SameSite cookie attribute and how does it relate to CSRF?"

Q163 · Password encoders — BCrypt vs Argon2 vs Scrypt

Asked at: Senior rounds at Razorpay · Cred · PhonePe · Wells Fargo

Difficulty: Medium · **Topic:** Spring Security

The question: "What password encoder would you pick in 2026 and why?"

The 30-second answer: Argon2id is the modern OWASP recommendation — memory-hard, GPU-resistant. BCrypt is the safe traditional default — well-understood, widely supported, configurable cost. Avoid plain SHA / MD5 / unsalted hashes. Spring Security's `DelegatingPasswordEncoder` lets you store the algorithm with the hash for graceful migration.

Step-by-step thought process: 1. "Password storage must be slow on purpose — to make offline brute-force attacks expensive. BCrypt and friends use a configurable work factor; tune to take ~250-500ms per hash on your hardware." 2. "BCrypt cost (default 10) — `2^cost` rounds. Raise to 12+ for 2026 hardware. `BCryptPasswordEncoder(12)` ." 3. "Argon2id — winner of the Password Hashing Competition. Tunable memory cost (resists GPU/ASIC attacks better than BCrypt). Spring Security: `Argon2PasswordEncoder.defaultsForSpringSecurity_v5_8()` ." 4. "`DelegatingPasswordEncoder` — Spring Security default. Stores `{bcrypt}$2a$10$...` — the prefix tells which encoder to use. Lets you change algorithms without invalidating existing passwords."

Edge cases to mention: - Never reuse a single salt — each password gets its own random salt, stored alongside the hash - Peppering (extra secret added to all hashes) is a separate technique — store the pepper in a KMS / secret manager - Migration strategy — on user login, re-hash with the new algorithm if the stored prefix is outdated

What NOT to say: - "SHA-256 with salt is fine" — too fast; GPU farms crack billions per second - "Just use plain text behind HTTPS" — never

Common follow-up: "How does `DelegatingPasswordEncoder` handle migration when you change the encoder?"

Q164 · CORS configuration in Spring Security

Asked at: Senior rounds at Razorpay · Swiggy · TCS Digital

Difficulty: Medium · **Topic:** Spring Security

The question: "My React frontend at `https://app.example.com` can't call my API at `https://api.example.com` . What's CORS and how do I fix it in Spring?"

The 30-second answer: Browsers block cross-origin requests by default. The server has to explicitly allow them by returning `Access-Control-Allow-Origin` and friends. In Spring Security, configure a `CorsConfigurationSource` bean and reference it from `http.cors(...)` . Don't use `*` for production — list exact origins.

Step-by-step thought process: 1. "Same-origin policy — JavaScript at origin A can read responses from origin A only. Different protocol/host/port = different origin. CORS is the explicit opt-in." 2. "Preflight request — for anything beyond simple GET/POST with safe headers, the browser sends an OPTIONS request first. Server responds with allowed origins/methods/headers. Then the real request goes." 3. "Spring config — `@Bean CorsConfigurationSource corsConfigurationSource()` `{ ... }` . Set allowed origins, methods, headers, `allowCredentials = true` if you need

cookies." 4. "With Spring Security, `.cors(Customizer.withDefaults())` enables CORS handling in the filter chain. Without it, security can block before CORS headers are added."

Edge cases to mention: - `allowedOrigins = "*" + allowCredentials = true` is invalid per spec — you must list explicit origins - `setAllowedOriginPatterns` (Spring 5.3+) lets you use wildcards like `https://*.example.com` even with credentials - Preflight responses are cached — `setMaxAge(3600)` reduces preflight overhead

What NOT to say: - "Disable CORS to fix it" — CORS is a browser-enforced policy; you can't disable the browser - "Use a wildcard in production" — opens you to arbitrary-origin attacks

Common follow-up: "What's the difference between a simple request and a preflight request?"

Q165 · Second-level cache in Hibernate

Asked at: Senior rounds at Razorpay · Cred · Swiggy · PhonePe

Difficulty: Medium-Hard · **Topic:** Hibernate advanced

The question: "What's the second-level cache in Hibernate? When does it help?"

The 30-second answer: First-level cache = Session / EntityManager scope (per-transaction). Second-level cache = SessionFactory scope, shared across sessions. Stores entities by ID across the entire app. Helps for read-mostly entities (`Country` , `MerchantConfig`). Hibernate ships pluggable providers — Ehcache, Hazelcast, Infinispan, Caffeine.

Step-by-step thought process: 1. "Enable globally — `hibernate.cache.use_second_level_cache=true` , set provider with `hibernate.cache.region.factory_class` . Annotate cacheable entities with `@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)` ." 2. "L2 stores hydrated state (field values), not object instances — so each session that loads an entity from L2 gets its own instance. This is correct for the JPA contract." 3. " `@Cache` concurrency strategies — `READ_ONLY` (no updates, fastest), `NONSTRICT_READ_WRITE` (no locking, can stale briefly), `READ_WRITE` (soft lock during commit), `TRANSACTIONAL` (full XA — rarely worth it)." 4. "L2 only helps for `EntityManager.find(id)` style lookups. JPQL queries don't hit L2 by default — they hit the query cache, which is separate."

Edge cases to mention: - Stale data — if another app or raw SQL updates the table, L2 doesn't know. Bypass with `cache.evict(...)` or design L2 only for truly immutable entities - Distributed L2 (Infinispan, Hazelcast) — invalidation messages across nodes. Adds network cost; benchmark before assuming benefit - Collection caching — `@Cache` on the entity is separate from `@Cache` on a `@OneToMany` collection; both must be annotated

What NOT to say: - "L2 makes Hibernate fast" — only for specific patterns; bad fit for write-heavy entities - "L2 is global cache" — it's a per-SessionFactory cache; multiple apps each have their own

Common follow-up: "What's the query cache and why is it usually a bad idea?"

Q166 · N+1 problem — find it and fix it

Asked at: Senior rounds at Razorpay · Swiggy · Cred · TCS Digital · Wells Fargo

Difficulty: Medium-Hard · **Topic:** Hibernate advanced

The question: "What's the N+1 problem and how do you diagnose it?"

The 30-second answer: You execute one query to fetch N parent entities (`SELECT * FROM order`), then your code accesses a lazy collection on each, causing N more queries (`SELECT * FROM order_item WHERE order_id = ?`). Total: N+1 queries. Diagnose: enable SQL logging or add a query counter assertion. Fix: `JOIN FETCH` , `@EntityGraph` , batch fetching.

Step-by-step thought process: 1. "Symptom — a page response that should be one query ends up being 100. SQL log shows the same query template repeating." 2. " `JOIN FETCH` in JPQL — `SELECT o FROM Order o JOIN FETCH o.items WHERE o.status = :status` . Single query with a join. Watch out for Cartesian products with multiple bag fetches." 3. " `@EntityGraph` — declarative version. `@EntityGraph(attributePaths = {"items", "customer"})` on the repository method. Hibernate generates the joins automatically." 4. "Batch fetching — `@BatchSize(size = 25)` on the collection. Instead of N queries, Hibernate batches them: `WHERE order_id IN (?, ?, ?, ...)` . Constant 1 extra query regardless of N. Works for many one-to-many relationships at once."

Edge cases to mention: - `JOIN FETCH` with pagination is a footgun — Hibernate fetches everything then paginates in memory. Use `EntityGraph` + native pagination instead - Multiple bag fetches in one query — `MultipleBagFetchException` . Either use `Set` instead of `List` or fetch separately - Hibernate stats — `hibernate.generate_statistics=true` exposes query count via JMX; assert on it in integration tests

What NOT to say: - "Use eager loading" — fixes N+1 but creates over-fetching; lazy + batch is usually better - "Hibernate is slow" — N+1 is a usage issue, not a Hibernate flaw

Common follow-up: "Show me a JPQL JOIN FETCH that would fail and why."

Q167 · EntityGraph vs JOIN FETCH vs @BatchSize

Asked at: Senior rounds at Razorpay · Swiggy · Cred · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Hibernate advanced

The question: "You have three tools to fix N+1. When do you reach for which?"

The 30-second answer: `JOIN FETCH` for one-off ad-hoc queries — quick fix. `@EntityGraph` for repository-level reusable graphs — cleaner long-term. `@BatchSize` as a global default for relationships where you usually want them loaded — set-and-forget.

Step-by-step thought process: 1. "JOIN FETCH — JPQL keyword. Inline in your query. Fast to add, easy to misuse with pagination. Per-query." 2. "@EntityGraph — declares which subgraph to load. Attach to a repository method via `@EntityGraph(attributePaths = ...)`. Generates the joins. Reusable across multiple queries." 3. "@BatchSize — on a collection (`@OneToMany @BatchSize(size = 25)`) or entity. When Hibernate is about to lazy-load N proxies, it batches them. Doesn't avoid the problem entirely but reduces $N+1$ to $N/25 + 1$." 4. "Pattern — make collections lazy + BatchSize globally. Use EntityGraph for known-fat operations (admin views, exports). JOIN FETCH for one-off endpoints."

Edge cases to mention: - EntityGraphs come in two flavours — `LOAD` (graph attributes + entity defaults) and `FETCH` (graph attributes only, everything else lazy) - `@BatchSize` on the entity class affects how lazy proxies of that entity are batched — different from collection BatchSize - Open-Session-In-View pattern hides $N+1$ — endpoints work but performance is poor; either disable OSIV or strictly use DTOs

What NOT to say: - "Just eager-load everything" — kills performance and creates Cartesian explosions - "Hibernate auto-detects $N+1$ " — no, you have to spot it

Common follow-up: "What's `MultipleBagFetchException` and how do you fix it?"

Q168 · JPA Criteria API — type-safe dynamic queries

Asked at: Senior rounds at JP Morgan India · Wells Fargo · Goldman · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Hibernate advanced

The question: "You need a search endpoint with 8 optional filters. JPQL is painful. What's the alternative?"

The 30-second answer: JPA Criteria API — build queries programmatically as a tree of `Predicate` objects. Type-safe via the generated metamodel (`Order_.merchantId`). Build a list of predicates conditionally based on which filters are set, combine with `cb.and(...)`. Spring Data JPA `Specification` is a thin wrapper that's more ergonomic.

Step-by-step thought process: 1. "`CriteriaBuilder cb = entityManager.getCriteriaBuilder(); CriteriaQuery<Order> q = cb.createQuery(Order.class); Root<Order> root = q.from(Order.class);` — boilerplate to get started." 2. "Build predicates conditionally — `List<Predicate> ps = new ArrayList<>(); if (status != null) ps.add(cb.equal(root.get('status'), status));`. Combine: `q.where(cb.and(ps.toArray(new Predicate[0])));`" 3. "Type-safe access via metamodel — `root.get(Order_.merchantId)` instead of string `"merchantId"`. Generated at compile time by `hibernate-jpamodelgen`." 4. "Spring Data Specification — `Specification<Order> hasStatus(Status s) = (root, q, cb) -> cb.equal(root.get(Order_.status), s);`. Compose with `.and()` / `.or()` / `Specification.where(...)`."

Edge cases to mention: - Criteria queries are verbose — for static queries with no conditionals, JPQL is far cleaner - jOOQ and QueryDSL are popular alternatives — more concise DSLs, but extra build tooling - Pagination with Criteria — `q.setFirstResult` / `setMaxResults` on the typed query, separate from the criteria builder

What NOT to say: - "Use string concatenation to build JPQL dynamically" — SQL injection risk, fragile - "Always use Criteria" — over-engineering for static queries

Common follow-up: "What's QueryDSL and how does it compare to Criteria?"

Q169 · Hibernate batch fetching, batch inserts, batch updates

Asked at: Senior rounds at Razorpay · Cred · Wells Fargo · PhonePe

Difficulty: Medium-Hard · **Topic:** Hibernate advanced

The question: "I'm inserting 10,000 entities and Hibernate is slow. What do I tune?"

The 30-second answer: Set `hibernate.jdbc.batch_size=50` to enable JDBC batching, `hibernate.order_inserts=true` and `hibernate.order_updates=true` so Hibernate groups same-type statements together. Flush + clear the session every batch to keep memory bounded. Use `IDENTITY` generators carefully — they disable batching.

Step-by-step thought process: 1. "By default Hibernate sends each INSERT as a separate JDBC round-trip. With `batch_size=50`, it accumulates 50 then sends them as one batch — orders of magnitude faster network-wise." 2. "Order inserts/updates — without ordering, an `INSERT order, INSERT customer, INSERT order` sequence breaks the batch (different SQL statements). Ordering groups same-table operations together." 3. "Memory — Session keeps every persisted entity in the first-level cache. After every 50 entities, call `em.flush(); em.clear();` to evict and avoid OOM." 4. "`@GeneratedValue(strategy = IDENTITY)` — the DB assigns the ID, but Hibernate needs the ID before batching can complete. Switch to `SEQUENCE` (PostgreSQL/Oracle) or pre-allocate via `hi/lo` optimizer."

Edge cases to mention: - `StatelessSession` — bypasses first-level cache entirely. Best for bulk ETL where you don't need entity lifecycle - For pure SQL-fast bulk, drop Hibernate and use `JdbcTemplate.batchUpdate(...)` directly - MySQL has `rewriteBatchedStatements=true` JDBC URL param — actual batching at the driver level; without it the driver fakes batching

What NOT to say: - "Use `saveAll(...)` and it'll batch" — only if you've also set `batch_size` and ID generation supports it - "Hibernate is too slow for bulk" — depends on tuning; well-tuned Hibernate is close to raw JDBC

Common follow-up: "Why does `@GeneratedValue(IDENTITY)` disable batching?"

Q170 · Custom Hibernate dialects

Asked at: Senior rounds at Wells Fargo · Goldman · TCS Digital

Difficulty: Medium-Hard · **Topic:** Hibernate advanced

The question: "When would you write a custom Hibernate dialect?"

The 30-second answer: Dialects translate JPQL/HQL to vendor-specific SQL — pagination syntax, sequence syntax, function names. Write a custom dialect to register vendor-specific SQL functions (Postgres `array_agg`, MySQL `JSON_EXTRACT`), override pagination, or change identifier quoting.

Step-by-step thought process: 1. "Out of the box — `PostgreSQLDialect`, `MySQLDialect`, `OracleDialect`, etc. Extending one and overriding a couple of methods is the normal path." 2. "Custom function — `registerFunction("jsonb_path", new StandardSQLFunction("jsonb_path_query", StandardBasicTypes.STRING))`. Now JPQL can call `jsonb_path(...)` and Hibernate emits the right SQL." 3. "Identifier quoting — override `quote(...)` if your DB has unusual escape rules. Rare for mainstream DBs." 4. "Hibernate 6 changes — dialects modernized, version-specific dialects auto-selected, much less manual config than Hibernate 5 needed."

Edge cases to mention: - Hibernate 6 dropped many legacy dialects (`MySQL5Dialect`, `MySQL57Dialect` — now just `MySQLDialect` with version detection) - For Postgres JSONB, hibernate-types library has community-maintained user types worth using - Don't write a dialect to fix a JPA / Hibernate version mismatch — fix the version

What NOT to say: - "Just use native SQL everywhere" — fine but loses JPQL portability and Hibernate's L1 cache integration - "Dialects are deprecated" — no, just consolidated

Common follow-up: "How do you map a Postgres JSONB column to a Java object?"

Q171 · Circuit breaker — what does it actually do?

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Cred · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Microservices / distributed

The question: "Explain the circuit breaker pattern. Walk me through the state transitions."

The 30-second answer: Three states — CLOSED (passes calls through, counts failures), OPEN (fails fast without calling — usually after failure threshold hits), HALF_OPEN (lets a few trial calls through after a cooldown; if they succeed, back to CLOSED). Protects you from cascading failures when a downstream is dead — your threads aren't blocked waiting for a doomed call.

Step-by-step thought process: 1. "CLOSED — normal mode. Calls pass through. We count failures over a window (count-based or time-based)." 2. "OPEN — failure threshold exceeded (e.g., 50% failure rate over the last 100 calls). Now we fail fast for every call without touching the downstream. Saves us from blowing up thread pools waiting on a dead service." 3. "HALF_OPEN — after a configured wait

(`waitDurationInOpenState` , e.g., 60s), allow N trial calls. If they mostly succeed, transition back to CLOSED. If they fail, back to OPEN." 4. "Resilience4j is the modern Java library — Hystrix is dead (retired by Netflix). Spring Cloud Circuit Breaker has Resilience4j as the default."

Edge cases to mention: - Tune thresholds carefully — too sensitive flaps the breaker open on transient noise; too loose lets bad services drain your threads - Use `slidingWindowType = COUNT_BASED` for predictable evaluation; `TIME_BASED` for variable call rates - Combine with retry, bulkhead, timeout — Resilience4j composes them via decorators

What NOT to say: - "Use Hystrix" — retired in 2018; new code goes to Resilience4j - "Circuit breaker replaces timeouts" — they're complementary; breaker reacts to repeated timeouts

Common follow-up: "What's a bulkhead and how is it different from a circuit breaker?"

Q172 · Distributed tracing — span, trace, context propagation

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Cred · Flipkart

Difficulty: Medium-Hard · **Topic:** Microservices / distributed

The question: "How does distributed tracing work? Explain trace, span, and context propagation."

The 30-second answer: A *trace* is the full journey of a request across services. Each unit of work is a *span* (e.g., one HTTP call, one DB query). Spans have parent/child relationships. Each carries a `traceId` (shared across the trace) and `spanId` (unique). Context propagation = passing the IDs via HTTP headers (`traceparent` , `tracestate` — W3C Trace Context standard) so downstream services join the trace.

Step-by-step thought process: 1. "Client calls service A → A creates root span with new `traceId`. A calls B → A passes `traceId` + its `spanId` as B's parent via `traceparent` header." 2. "Each service emits spans to a collector (Jaeger, Zipkin, Tempo). Collector reconstructs the tree by joining `traceId` + `parentSpanId`." 3. "Spring Boot 3 uses Micrometer Tracing as the abstraction — defaults to Brave (Zipkin) or OpenTelemetry as the backend. Replaces Spring Cloud Sleuth from Boot 2.x." 4. "Instrumentation — `RestTemplate`, `WebClient`, `Kafka`, `JDBC` all have auto-instrumentation; spans are created automatically. Custom spans via `Tracer.nextSpan().start()` for business logic boundaries."

Edge cases to mention: - Sampling — at high volume you can't store every trace. Head-based sampling (decide at root, propagate) vs tail-based (decide after seeing the full trace, smarter but harder) - Log correlation — inject `traceId` into MDC so each log line has it; combine with central logging for fast incident response - Baggage — propagate arbitrary key/value alongside trace IDs (e.g., `userId` , `tenantId`) for filtered traces

What NOT to say: - "Just log everything" — logs don't reconstruct the call tree; traces do - "Tracing is for debugging only" — also for SLO monitoring, dependency mapping, error attribution

Common follow-up: "What's tail-based vs head-based sampling?"

Q173 · Saga pattern — choreography vs orchestration

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Flipkart

Difficulty: Hard · **Topic:** Microservices / distributed

The question: "Explain the saga pattern. What's choreography vs orchestration?"

The 30-second answer: Saga = long-running multi-service transaction implemented as a sequence of local transactions, each with a compensating action. *Choreography* — services react to events from each other, no central coordinator. *Orchestration* — a central saga orchestrator (e.g., Camunda, Temporal) drives the steps explicitly.

Step-by-step thought process: 1. "Distributed transactions (2PC) don't scale — locking across services is a SPOF. Saga replaces ACID across services with eventually-consistent business workflow." 2. "Order saga — `OrderCreated` → `PaymentReserved` → `InventoryReserved` → `OrderConfirmed`. Each step is a local transaction. If payment fails, the saga compensates: cancel reservation, mark order failed." 3. "Choreography — Payment service listens to `OrderCreated`, charges, emits `PaymentReserved`. Inventory listens to `PaymentReserved`. No coordinator. Simple at small scale, hard to reason about at large scale." 4. "Orchestration — A saga orchestrator (Temporal, Camunda, Spring State Machine) calls each service in turn and tracks state. Easier to visualize and debug, but introduces a central service to operate."

Edge cases to mention: - Compensating actions are not always perfect inverses — refund != un-charge if the order was partially shipped. Sagas need explicit business semantics for compensation - Idempotency is essential — every saga step might be re-delivered (at-least-once messaging); steps must tolerate duplicates - Saga timeout — what happens if a step never completes? Orchestrator usually has a per-step timeout that triggers compensation

What NOT to say: - "Use 2PC instead" — doesn't scale across heterogeneous services - "Saga is just retry logic" — undersells; compensation is the key insight

Common follow-up: "Why is idempotency so important in saga steps?"

Q174 · Idempotency keys — design and storage

Asked at: Senior rounds at Razorpay · PhonePe · Swiggy · Cred

Difficulty: Medium-Hard · **Topic:** Microservices / distributed

The question: "How do you implement idempotency for a payment API?"

The 30-second answer: Client sends an `Idempotency-Key` header (UUID) on every payment request. Server stores `(key → response)` mapping. On duplicate request with the same key, return the stored response instead of processing again. Storage in Redis with TTL (24 hours typical). Race condition: two simultaneous requests with same key — use atomic SETNX or a state machine.

Step-by-step thought process: 1. "Stripe popularized this — client generates a UUID per logical request. Header `Idempotency-Key: <uuid>`. Server looks it up before processing." 2. "Storage — Redis is typical. Key = idempotency key. Value = serialized response + status. TTL 24h to bound storage." 3. "Race — two retries hit the server within ms. Both look up the key (not found), both start processing. Use `SET NX` to atomically claim the key with status `PROCESSING`. Loser waits or returns `409 Conflict`." 4. "On completion, write the final response into the key. Next duplicate retry returns the cached response."

Edge cases to mention: - Validate request payload matches stored payload — same key with different body should reject (`422 Unprocessable Entity`) - For very long operations, return `202 Accepted` immediately and poll status by the key - Cleanup — TTL handles expiration; for forever-idempotent operations (refunds), store in a real DB instead

What NOT to say: - "Use the request hash as the key" — clients can't reliably reproduce the hash on retry across SDK versions - "DB unique constraint is enough" — only after the row is created; pre-insert race still exists

Common follow-up: "How would you design the database schema to enforce idempotency on a refund?"

Q175 · Eventual consistency — how to communicate to product

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe

Difficulty: Medium-Hard · **Topic:** Microservices / distributed

The question: "Product wants 'real-time consistency' across all services. How do you handle that conversation?"

The 30-second answer: In a distributed system, strong consistency requires synchronous coordination — every service waits for every other. That kills latency and availability. Eventual consistency is the price of horizontal scale. The conversation is about acceptable convergence window (50ms? 500ms? 5s?) and what UX patterns mask the gap (optimistic UI, polling, retries, read-after-write hints).

Step-by-step thought process: 1. "Distributed transactions across services either lock everything (no throughput) or fail open (data inconsistency). Eventual consistency is the practical choice." 2. "User pays → payment service updates → emits event → order service updates → emits event → notification service sends SMS. Total convergence: 100-500ms typically." 3. "Mask the gap — show optimistic UI (assume success, roll back on failure), use 'read your own writes' guarantees by routing the user's reads back to the writer node, poll for completion." 4. "Read-after-write — within the same session, route reads to the primary or to a session-pinned replica. Beyond that session, eventual consistency is fine."

Edge cases to mention: - Some operations are inherently strongly consistent (deducting balance, reserving inventory) — these stay synchronous or use sagas with compensation - CRDTs (conflict-free replicated data types) for distributed counters / sets where order doesn't matter - Discuss SLOs in terms of convergence percentiles — p99 < 1s — not "instantaneous"

What NOT to say: - "Stronger consistency is always better" — performance and availability suffer - "Just use Kafka and it'll be fine" — Kafka is at-least-once, not magic consistency

Common follow-up: "What's the CAP theorem and how does it apply here?"

Q176 · Retry strategies — exponential backoff, jitter, retry budget

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Cred · Walmart Labs

Difficulty: Medium · **Topic:** Microservices / distributed

The question: "You're retrying a flaky downstream. Walk me through your retry strategy."

The 30-second answer: Exponential backoff (100ms, 200ms, 400ms, 800ms) with jitter (randomize $\pm 20\%$) to avoid synchronized retries thundering the downstream. Cap retries (3-5 typical). Differentiate retryable errors (5xx, timeout, connection reset) from non-retryable (4xx). Wrap in a circuit breaker so a sustained outage doesn't cause retries forever.

Step-by-step thought process: 1. "Naive retry — fixed delay, fixed count. Causes thundering herd: every client retries at the same moment, downstream gets a synchronized spike." 2. "Exponential backoff — double the delay each time. Spreads retries out. With jitter (random $\pm 20\%$), clients diverge instead of synchronizing." 3. "Retry only on retryable errors — 5xx, timeout, connection reset. Don't retry 400, 401, 404 — they're permanent (idempotent retries only)." 4. "Retry budget — limit aggregate retries (e.g., max 10% of total calls). Without it, retries during an outage amplify load by 5x and hurt recovery."

Edge cases to mention: - Idempotency — retrying a POST that creates a resource without an idempotency key creates duplicates - Retry-After header — server can tell client when to retry; respect it - Resilience4j and Spring Retry both implement the pattern — Resilience4j composes better with circuit breaker

What NOT to say: - "Retry until it works" — guarantees you'll DDoS your downstream during their outage - "Retry every error" — 400 means "your request is broken", retrying won't fix it

Common follow-up: "What's a retry storm and how do retry budgets prevent it?"

Q177 · Reflection — performance cost and how to mitigate

Asked at: Senior rounds at Razorpay · Swiggy · Flipkart

Difficulty: Medium-Hard · **Topic:** Reflection / Annotations / Bytecode

The question: "How expensive is reflection? When does it actually matter?"

The 30-second answer: Field access via reflection is 10-100x slower than direct access; method invocation similar. Matters in hot loops, not in framework startup. Mitigate: cache `Method` / `Field`

objects (don't `getMethod` per call), use `MethodHandle` (faster than `Method.invoke`), or generate bytecode at startup (ASM/ByteBuddy) and invoke directly.

Step-by-step thought process: 1. "`Class.getMethod / getField` does linear search + security check. Costly. Call once at init, cache the `Method` object." 2. "Even with cached `Method`, `invoke()` boxes primitives, packs args into `Object[]`, has security checks. 10x slower than direct call." 3. "`MethodHandle` (Java 7+) — JIT-friendly, comparable to direct call after warmup. The modern fast-reflection answer." 4. "Bytecode generation — frameworks like Jackson and Spring generate per-class accessor classes (ASM / ByteBuddy) at startup, then invoke them with no reflection cost. Highest setup cost, lowest steady-state cost."

Edge cases to mention: - `setAccessible(true)` for private fields — Java 9+ module system restricts this; add `--add-opens` to JVM args or use `Lookup.privateLookupIn` (Java 9+) - Reflection in native image (GraalVM) requires explicit reflection-config hints - For one-off CLI tools, reflection is fine — no warmup, no high QPS

What NOT to say: - "Always avoid reflection" — frameworks rely on it; it's a steady-state hot-path question, not blanket - "Reflection caches itself" — partial; JVM caches some lookups but you can't rely on it

Common follow-up: "How does `MethodHandle` differ from `Method.invoke`?"

Q178 · Retention policies — SOURCE, CLASS, RUNTIME

Asked at: Senior rounds at Razorpay · TCS Digital · Cognizant

Difficulty: Medium · **Topic:** Reflection / Annotations / Bytecode

The question: "`@Retention(RetentionPolicy.RUNTIME)` — what's the alternative?"

The 30-second answer: SOURCE — annotation discarded by the compiler, never in `.class` files (Lombok's `@Getter`, `@Override`). CLASS (default) — in the `.class` file but not loaded by the JVM, so unreadable at runtime (used by static-analysis tools). RUNTIME — loaded into Class metadata, reflectable (Spring's `@Component`, JPA's `@Entity`).

Step-by-step thought process: 1. "SOURCE — annotation processor sees it (during compilation), then it disappears. Lombok generates getters from `@Getter` at compile time, then strips the annotation." 2. "CLASS — annotation lives in bytecode but JVM doesn't load it into reflection-accessible metadata. Useful for tools like SpotBugs, Checker Framework that parse class files directly." 3. "RUNTIME — JVM loads it, accessible via `clazz.getAnnotation(...)`. Required for runtime frameworks (Spring, JPA, JAX-RS)." 4. "Default if no `@Retention` specified is CLASS, which is almost never what you want — be explicit."

Edge cases to mention: - Memory cost — RUNTIME annotations live in Metaspace; thousands of annotations per class adds up - Annotation processors

(`javax.annotation.processing.Processor`) run at compile time regardless of retention — Lombok uses this hook - `@Inherited` works only on class-level annotations and only at one level up

What NOT to say: - "RUNTIME is always best" — wastes memory if you only need compile-time - "Annotations execute code" — no, annotations are metadata; processors/frameworks read them

Common follow-up: *"Write me a quick annotation processor that fails compilation if a class is missing `@Override`."*

Q179 · ASM, Javassist, ByteBuddy — when to reach for which

Asked at: Staff rounds at Razorpay · Flipkart · Cred · Walmart Labs

Difficulty: Hard · **Topic:** Reflection / Annotations / Bytecode

The question: *"Three Java bytecode libraries. When do you use each?"*

The 30-second answer: ASM — low-level, byte-by-byte control, maximum performance, painful API. Javassist — high-level, accepts Java source strings, slower at runtime, less popular today. ByteBuddy — modern fluent API, type-safe builders, used internally by Mockito and many recent frameworks. Default choice for new code: ByteBuddy.

Step-by-step thought process: 1. "ASM — the foundation. Used by every major bytecode library under the hood. Direct visitor-based API, you write opcode-by-opcode. Hard to use, fast." 2. "Javassist — older alternative. Lets you write Java source code as a String, compiles it to bytecode internally. Easier for simple modifications, slower at scale." 3. "ByteBuddy (Rafael Winterhalter, ~2014) — modern. Fluent builder pattern, AspectJ-style matchers, type-safe. Mockito uses it to subclass mock classes; many AOP libraries adopted it." 4. "Pick: ByteBuddy for new code (90% of cases). ASM only when you need raw control and performance. Javassist if you're maintaining legacy."

Edge cases to mention: - Java module system (Java 9+) restricts bytecode manipulation across module boundaries — ByteBuddy / ASM need `--add-opens` for some operations - Generated classes inflate Metaspace — cache and reuse them, don't generate per-call - GraalVM native image can't see runtime-generated classes — use AOT processing instead

What NOT to say: - "Use the JDK Proxy API" — only handles interfaces, no class extension - "Bytecode manipulation is hacky" — it powers Mockito, Spring proxies, Hibernate, distributed tracing agents

Common follow-up: *"How does Mockito create mocks for non-final classes?"*

Q180 · Annotation processors at compile time

Asked at: Senior rounds at Razorpay · Cred · Walmart Labs

Difficulty: Hard · **Topic:** Reflection / Annotations / Bytecode

The question: "Lombok works at compile time. How?"

The 30-second answer: Lombok implements a `javax.annotation.processing.Processor`. The Java compiler discovers it via `META-INF/services/javax.annotation.processing.Processor`. The processor inspects annotated source elements, generates new code (Lombok modifies the AST directly — a controversial design), and the compiler emits classes with the generated code baked in.

Step-by-step thought process: 1. "Standard pattern — implement `AbstractProcessor`, declare which annotations you handle (`@SupportedAnnotationTypes`), implement `process(...)` to inspect the AST." 2. "Most processors generate new source files via `Filer` — MapStruct, Dagger, AutoValue. Compiler sees them and compiles them in the same round." 3. "Lombok cheats — it modifies the existing AST in place using internal compiler APIs. Powerful, but breaks across compiler versions." 4. "Build integration — processors are on the annotationProcessor classpath. Maven: `<annotationProcessorPaths>` in compiler plugin. Gradle: `annotationProcessor` configuration."

Edge cases to mention: - IDE integration — IntelliJ has built-in support for many processors; for unknown ones, enable annotation processing in settings - Processors can produce errors via `Messenger` — fail the build with descriptive messages - JEP 8242972 — Java 23+ improvements to processor APIs for streaming compilation models

What NOT to say: - "Lombok is reflection-based" — opposite; it's compile-time, no runtime cost - "Annotation processors run at runtime" — no, compile time only

Common follow-up: "What's the difference between Lombok and MapStruct philosophically?"

Q181 · Strategy pattern — when to actually use it in Java

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · TCS Digital

Difficulty: Medium · **Topic:** Design patterns

The question: "Walk me through a real Strategy pattern use in Java code — not the textbook example."

The 30-second answer: Strategy = encapsulate interchangeable algorithms behind a common interface. Real Java use — payment gateway selection at runtime. `PaymentGateway` interface, `RazorpayGateway`, `StripeGateway`, `PayUGateway` implementations, a registry (`Map<String, PaymentGateway>` autowired by Spring) picks one by name. Avoids massive `switch` over gateway type.

Step-by-function thought process: 1. "Define the contract — `interface PaymentGateway { PaymentResult charge(Order order); }`" 2. "Implement per provider — `@Component("razorpay") class RazorpayGateway implements PaymentGateway`, same for stripe, payu." 3. "Spring autowires `Map<String, PaymentGateway>` — keys are bean names. Caller does `gateways.get(order.getGatewayCode()).charge(order)`. No switch." 4.

"Adding a new gateway = adding one class. No edits to the dispatcher. That's the Strategy win — open/closed principle."

Edge cases to mention: - For two strategies, the pattern is overkill — just use an `if`. Reach for it when you have 3+ - Java 8+ — for stateless strategies, a `Function<X, Y>` lambda often suffices over a full interface - Don't conflate Strategy with State — Strategy is set externally; State changes itself based on transitions

What NOT to say: - "Use enums with abstract methods" — works for closed sets but breaks open/closed for unknown providers - "Inheritance is simpler" — couples behavior; the point of Strategy is composition

Common follow-up: "How would Spring help you implement this without a giant switch?"

Q182 · Observer pattern in Spring — ApplicationEventPublisher

Asked at: Senior rounds at Razorpay · Swiggy · TCS Digital

Difficulty: Medium · **Topic:** Design patterns

The question: "Show me Observer pattern in real Spring code."

The 30-second answer: Spring's `ApplicationEventPublisher` and `@EventListener` are Observer pattern. Publisher emits events (`publishEvent(new OrderPlacedEvent(...))`), listeners subscribe via annotation. Listener can be `@Async` for off-thread processing. Decouples the publisher from listeners — publisher doesn't know who's listening.

Step-by-step thought process: 1. "Inject `ApplicationEventPublisher` and emit — `publisher.publishEvent(new OrderPlacedEvent(orderId))`. No reference to listeners anywhere." 2. "Listener — `@Component class NotificationListener { @EventListener public void onOrderPlaced(OrderPlacedEvent e) { ... } }`. Spring finds the matching method by parameter type." 3. "Default events are synchronous — they run in the publisher's thread inside the publisher's transaction. Use `@Async @EventListener` for fire-and-forget." 4. "`@TransactionalEventListener(phase = AFTER_COMMIT)` — listener fires only after the publishing transaction commits. Avoids the trap of acting on data that may roll back."

Edge cases to mention: - Synchronous events propagate exceptions back to the publisher — failures abort the transaction - Multiple listeners for one event — order via `@Order(1)`, parallel execution if all `@Async` - Event hierarchy — listener on a parent class catches subclass events too

What NOT to say: - "Spring events are like Kafka" — same conceptual model but in-process only, not durable - "Use static observers" — fragile, hard to test, no DI

Common follow-up: "What's the difference between `@EventListener` and `@TransactionalEventListener`?"

Q183 · Template method — real-world use

Asked at: Senior rounds at TCS Digital · Cognizant · Wells Fargo

Difficulty: Medium · **Topic:** Design patterns

The question: "Where do you see Template Method pattern in Spring / JDK?"

The 30-second answer: Spring's `JdbcTemplate.execute(...)` — the template handles connection acquisition, exception translation, and cleanup; you fill in the SQL bit via a callback. JDK's `AbstractList`, `AbstractMap` — concrete subclasses fill in the minimum, base class provides everything else. The pattern: invariant scaffolding in the base, variable steps as abstract / protected hooks.

Step-by-step thought process: 1. "`JdbcTemplate.query(sql, rowMapper)` — `JdbcTemplate` handles getting a connection from the pool, preparing the statement, looping the `ResultSet`, closing everything. You only write the `RowMapper`." 2. "`AbstractList` implements `iterator`, `containsAll`, `equals`, `hashCode` in terms of `get(int)` and `size()` — subclasses only define those two." 3. "The pattern shape — base class has a `public final` method that orchestrates, calling `protected abstract` steps. Subclasses can't change the orchestration, only the steps." 4. "`final` on the public method is the contract — it's what prevents subclasses from breaking the invariants."

Edge cases to mention: - Template Method is one of the few GoF patterns where inheritance is the right answer, not composition - Functional style — pass the variable behavior as a lambda;

`JdbcTemplate.execute(con -> { ... })` is closer to that style - Don't over-template — if you have one varying step, just pass a callback

What NOT to say: - "Just use the Strategy pattern" — different shape; Strategy is full algorithm replacement, Template fills in steps - "It's the same as Builder" — no, Builder constructs an object; Template runs an algorithm

Common follow-up: "How does `JdbcTemplate` translate `SQLException` into Spring's `DataAccessException` hierarchy?"

Q184 · Visitor pattern — when does it earn its complexity?

Asked at: Senior rounds at Cred · Goldman · Flipkart

Difficulty: Hard · **Topic:** Design patterns

The question: "Visitor is notoriously hated. When is it the right tool?"

The 30-second answer: Visitor when you have a closed hierarchy of types (sealed classes, AST nodes, geometry shapes) and an open set of operations you want to add without modifying the types. Each operation = a Visitor implementation. Without it: `instanceof` chains scattered across the codebase or polluting the types with every operation.

Step-by-step thought process: 1. "Use case — interpreter or AST processor. You have `Expression` types (`Literal` , `BinaryOp` , `Variable`). You want to add evaluation, type-checking, code generation, formatting — without piling them into each `Expression` class." 2. "Each type accepts a visitor: `interface Expression { <R> R accept(Visitor<R> v); }` . Each type implements `accept` by calling the matching visitor method: `v.visitLiteral(this)` ." 3. "Operations become standalone classes: `class Evaluator implements Visitor<Integer>` , `class Formatter implements Visitor<String>` . Add a new op = add a new Visitor." 4. "Java 21+ sealed types + pattern matching switch make Visitor mostly obsolete — switch over a sealed hierarchy is exhaustive at compile time, no Visitor boilerplate."

Edge cases to mention: - Adding a new type requires updating every Visitor (closed-types, open-operations is the trade-off) - Double dispatch — Java has no native double dispatch, Visitor simulates it via the accept-then-callback pattern - Hibernate's HQL parser, Jackson's JsonNode tree, Eclipse JDT AST all use Visitor

What NOT to say: - "Use it for all polymorphism" — over-engineering for non-AST cases - "It's the same as Strategy" — Strategy is one algorithm at a time; Visitor handles many type-specific behaviors at once

Common follow-up: "How do sealed classes and pattern matching switches replace Visitor in Java 21+?"

Q185 · Builder pattern — JDK's `StringBuilder` is misleading

Asked at: Senior rounds at Razorpay · TCS Digital

Difficulty: Medium · **Topic:** Design patterns

The question: "What's the Builder pattern? Show me a real Java example."

The 30-second answer: Builder solves constructors with many parameters — instead of `new Order(a, b, c, d, e, f)` , you get `Order.builder().merchantId(...).amount(...).currency(...).build()` . Each setter returns `this` for chaining. `build()` validates invariants and constructs the immutable result. Lombok's `@Builder` generates the boilerplate.

Step-by-step thought process: 1. "Telescoping constructors get unwieldy with 5+ params. Builder makes the call site readable — you see field names, not positions." 2. "Pattern — inner static `Builder` class with mutable fields, fluent setters returning `this` , `build()` constructs the outer (often immutable) object." 3. " `StringBuilder` is not this pattern — it's an `Appendable` for mutable string construction, not a builder for a target type. Common interview confusion." 4. "With records (Java 16+), the use case shrinks — records compress simple value types. For records with 10+ fields, Builder still beats positional constructor."

Edge cases to mention: - Immutability — Builder fields can be mutable during construction; the built object is typically immutable - Validation in `build()` — throw if required fields missing - Lombok `@Builder` generates the inner class for you; `@SuperBuilder` handles inheritance

What NOT to say: - "StringBuilder is the textbook example" — it's not Builder pattern - "Builders are slow" — one extra object allocation; negligible

Common follow-up: "How would Lombok's @Builder differ from a hand-written one?"

Q186 · Virtual threads — when NOT to use them

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Cred · Walmart Labs

Difficulty: Hard · **Topic:** Java 21+ cutting-edge

The question: "Virtual threads are GA in Java 21. Where do they not fit?"

The 30-second answer: Virtual threads excel at blocking I/O (HTTP servers, JDBC, blocking queues). They do NOT help CPU-bound work — carriers are still bound by core count. Don't use them for ThreadLocal-heavy code (millions of threads × ThreadLocals = OOM). Don't use them with code that pins inside `synchronized` blocks on Java 21-23 (fixed in Java 24 JEP 491).

Step-by-step thought process: 1. "Virtual thread benefit comes from cheap blocking — when one parks, the carrier picks another. CPU-bound work doesn't park, so there's no benefit (slight overhead from continuation machinery)." 2. "ThreadLocal — each virtual thread inherits or copies. Millions of threads × ThreadLocals = heap explosion. Consider `ScopedValue` (still preview Java 21-23) as the replacement." 3. "Pinning — `synchronized` and JNI pin the virtual thread to its carrier on Java 21. If your hot path is synchronized, no benefit until you switch to `ReentrantLock` or Java 24." 4. "Profile first — virtual threads aren't free. The win is for I/O-blocked services with thousands of concurrent connections."

Edge cases to mention: - Frameworks must opt-in — Spring Boot 3.2+ enables virtual threads for Tomcat / Jetty via `spring.threads.virtual.enabled=true` - JDBC drivers: most modern drivers (PostgreSQL, MySQL) work fine; some older ones pin - Don't pool virtual threads — they're cheap; use `Executors.newVirtualThreadPerTaskExecutor()` (thread-per-task)

What NOT to say: - "Virtual threads replace all thread pools" — only blocking-I/O pools - "They're like Goroutines" — similar idea, different implementation; Goroutines are full preemptive, virtual threads are cooperative

Common follow-up: "What's a ScopedValue and why is it the modern alternative to ThreadLocal?"

Q187 · Structured concurrency — what problem does it solve?

Asked at: Senior rounds at Razorpay · Cred · Walmart Labs

Difficulty: Hard · **Topic:** Java 21+ cutting-edge

The question: "What's structured concurrency? Is it production-ready?"

The 30-second answer: Structured concurrency (JEP 480, *still preview* in Java 22-23 — not GA) treats a group of concurrent subtasks as a single unit. If the parent is cancelled, all subtasks are cancelled. If any subtask fails, siblings are cancelled too. `StructuredTaskScope.ShutdownOnFailure` for fan-out-and-fail-fast; `ShutdownOnSuccess` for first-completed-wins.

Step-by-step thought process: 1. "Problem — `CompletableFuture` lets subtasks outlive their parent invisibly. You start three async calls, your method returns, a subtask fails — silent leak." 2. "Structured fix — `try (var scope = new StructuredTaskScope.ShutdownOnFailure()) { var p = scope.fork(() -> paymentApi.fetch()); var u = scope.fork(() -> userApi.fetch()); scope.join(); scope.throwIfFailed(); ... } .`" 3. "When the try block exits, the scope closes — any unfinished subtasks are cancelled. Clean lifecycle, easier reasoning." 4. "Status — preview in Java 21, 22, 23. Don't deploy to production until GA. Watch JEPs 462 / 480 / 499 for the GA target."

Edge cases to mention: - Combines best with virtual threads — both are about making concurrency cheap and bounded - Custom `StructuredTaskScope` subclasses for app-specific aggregation semantics - Java agent / IDE / dev-mode preview features need `--enable-preview` flag

What NOT to say: - "Use it in prod now" — preview means API can change without notice; risky for production - "Replaces `CompletableFuture`" — overlap, but CF is broader (callbacks, composition without scope semantics)

Common follow-up: "How does `StructuredTaskScope.ShutdownOnSuccess` work?"

Q188 · Scoped values — alternative to `ThreadLocal`

Asked at: Senior rounds at Razorpay · Cred · Walmart Labs

Difficulty: Hard · **Topic:** Java 21+ cutting-edge

The question: "What's `ScopedValue` and why is it relevant in the virtual threads world?"

The 30-second answer: `ScopedValue` (JEP 446, *still preview* in Java 21-23, expected GA target Java 25) is a scoped, immutable alternative to `ThreadLocal`. Set via `ScopedValue.where(VAL, value).run(...)` — the value is visible to that runnable and its child threads, and is gone when the scope exits. Cheaper than `ThreadLocal` for virtual threads (no per-thread map allocation).

Step-by-step thought process: 1. "`ThreadLocal` allocates a map per thread to hold values. With millions of virtual threads, the cost adds up." 2. "`ScopedValue` stores values in the call stack frame instead. Read is via the binding active in the current scope — $O(\text{stack depth})$ lookup, no per-thread storage." 3. "`ScopedValue<UserId> USER = ScopedValue.newInstance(); ScopedValue.where(USER, currentUserId).run(() -> service.process());` — `USER.get()` inside `process` returns the bound value." 4. "Immutable — you can't `set()` a `ScopedValue` from within the scope. Only the binding-establishing call can do it. Eliminates mutation bugs that `ThreadLocal` allows."

Edge cases to mention: - Inheritance to child threads is opt-in via `Scope.where(...)` semantics — virtual threads forked inside the scope inherit - Preview status — same `--enable-preview` caveat - Use case sweet spot — request-scoped context (current user, trace ID, locale) passed through a deep call stack

What NOT to say: - "ScopedValue is GA" — still preview as of Java 23 - "It replaces ThreadLocal everywhere" — ThreadLocal-with-mutation still has its place

Common follow-up: "Why does ThreadLocal hurt under virtual threads?"

Q189 · Foreign function and memory API — replacing JNI

Asked at: Senior rounds at Goldman · Walmart Labs · Cred

Difficulty: Hard · **Topic:** Java 21+ cutting-edge

The question: "What's the Foreign Function and Memory API? Why is it a big deal?"

The 30-second answer: FFM (JEP 442, GA in Java 22) replaces JNI for calling native code from Java. No C glue code required — describe the function signature in Java, JVM generates the trampoline. Also gives you `MemorySegment` for off-heap memory with explicit lifetime, much safer than `Unsafe`.

Step-by-step thought process: 1. "JNI for 25 years has been the way to call C from Java — required hand-written .c files, JNIEXPORT functions, painful build setup." 2. "FFM — `Linker linker = Linker.nativeLinker(); MemorySegment strlen = linker.defaultLookup().find("strlen").get(); MethodHandle h = linker.downcallHandle(strlen, FunctionDescriptor.of(ValueLayout.JAVA_LONG, ValueLayout.ADDRESS));` . Pure Java." 3. "MemorySegment + Arena — replace Unsafe and DirectByteBuffer for off-heap memory. Arena gives you explicit free semantics — `try (Arena arena = Arena.ofConfined()) { ... }` ." 4. "Use cases — calling LMDB, OpenSSL, native ML libraries (LLaMA, Whisper) from Java without JNI overhead."

Edge cases to mention: - Confined vs shared arenas — confined is single-thread, shared allows multi-thread access at slight overhead - Memory layout API — describe native struct layouts in Java, FFM handles padding and field offsets - Project Panama is the umbrella project; FFM is one deliverable

What NOT to say: - "Still preview" — GA in Java 22, fully production-ready by Java 23 - "Just like JNI" — fundamentally simpler; no glue code

Common follow-up: "What's the difference between Arena.ofConfined and Arena.ofShared?"

Q190 · Vector API — SIMD from Java

Asked at: Staff rounds at Goldman · Flipkart · Walmart Labs

Difficulty: Hard · **Topic:** Java 21+ cutting-edge

The question: "What's the Vector API and what's its status?"

The 30-second answer: Vector API (JEP 469 in Java 23 — *still incubator*, not finalized) exposes CPU SIMD instructions (AVX, NEON) to Java code in a portable way. Write vectorized loops in Java that compile to wide vector instructions. Big wins for numeric workloads — image processing, ML, finance.

Step-by-step thought process: 1. "Without Vector API, the JIT auto-vectorizes only simple loops — anything non-trivial stays scalar. Vector API gives explicit control." 2. "Define a species: `VectorSpecies<Float> SPECIES = FloatVector.SPECIES_PREFERRED;` . Loop: load N floats, do the op, store. JIT lowers to AVX-512 on supported chips, falls back to scalar elsewhere." 3. "Status — incubator since Java 16, still incubator in Java 23. API stable in spirit, finalization waits on Valhalla (value types)." 4. "Use cases — workloads where you'd reach for C / Rust / GPU for vectorization. Now Java can compete on the same workloads at near-native throughput."

Edge cases to mention: - Hardware-specific — `SPECIES_PREFERRED` adapts at runtime to the JVM's available SIMD width - Don't reach for it unless you've measured a scalar bottleneck — adds complexity for routine code - Combines well with FFM for hybrid hot loops

What NOT to say: - "It's a numerical library" — it's an SIMD API; libraries built on top would be the numerical layer - "GA in Java 21" — still incubator

Common follow-up: "Why does the Vector API finalization wait on Valhalla?"

Q191 · Pattern matching for switch — exhaustiveness

Asked at: Senior rounds at Razorpay · Cred · Swiggy · PhonePe

Difficulty: Medium-Hard · **Topic:** Java 21+ cutting-edge

The question: "Pattern matching switch went GA in Java 21. What's it actually buy you?"

The 30-second answer: Switch can pattern-match on type (`case Integer i -> ...`), with guards (`case Integer i when i > 0 -> ...`), and on sealed hierarchies the compiler verifies exhaustiveness — no `default` needed if you cover every subtype. Combines with records for destructuring (`case Order(var id, var amount) -> ...`).

Step-by-step thought process: 1. "Old switch — limited to constants of `int`, `enum`, `String` . New switch (Java 21 GA) — patterns over any reference type." 2. " `switch (event) { case OrderPlaced o -> handle(o); case PaymentReceived p -> handle(p); case OrderCancelled c -> handle(c); }` — if `Event` is sealed and these are all its subtypes, compiler accepts (no default)." 3. "Guards — `case Integer i when i > 0 -> ...; case Integer i -> ...` (the second catches zero/negative)." 4. "Record deconstruction — `case Order(long id, BigDecimal amount, String currency) -> processOrder(id, amount, currency)` . Cleaner than nested ifs and casts."

Edge cases to mention: - `null` case — explicit `case null -> ...` or it goes to default (NPE in old switch); explicit handling is recommended - Pattern-matching switch is an expression (returns a value) or a statement; both work - Sealed + pattern switch replaces most Visitor pattern uses

What NOT to say: - "Still preview" — GA in Java 21 - "It's just type switch" — adds guards, record deconstruction, exhaustiveness checking — much more

Common follow-up: "How does pattern matching switch combined with sealed classes replace the Visitor pattern?"

Q192 · JUnit 5 — what's new vs JUnit 4

Asked at: Senior rounds at Razorpay · TCS Digital · Swiggy · Wells Fargo

Difficulty: Medium · **Topic:** Testing / quality

The question: "What changed from JUnit 4 to JUnit 5?"

The 30-second answer: New annotations (`@Test` from `org.junit.jupiter.api` , `@BeforeEach` / `@AfterEach` replace `@Before` / `@After` , `@BeforeAll` / `@AfterAll` for static-like setup). Native parameterized tests (`@ParameterizedTest` + `@ValueSource` / `@CsvSource` / `@MethodSource`). Nested test classes (`@Nested`). Test lifecycle modes (per-method default, per-class opt-in). Extensions replace Rules / Runners.

Step-by-step thought process: 1. "Modular — JUnit Platform (runner), JUnit Jupiter (programming model), JUnit Vintage (JUnit 4 compatibility). Run JUnit 4 tests alongside 5 via Vintage." 2.

"Parameterized — `@ParameterizedTest @ValueSource(ints = {1, 2, 3}) void test(int x)` — generates one test per value. No external runner needed." 3. "Extensions — `@ExtendWith(MockitoExtension.class)` , `@ExtendWith(SpringExtension.class)` . Single composition mechanism replaces JUnit 4's `@Rule` and `@RunWith` ." 4. " `assertAll(...)` groups multiple assertions — all run even if some fail, all failures reported together. Better than chains of `assertEquals` that stop at first failure."

Edge cases to mention: - `@DisplayName("...")` for human-readable test names in reports - `@TestInstance(Lifecycle.PER_CLASS)` — instance reused across tests; lets `@BeforeAll` / `@AfterAll` be non-static - `Assertions.assertThrows(IOException.class, () -> ...)` — replaces `@Test(expected = ...)` from JUnit 4

What NOT to say: - "Just like JUnit 4" — different module structure, different extension model - "JUnit 4 is deprecated" — not deprecated; Vintage runs it indefinitely

Common follow-up: "How would you write a custom extension for setting up a TestContainers database?"

Q193 · Mockito argument captors and BDD style

Asked at: Senior rounds at Razorpay · Swiggy · TCS Digital

Difficulty: Medium · **Topic:** Testing / quality

The question: "How do you verify a mock was called with a specific argument shape?"

The 30-second answer: `ArgumentCaptor<Order>.forClass(Order.class)` . Use it in `verify(repo).save(captor.capture())` , then `assertThat(captor.getValue().getAmount()).isEqualTo(...)` . For BDD style — `given(...).willReturn(...)` reads better than `when(...).thenReturn(...)` for some teams.

Step-by-step thought process: 1. "For simple cases — `verify(emailService).send(eq("user@x.com"), anyString())` . Matchers like `eq` , `any` , `anyString` ." 2. "For complex object verification — `ArgumentCaptor<Order> captor = ArgumentCaptor.forClass(Order.class); verify(repo).save(captor.capture()); assertThat(captor.getValue().getStatus()).isEqualTo(PAID);` . Inspect any field." 3. "BDD style — `given(repo.findById(1L)).willReturn(Optional.of(order));` and `then(emailService).should().send(...)` . Same mechanics, more natural for Given-When-Then." 4. " `ArgumentMatchers.argThat(o -> o.getAmount().compareTo(...) > 0)` — inline custom predicate when captor is overkill."

Edge cases to mention: - Mixing matchers and literal values — if any argument uses a matcher, all must. `verify(repo).save(eq(orderObj))` not `verify(repo).save(orderObj)` when other args have matchers - Strict stubbing (Mockito 3+) fails the test if you stub something never called — catches stale tests - `@InjectMocks` constructs the SUT with mocks injected — handy with `@Mock` field annotations

What NOT to say: - "Just use raw assertions on the captured object" — yes, that's the workflow - "Mockito can mock final classes? No, you need PowerMock" — outdated; Mockito 2+ supports finals via the mock-maker-inline config

Common follow-up: "What's strict stubbing and why is it the default in modern Mockito?"

Q194 · TestContainers — Docker for integration tests

Asked at: Senior rounds at Razorpay · Swiggy · PhonePe · Cred · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Testing / quality

The question: "How do you test code that hits a real Postgres without slow CI?"

The 30-second answer: TestContainers — Java library that starts Docker containers on demand for tests. `@Container PostgreSQLContainer<?> postgres = new`

`PostgreSQLContainer<>("postgres:16");` — JUnit lifecycle starts and stops it. Get JDBC URL via `postgres.getJdbcUrl()`. Real Postgres, ephemeral, isolated per test class.

Step-by-step thought process: 1. "H2 in-memory was the old answer — fake DB. Diverges from real Postgres in many ways (SQL dialects, transaction behavior, lock semantics)." 2. "TestContainers spins up the real DB image as a Docker container. Tests get a real Postgres, MySQL, Kafka, Redis — whatever you're using in prod." 3. "Spring Boot 3.1+ has first-class integration — `@ServiceConnection` on the container bean auto-wires the connection details into Spring's DataSource." 4. "Reuse — `withReuse(true)` keeps the container alive across test runs (locally only), big speedup for dev iteration."

Edge cases to mention: - CI runners need Docker-in-Docker or Docker socket access — most modern CI (GitHub Actions, GitLab CI) supports it - Startup time per container is 2-10s — group tests in one class to reuse, or use class-level container with cleanup between tests - TestContainers Cloud and Desktop reduce local Docker overhead

What NOT to say: - "Mock the database" — for repository tests, you want the real DB behavior - "Use H2 for everything" — diverges from production DB in subtle ways that bite

Common follow-up: "How do you reset state between tests when sharing a container?"

Q195 · Mutation testing — what is it and when to use

Asked at: Senior rounds at Cred · Goldman · Walmart Labs

Difficulty: Hard · **Topic:** Testing / quality

The question: "Code coverage is 95%. Is that good?"

The 30-second answer: Coverage measures *executed* code, not *tested* code — a test can execute a line without asserting anything meaningful. Mutation testing (PITest in Java) injects deliberate bugs into your code; if tests still pass, the test suite doesn't actually verify the mutated behavior. Higher mutation score = stronger test suite.

Step-by-step thought process: 1. "Coverage can be high while tests are weak. `assertNotNull(service.compute())` covers every line but proves almost nothing." 2. "PITest mutates your code — flips operators (`>` to `>=`), removes return statements, swaps constants. Runs your test suite per mutation." 3. "If a mutation survives (tests still pass), that's a gap. The mutation didn't change behavior in any way your tests check." 4. "Run mutation testing periodically (slow), not per-commit. It's a quality audit, not a CI gate."

Edge cases to mention: - Equivalent mutations — some mutations don't change behavior at all (e.g., dead code paths). Mutation score accounts for these - PITest integrates with Maven and Gradle; run on changed files via `--features=+CHANGED` for faster local runs - Combine with line coverage — high coverage + high mutation score = strong suite; high coverage + low mutation score = empty tests

What NOT to say: - "100% coverage means perfect tests" — false; coverage is necessary, not sufficient
- "Mutation testing is too slow" — yes for CI, run nightly or on PR

Common follow-up: *"What's an equivalent mutation and how do tools handle them?"*

Q196 · Gradle vs Maven — pick a build tool

Asked at: Senior rounds at Razorpay · Swiggy · Cred · Walmart Labs

Difficulty: Medium · **Topic:** Build / tooling

The question: *"You're starting a new Java service. Maven or Gradle? Why?"*

The 30-second answer: Maven — XML, declarative, larger plugin ecosystem, slower builds, easier for newcomers. Gradle — Groovy/Kotlin DSL, programmatic, much faster (incremental + build cache + daemon), used by Android and modern OSS. For greenfield Spring Boot services in 2026, Gradle (with Kotlin DSL) is the modern pick; for legacy enterprises, Maven dominance is real.

Step-by-step thought process: 1. "Maven — `pom.xml`, lifecycle phases (`validate`, `compile`, `test`, `package`, `install`, `deploy`), convention over configuration. Easy to read, hard to customize." 2. "Gradle — `build.gradle.kts` (Kotlin DSL preferred over Groovy), task-based DAG. Incremental compilation, build cache, daemon — typically 2-5x faster than Maven for full builds, 10x for incrementals." 3. "Plugin ecosystem — Maven has more, Gradle catching up. Spring Boot, Kotlin, Android all first-class on Gradle." 4. "Switching cost — Maven config is more portable; Gradle DSL is more powerful but harder to read for newcomers. Pick what your team can maintain."

Edge cases to mention: - Gradle's dependency resolution can produce different versions than Maven's "nearest wins" — read the resolution rules - Maven Wrapper (`mvnw`) and Gradle Wrapper (`gradlew`) — both let CI use a pinned tool version without global install; always use the wrapper in repo - Multi-module — Gradle composite builds vs Maven reactor; Gradle handles cross-module incremental better

What NOT to say: - "Gradle is always better" — Maven still dominates enterprise; team familiarity matters - "Maven is dead" — no, deeply entrenched in Apache, Spring ecosystem

Common follow-up: *"What's the Gradle build cache and how does it work?"*

Q197 · Dependency convergence and conflicts

Asked at: Senior rounds at Razorpay · Swiggy · TCS Digital · Wells Fargo

Difficulty: Medium-Hard · **Topic:** Build / tooling

The question: " `NoSuchMethodError` at runtime that wasn't there yesterday. How do you debug?"

The 30-second answer: Transitive dependency conflict — you compile against Jackson 2.15 but a different library brought in Jackson 2.13 that won. Diagnose: `mvn dependency:tree` or `gradle`

`dependencies` . Force the version via `<dependencyManagement>` (Maven) or `constraints` (Gradle). Spring Boot's BOM (`spring-boot-dependencies`) is the easy answer in Spring projects.

Step-by-step thought process: 1. "The cause — Java classpath has one class per FQN. Two versions on the path, classloader picks one based on order. `NoSuchMethodError` happens when the chosen version doesn't have the method your code expects." 2. " `mvn dependency:tree -Dincludes=com.fasterxml.jackson.core` — shows the tree filtered to Jackson. Look for multiple versions appearing." 3. "Resolution — Maven uses nearest-wins (shortest path from root). Gradle uses highest-version-wins by default. Both can be overridden." 4. "Best practice — use a BOM (Bill of Materials). Spring Boot's BOM aligns all Spring + commonly-used deps to compatible versions. Just `<scope>import</scope>` it or use `platform(...)` in Gradle."

Edge cases to mention: - `mvn enforcer:enforce` with `dependencyConvergence` rule fails build on conflicts — opinionated but catches issues early - Shading (`maven-shade-plugin`) repackages dependencies under different package names to avoid conflicts; common for libraries like AWS SDK - Java module system can also catch some issues at module load time, but classpath mode is what most apps run in

What NOT to say: - "Exclude all transitive dependencies" — fragile; you'll miss something - "Use the latest version of everything" — incompatible upgrades happen; pin via BOM

Common follow-up: "What's a BOM and why does Spring Boot ship one?"

Q198 · Multi-module project structure

Asked at: Senior rounds at Razorpay · Swiggy · Wells Fargo · TCS Digital

Difficulty: Medium · **Topic:** Build / tooling

The question: "How would you structure a multi-module Maven / Gradle project?"

The 30-second answer: One root with shared config. Modules split by *bounded context* or *layer* — `core` (domain models), `service` (business logic), `web` (controllers), `infra` (external integrations). Avoid `common` modules that grow into kitchen-sink dependencies. Spring Boot 3 + Gradle multi-project is a clean default.

Step-by-step thought process: 1. "Maven — `<modules>` in parent pom. Each module is a directory with its own pom inheriting from parent. Build via `mvn install` at root." 2. "Gradle — `settings.gradle.kts` lists `include(":core", ":service", ":web")` . Modules depend on each other via `implementation(project(":core"))` ." 3. "Module split — by bounded context (orders, payments, inventory) for large services, by layer (domain / app / infra) for hexagonal architecture. Avoid mixing styles." 4. "Single deployment artifact — Spring Boot module produces the executable jar, depends on all the others. Other modules are libraries with no main class."

Edge cases to mention: - Cyclic dependencies — both Maven and Gradle reject them. If you have one, your module split is wrong - Test source dependencies — Gradle `testFixtures` lets one module's

test code reuse fixtures from another; Maven needs `jar` + `test-jar` packaging - Version alignment — define versions in root `dependencyManagement` (Maven) or `gradle.properties` (Gradle), inherit everywhere

What NOT to say: - "Put everything in one module for simplicity" — works to ~50k LOC, painful beyond - "More modules is always better" — over-modularization adds build overhead without benefit

Common follow-up: "How would you avoid a 'common' module turning into a dumping ground?"

Q199 · JLink and JPackage — bundling a runtime

Asked at: Senior rounds at Razorpay · Goldman · Walmart Labs

Difficulty: Medium-Hard · **Topic:** Build / tooling

The question: "How do you ship a Java app that doesn't require the user to install a JDK?"

The 30-second answer: `jlink` (Java 9+) creates a custom JRE containing only the modules your app uses — typically 30-50MB instead of 200MB+ full JDK. `jpackage` (Java 14+) wraps that into an OS-native installer (.msi, .dmg, .deb, .rpm) with launcher script and app icon. Alternative: GraalVM Native Image for a single static binary.

Step-by-step thought process: 1. "`jdeps --print-module-deps myapp.jar` — lists the JDK modules your app uses. Maybe 30, out of ~80 in the full JDK." 2. "`jlink --module-path $JAVA_HOME/jmods --add-modules <list> --output myruntime` — builds a stripped JRE. Distribute alongside your app." 3. "`jpackage --type msi --name MyApp --input app/ --main-jar myapp.jar --runtime-image myruntime` — wraps it into a Windows installer. Same flags for `dmg`, `deb`, `rpm`, etc." 4. "For server-side, container images (Docker `eclipse-temurin:21-jre-jammy` base) are easier. JLink/JPackage are about desktop / CLI tools targeting end users."

Edge cases to mention: - Reflection / dynamic class loading breaks JLink — if your app loads classes by name not in `jdeps` output, JLink misses them. Use `--add-modules` to force - Cross-platform — JLink must run on the target OS (you can't build a Windows runtime from Linux without a Windows toolchain). JPackage similarly - Native Image alternative — single static binary, even smaller, but loses some dynamic Java features

What NOT to say: - "Just bundle the full JDK" — wasteful (10x size), increases attack surface - "JLink works for any jar" — non-modular jars (no `module-info.class`) work as automatic modules, but reflection-heavy ones may break

Common follow-up: "What's the difference between JLink + JPackage and GraalVM Native Image?"

Q200 · Concurrency final boss — design a thread-safe cache with TTL

Asked at: Staff rounds at Razorpay · Swiggy · Cred · Flipkart · Walmart Labs · Goldman

Difficulty: Hard · **Topic:** Concurrency advanced

The question: "Design a thread-safe in-memory cache with per-entry TTL. Walk me through your design choices."

The 30-second answer: Use `ConcurrentHashMap<K, CacheEntry<V>>` where `CacheEntry` holds value + expiry timestamp. Read path: get entry, check expiry, return value or null. Write path: `put` with current time + ttl. Cleanup: lazy on read + a scheduled task to evict expired keys. Honest answer: just use Caffeine — it solves all of this and more.

Step-by-step thought process: 1. "Naive — `Map<K, V> + Map<K, expiryTimestamp>`. Two maps means atomicity problems; combine into one entry class." 2. "record `CacheEntry<V>(V value, long expiresAt)`. Map is `ConcurrentHashMap<K, CacheEntry<V>>`. Get: lookup, check `expiresAt > now`, return value or evict + return null." 3. "Eviction policy — purely lazy (only on read) lets stale entries pile up if rarely read. Background `ScheduledExecutorService.scheduleAtFixedRate(this::evictExpired, 1, 1, MINUTES)` walks and removes." 4. "Honest answer in an interview — Caffeine is the production answer. It does W-TinyLFU eviction, size-based and time-based, with no allocation in the hot path. Mention it explicitly."

Edge cases to mention: - Read-through cache pattern — on miss, call a loader function, store the result. Caffeine's `LoadingCache` handles concurrent loads for the same key (only one loader runs) - Refresh-ahead — when entry is close to expiry, refresh proactively to keep cache warm. Caffeine `refreshAfterWrite` - Memory bound — without max size, cache leaks. Always set both `expireAfterWrite` and `maximumSize`

What NOT to say: - "Use `Hashtable`" — synchronized on every op, terrible under concurrency - "Use `synchronized` blocks" — works but loses `ConcurrentHashMap`'s segment-free concurrency

Common follow-up: "Why is Caffeine's eviction algorithm (W-TinyLFU) better than LRU for typical workloads?"